

CURSO DE CAPACITAÇÃO

SQL SERVER

Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR)

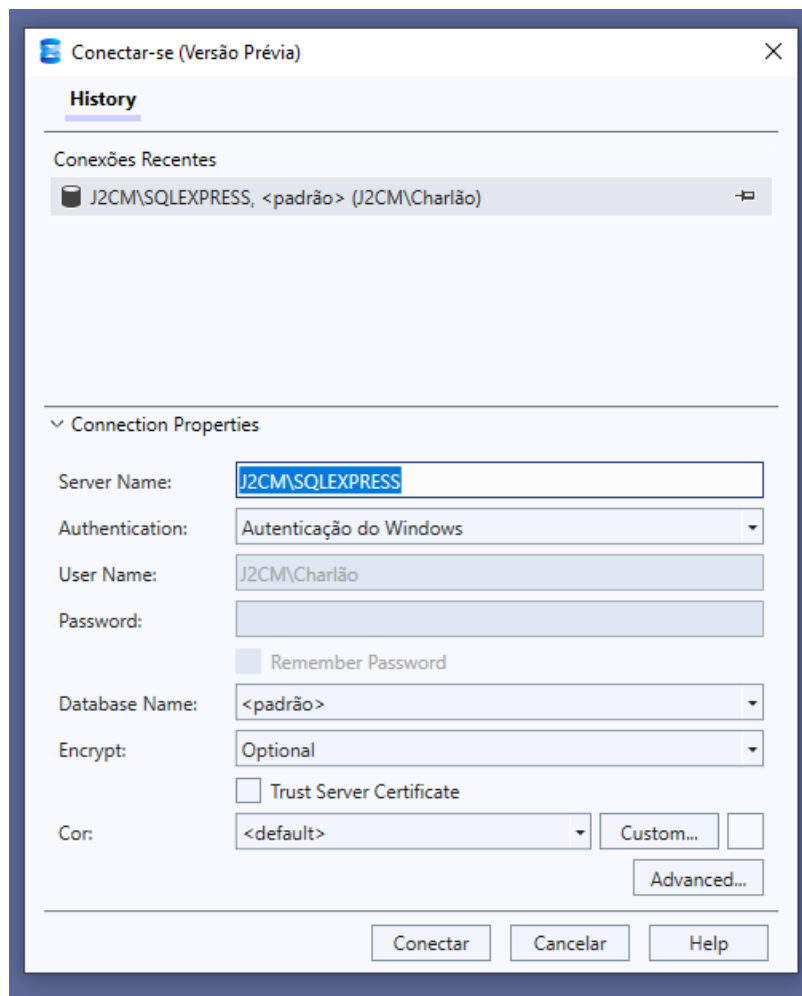
Módulo Básico ao Intermediário

Instrutor: José Charles Curvina de Menezes
Auxiliar Tecnológico / Técnico de Informática

Carga Horária: 40 horas
Local: Laboratório de Informática – J2CM
Data: Agosto de 2025

ROTEIRO DO CURSO DE SQL SERVER – J2CM

- 1º – CRIE UM REPOSITÓRIO NO GITHUB.COM
- 2º – COPIAR A URL PARA CRIAÇÃO DO REPOSITÓRIO LOCAL
 - CÓDIGO
 - Copiar a URL para criação do Repositório LOCAL
 - <https://github.com/jcharlescm/Curso-SQL.git>
- 3º – ABRIR A PASTA DO REPOSITÓRIO LOCAL COM O VISUAL CODE – VSCODE
 - Vá até a pasta criada para o repositório local com o Windows Explorer
 - Selecione a barra de endereço e digite \$ CMD
 - Depois digite \$ CODE . (abrirar o VSCODE)
- 4º – AO ABRIR O VSCODE
 - Clicar em Terminal / Novo Terminal
 - Ao abrir o terminal no lado direito na seta(+V) para abrir o Terminal bash
 - Clique em bash e abrirá o terminal \$
 - \$ git clone <https://github.com/jcharlescm/Curso-SQL.git>
- 5º – INSTALAR SQL SERVER
 - <https://www.microsoft.com/pt-br/sql-server/sql-server-downloads>
 - SQL Server 2022 Express
 - SQL2022-SSEI-Expr.exe
 - Instalar o SSMS – SQL Server Management Studio
 - Obs.: Não vincular o SSMS a uma conta da Microsoft ou GITHUB
- 6º – Configurar o SSMS
 - Quando abrir o SSMS(executar como ADMINISTRADOR) o nome da máquina e servidor.Ex.: SICTEC-0041\SQLEXPRESS ---->>> nome da minha máquina e o nome do meu servidor Windows(fictício)
- Selecionar OPCIONAL



7º - RESTAURAR BANCO DE DADOS

- Clicar com o botão direito em "BANCOS DE DADOS / RESTAURAR BANCO DE DADOS (O BD já está criado é o CONTOSO da microsoft)
- Clicar em DISPOSITIVO e ... (3 pontos) e procurar/adicionar o arquivo do BD CONTOSO (ContosRetailDW.bak)
- OK - Banco de dados CONTOSORETAILDW restaurado com sucesso.

Material de Pesquisa:

<https://learn.microsoft.com/pt-br/sql/sql-server/?view=sql-server-ver17>

/*

MÓDULO 7: EXERCÍCIOS

Os Joins têm como principal objetivo deixar as tabelas mais informativas, permitindo buscar dados de uma tabela para outra, de acordo com uma coluna em comum que permita essa relação.

Dito isso, faça os seguintes exercícios:

1. Utilize o INNER JOIN para trazer os nomes das subcategorias dos produtos, da tabela DimProductSubcategory para a tabela DimProduct.

```
*/
SELECT top(10) * FROM DimProduct
SELECT top(10) * FROM DimProductSubcategory

SELECT
    ProductKey AS 'ID Produto',
    ProductName AS 'Produto',
    ProductSubcategoryName AS 'SubCategoria'
FROM DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
/*
```

2. Identifique uma coluna em comum entre as tabelas DimProductSubcategory e DimProductCategory. Utilize essa coluna para complementar informações na tabela DimProductSubcategory a partir da DimProductCategory. Utilize o LEFT JOIN.

```
*/
SELECT top(10) * FROM DimProductSubcategory
SELECT top(10) * FROM DimProductCategory

SELECT
    ProductSubcategoryKey AS 'ID SubCategoria',
    ProductSubcategoryName AS 'SubCategoria',
    ProductCategoryName AS 'Categoria'
FROM
    DimProductSubcategory
LEFT JOIN DimProductCategory
    ON DimProductSubcategory.ProductCategoryKey =
DimProductCategory.ProductCategoryKey
/*
```

3. Para cada loja da tabela DimStore, descubra qual o Continente e o Nome do País associados (de acordo com DimGeography). Seu SELECT final deve conter apenas as seguintes colunas: StoreKey, StoreName, EmployeeCount, ContinentName e RegionCountryName. Utilize o LEFT JOIN neste exercício.

```
*/
SELECT top(10) * FROM DimStore
SELECT top(10) * FROM DimGeography
```

```

SELECT
    StoreKey AS 'ID Loja',
    StoreName AS 'Loja',
    EmployeeCount AS 'QT. Funcionarios',
    ContinentName AS 'Continente',
    RegionCountryName AS 'Pais'
FROM
    DimStore
LEFT JOIN DimGeography
    ON DimStore.GeographyKey = DimGeography.GeographyKey
/*

```

4. Complementa a tabela DimProduct com a informação de ProductCategoryDescription. Utilize o LEFT JOIN e retorne em seu SELECT apenas as 5 colunas que considerar mais relevantes.

```

*/
SELECT top(10) * FROM DimProduct
SELECT top(10) * FROM DimProductCategory

SELECT
    ProductName,
    ProductCategoryDescription
FROM
    DimProduct
LEFT JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
    LEFT JOIN DimProductCategory
        on DimProductSubcategory.ProductCategoryKey =
DimProductCategory.ProductCategoryKey
/*

```

5. A tabela FactStrategyPlan resume o planejamento estratégico da empresa. Cada linha representa um montante destinado a uma determinada AccountKey.
a) Faça um SELECT das 100 primeiras linhas de FactStrategyPlan para reconhecer a tabela.

```

*/
SELECT TOP(100) * FROM FactStrategyPlan
/*

```

b) Faça um INNER JOIN para criar uma tabela contendo o AccountName para cada AccountKey da tabela FactStrategyPlan. O seu SELECT final deve conter as colunas: StrategyPlanKey, DateKey, AccountName e Amount

```

*/
SELECT
    StrategyPlanKey,
    DateKey,
    AccountName,
    Amount
FROM
    FactStrategyPlan
INNER JOIN DimAccount
    ON FactStrategyPlan.AccountKey = DimAccount.AccountKey
/*

```

6. Vamos continuar analisando a tabela FactStrategyPlan. Além da coluna AccountKey que identifica o tipo de conta, há também uma outra coluna chamada ScenarioKey. Essa coluna possui a numeração que identifica o tipo de cenário: Real, Orçado e Previsão. Faça um INNER JOIN para criar uma tabela contendo o ScenarioName para cada ScenarioKey da tabela FactStrategyPlan. O seu SELECT final deve conter as colunas: StrategyPlanKey, DateKey, ScenarioName, Amount

```

*/
SELECT
    StrategyPlanKey,
    DateKey,
    ScenarioName,
    Amount
FROM
    FactStrategyPlan
INNER JOIN DimScenario
    ON FactStrategyPlan.ScenarioKey = DimScenario.ScenarioKey
/*

```

7. Algumas subcategorias não possuem nenhum exemplar de produto. Identifique que subcategorias são essas.

```

*/
SELECT
    ProductSubcategoryName
FROM
    DimProduct
RIGHT JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
    DimProductSubcategory.ProductSubcategoryKey
WHERE ProductName IS NULL
/*

```

8. A tabela abaixo mostra a combinação entre Marca e Canal de Venda, para as marcas Contoso, Fabrikam e Litware. Crie um código SQL para chegar no mesmo resultado.

```

*/
SELECT DISTINCT BrandName FROM DimProduct

SELECT
    DISTINCT BrandName, ChannelName
FROM
    DimProduct CROSS JOIN DimChannel
WHERE BrandName IN ('Contoso', 'Fabrikam', 'Litware')

/*

```

9. Neste exercício, você deverá relacionar as tabelas FactOnlineSales com DimPromotion. Identifique a coluna que as duas tabelas têm em comum e utilize-a para criar esse relacionamento.

Retorne uma tabela contendo as seguintes colunas: OnlineSalesKey, DateKey, PromotionName e SalesAmount

A sua consulta deve considerar apenas as linhas de vendas referentes a produtos com desconto (PromotionName <> 'No Discount'). Além disso, você deverá ordenar essa tabela de acordo com a coluna DateKey, em ordem crescente.

```

*/
SELECT TOP(1000)
    OnlineSalesKey,
    DateKey,
    PromotionName,
    SalesAmount
FROM
    FactOnlineSales
INNER JOIN DimPromotion
    ON FactOnlineSales.PromotionKey = DimPromotion.PromotionKey
WHERE PromotionName <> 'No Discount'
ORDER BY DateKey ASC
/*

```

10. A tabela abaixo é resultado de um Join entre a tabela FactSales e as tabelas: DimChannel, DimStore e DimProduct.

Recrie esta consulta e classifique em ordem decrescente de acordo com SalesAmount.

```
*/
SELECT TOP(100)
    SalesKey,
    ChannelName,
    StoreName,
    ProductName,
    SalesAmount
FROM
    FactSales
INNER JOIN DimChannel
    ON FactSales.channelKey = DimChannel.ChannelKey
INNER JOIN DimStore
    ON FactSales.StoreKey = DimStore.StoreKey
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
ORDER BY SalesAmount DESC
```

/*

11. GROUP BY + JOIN

1.a) Crie um agrupamento mostrnado o total de venda (SalesQuntity) por ano (CalendarYear)

1.b) Considere apenas o mês (CalendarMonthLabel) de 'January'

1.C) Na tabela resultante, mostre apenas os anos com um total de vendas maior ou igual a 1200000

```
*/
SELECT TOP(10) * FROM FactSales
SELECT TOP(10) * FROM DimDate

SELECT
    CalendarYear AS 'Ano',
    SUM(SalesQuantity) AS 'Total Vendiso'
FROM FactSales
INNER JOIN DimDate
    ON FactSales.DateKey = DimDate.DateKey
WHERE CalendarMonthLabel = 'January'
GROUP BY CalendarYEAR
HAVING SUM(SalesQuantity) >= 1200000
```

```
SELECT
    CalendarYear AS 'Ano',
    SUM(SalesQuantity) AS 'Total Vendiso'
FROM FactSales
INNER JOIN DimDate
    ON FactSales.DateKey = DimDate.DateKey
WHERE CalendarMonthLabel = 'January'
GROUP BY CalendarYEAR
ORDER BY SUM (SalesQuantity) DESC
```

/*

MÓDULO 8: JOINS – EXERCÍCIOS

-- JOINS

-- JOINS – Aula 2:

-- Por que precisamos do JOIN?

```
SELECT TOP(1000) * FROM FactSales
SELECT * FROM DimChannel
```

```

SELECT
    channelKey,
    SUM(SalesQuantity) AS 'Qtd. Vendida'
FROM
    FactSales
GROUP BY channelKey

```

```

SELECT TOP(1000) * FROM FactSales
SELECT * FROM DimProduct

```

```

SELECT TOP(1000) * FROM FactOnlineSales
SELECT * FROM DimCustomer

```

```

SELECT * FROM FactStrategyPlan
SELECT * FROM DimScenario

```

```

-- JOINS - Aula 3:
-- Por que não criamos logo uma tabela com todas as informações pra facilitar?
SELECT TOP(1000) * FROM FactSales
SELECT * FROM DimProduct

```

```

-- JOINS - Aula 6:
-- CÓDIGO PARA CRIAR AS TABELAS AUXILIARES PARA AS EXPLICAÇÕES:
-- PRIMEIRO, GARANTA QUE NÃO EXISTE NENHUMA TABELA CHAMADA PRODUTOS OU SUBCATEGORIA NO
BANCO DE DADOS, EXECUTANDO OS COMANDOS DROP TABLE ABAIXO:
DROP TABLE IF EXISTS produtos
DROP TABLE IF EXISTS subcategoria

```

```

-- Consulta 1:
-- AGORA, CRIE A TABELA PRODUTOS:
CREATE TABLE produtos(
    id_produto int,
    nome_produto varchar(30),
    id_subcategoria int)

```

```

-- INSIRA AS LINHAS DA TABELA PRODUTOS:
INSERT INTO produtos(id_produto, nome_produto, id_subcategoria)
VALUES
    (1, 'Fone Bluetooth JBL', 5),
    (2, 'PS4', 6),
    (3, 'Notebook Samsung i3', 2),
    (4, 'iPhone 11', 1),
    (5, 'Moto G9', 1)

```

```

-- Consulta 2:
-- CRIE A TABELA SUBCATEGORIA:
CREATE TABLE subcategoria(
    id_subcategoria int,
    nome_subcategoria varchar(30))

```

```

-- INSIRA AS LINHAS DA TABELA SUBCATEGORIA:
INSERT INTO subcategoria(id_subcategoria, nome_subcategoria)
VALUES
    (1, 'Celular'),
    (2, 'Notebook'),
    (3, 'Câmera Digital'),
    (4, 'Televisão'),
    (5, 'Fone de Ouvido')

```

```

-- JOINS - Aula 7:

```

-- LEFT (OUTER) JOIN

```
SELECT * FROM produtos
SELECT * FROM subcategoria
```

```
SELECT
    produtos.id_produto,
    produtos.nome_produto,
    produtos.id_subcategoria,
    subcategoria.nome_subcategoria
FROM produtos
LEFT JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

-- JOINS - Aula 8:

-- RIGHT (OUTER) JOIN

```
SELECT * FROM produtos
SELECT * FROM subcategoria
```

```
SELECT
    produtos.id_produto,
    produtos.nome_produto,
    produtos.id_subcategoria,
    subcategoria.nome_subcategoria
FROM produtos
RIGHT JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

-- JOINS - Aula 9:

-- INNER JOIN

```
SELECT * FROM produtos
SELECT * FROM subcategoria
```

```
SELECT
    produtos.id_produto,
    produtos.nome_produto,
    produtos.id_subcategoria,
    subcategoria.nome_subcategoria
FROM produtos
INNER JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

-- JOINS - Aula 10:

-- FULL (OUTER) JOIN

```
SELECT * FROM produtos
SELECT * FROM subcategoria
```

-- Considerando a coluna id_subcategoria da tabela produtos

```
SELECT
    produtos.id_produto,
    produtos.nome_produto,
    produtos.id_subcategoria,
    subcategoria.nome_subcategoria
FROM produtos
FULL JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

-- Considerando a coluna id_subcategoria da tabela subcategoria

```
SELECT
    produtos.id_produto,
    produtos.nome_produto,
    subcategoria.id_subcategoria,
    subcategoria.nome_subcategoria
FROM produtos
FULL JOIN subcategoria
```

```
ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

```
-- JOINS - Aula 11:
```

```
SELECT * FROM produtos  
SELECT * FROM subcategoria
```

```
-- LEFT (ANTI) JOIN
```

```
SELECT  
    produtos.id_produto,  
    produtos.nome_produto,  
    produtos.id_subcategoria,  
    subcategoria.nome_subcategoria  
FROM produtos  
LEFT JOIN subcategoria  
    ON produtos.id_subcategoria = subcategoria.id_subcategoria  
WHERE nome_subcategoria IS NULL
```

```
-- RIGHT (ANTI) JOIN
```

```
SELECT  
    produtos.id_produto,  
    produtos.nome_produto,  
    produtos.id_subcategoria,  
    subcategoria.nome_subcategoria  
FROM produtos  
RIGHT JOIN subcategoria  
    ON produtos.id_subcategoria = subcategoria.id_subcategoria  
WHERE id_produto IS NULL
```

```
-- FULL (ANTI) JOIN
```

```
SELECT  
    produtos.id_produto,  
    produtos.nome_produto,  
    produtos.id_subcategoria,  
    subcategoria.nome_subcategoria  
FROM produtos  
FULL JOIN subcategoria  
    ON produtos.id_subcategoria = subcategoria.id_subcategoria  
WHERE id_produto IS NULL OR nome_subcategoria IS NULL
```

```
-- JOINS Aula 12:
```

```
-- Exemplo INNER JOIN, LEFT JOIN e RIGHT JOIN
```

```
SELECT ProductKey, ProductName, ProductSubcategoryKey FROM DimProduct  
SELECT ProductSubcategoryKey, ProductSubcategoryName FROM DimProductSubcategory
```

```
SELECT  
    ProductKey,  
    ProductName,  
    DimProduct.ProductSubcategoryKey,  
    ProductSubcategoryName  
FROM DimProduct  
INNER JOIN DimProductSubcategory  
    ON DimProduct.ProductSubcategoryKey =  
    DimProductSubcategory.ProductSubcategoryKey
```

```
SELECT  
    ProductKey,  
    ProductName,  
    DimProduct.ProductSubcategoryKey,  
    ProductSubcategoryName  
FROM DimProduct  
LEFT JOIN DimProductSubcategory  
    ON DimProduct.ProductSubcategoryKey =  
    DimProductSubcategory.ProductSubcategoryKey
```

```
SELECT
```

```

        ProductKey,
        ProductName,
        DimProduct.ProductSubcategoryKey,
        ProductSubcategoryName
FROM DimProduct
RIGHT JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey

-- JOINS - Aula 13:
-- COMO ESCOLHER QUEM É A TABELA DA ESQUERDA E A TABELA DA DIREITA?
-- 1. LEFT Join para complementar informações da tabela produtos (LEFT) de acordo com
a tabela de categoria (RIGHT)
SELECT
    id_produto,
    nome_produto,
    produtos.id_subcategoria,
    nome_subcategoria
FROM produtos
LEFT JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria

-- 2. Obtendo o LEFT Join do caso 1 usando um RIGHT Join e invertendo as tabelas de
lado
SELECT
    id_produto,
    nome_produto,
    produtos.id_subcategoria,
    nome_subcategoria
FROM subcategoria
RIGHT JOIN produtos
    ON subcategoria.id_subcategoria = produtos.id_subcategoria

-- JOINS - Aula 15:
-- CROSS JOIN
-- PARA ACOMPANHAR ESTA AULA, PRECISAREMOS CRIAR A TABELA MARCAS
-- PRIMEIRO, GARANTA QUE NÃO EXISTE NENHUMA TABELA CHAMADA MARCAS NO BANCO DE DADOS,
EXECUTANDO O COMANDO DROP TABLE ABAIXO:
DROP TABLE IF EXISTS marcas

-- AGORA CRIE A TABELA MARCAS:
CREATE TABLE marcas(
id_marca int,
marca varchar(30))

-- INSIRA AS LINHAS DA TABELA MARCAS:
INSERT INTO marcas(id_marca, marca)
VALUES
    (1, 'Apple'),
    (2, 'Samsung'),
    (3, 'Motorola')

SELECT * FROM marcas
SELECT * FROM subcategoria

SELECT
    marca,
    nome_subcategoria
FROM marcas CROSS JOIN subcategoria

-- JOINS - Aula 16:
-- Múltiplos Joins
SELECT ProductKey, ProductName, ProductSubcategoryKey FROM DimProduct
SELECT ProductSubcategoryKey, ProductSubcategoryName, ProductCategoryKey FROM
DimProductSubcategory

```

```
SELECT ProductCategoryKey, ProductCategoryName FROM DimProductCategory
```

```
SELECT
    ProductKey,
    ProductName,
    DimProduct.ProductSubcategoryKey,
    ProductCategoryName
FROM DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
    INNER JOIN DimProductCategory
        ON DimProductSubcategory.ProductCategoryKey =
DimProductCategory.ProductCategoryKey
```

```
-- JOINS - Aula 17:
```

```
-- UNION
```

```
SELECT
    *
FROM
    DimCustomer
WHERE Gender = 'F'
```

```
UNION
SELECT
    *
FROM
    DimCustomer
WHERE Gender = 'M'
```

```
-- UNION ALL
```

```
SELECT
    FirstName,
    BirthDate
FROM
    DimCustomer
WHERE Gender = 'F'
UNION ALL
SELECT
```

```
    FirstName,
    BirthDate
FROM
    DimCustomer
WHERE Gender = 'M'
```

```
/*
```

MÓDULO 9: EXERCÍCIOS

Os exercícios abaixo estão divididos de acordo com uma determinada tabela do Banco de Dados. Você pode utilizar o INNER JOIN para resolver todas as questões.

FACT SALES

1.a) Faça um resumo da quantidade vendida (Sales Quantity) de acordo com o nome do canal de vendas (ChannelName). Você deve ordenar a tabela final de acordo com SalesQuantity, em ordem decrescente.

```
*/
```

```
SELECT TOP(10) * FROM FactSales
SELECT TOP(10) * FROM DimChannel
```

```
SELECT
    channelName AS 'Canal de Venda',
    SUM(SalesQuantity) AS 'Total Vendido'
FROM
    FactSales
INNER JOIN DimChannel
    ON FactSales.channelKey = DimChannel.ChannelKey
```

```
GROUP BY ChannelName
ORDER BY SUM(SalesQuantity) DESC
```

```
/*
```

1.b) Faça um agrupamento mostrando a quantidade total vendida (Sales Quantity) e quantidade total devolvida (Return Quantity) de acordo com o nome das lojas (StoreName).

```
*/
```

```
SELECT TOP(10) * FROM FactSales
SELECT TOP(10) * FROM DimStore
```

```
SELECT
    StoreName AS 'Loja',
    SUM(SalesQuantity) AS 'Total Vendido',
    SUM(ReturnQuantity) AS 'Total Devolvido'
FROM
    FactSales
INNER JOIN DimStore
    ON FactSales.StoreKey = DimStore.StoreKey
GROUP BY storeName
ORDER BY storeName
/*
```

1.c) Faça um resumo do valor total vendido (Sales Amount) para cada mês (CalendarMonthLabel) e ano (CalendarYear).

```
*/
```

```
SELECT TOP(10) * FROM FactSales
SELECT TOP(10) * FROM DimDate
```

```
SELECT
    CalendarYear AS 'Ano',
    CalendarMonthLabel AS 'Mês',
    SUM(SalesAmount) AS 'Faturamento Total'
FROM
    FactSales
INNER JOIN DimDate
    ON FactSales.DateKey = DimDate.Datekey
GROUP BY CalendarYear, CalendarMonthLabel, CalendarMonth
ORDER BY CalendarMonth ASC
/*
```

2. Você precisa fazer uma análise de vendas por produtos. O objetivo final é descobrir o valor total vendido (SalesQuantity) por produto.

2.a) Descubra qual é a cor de produto que mais é vendida (de acordo com SalesQuantity).

```
*/
```

```
SELECT TOP(10) * FROM FactSales
SELECT TOP(10) * FROM DimProduct
```

```
SELECT
    ColorName AS 'Cor',
    SUM(SalesQuantity) AS 'Qt. Vendida'
FROM
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
GROUP BY ColorName
ORDER BY SUM(SalesQuantity) DESC
/*
```

2.b) Quantas cores tiveram uma quantidade vendida acima de 3.000.000.

```
*/
```

```
SELECT
    ColorName AS 'Cor',
```

```

SUM(SalesQuantity) AS 'Qt. Vendida'
FROM
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
GROUP BY ColorName
HAVING SUM(SalesQuantity) >= 3000000
ORDER BY SUM(SalesQuantity) DESC
/*

```

3. Crie um agrupamento de quantidade vendida (SalesQuantity) por categoria do produto (ProductCategoryName). Obs: Você precisará fazer mais de 1 INNER JOIN, dado que a relação entre FactSales e DimProductCategory não é direta.

```

FACTONLINEALES
*/
SELECT TOP(10) * FROM FactSales
SELECT TOP(10) * FROM DimProduct
SELECT TOP(10) * FROM DimProductSubcategory
SELECT TOP(10) * FROM DimProductCategory

SELECT
    ProductCategoryName AS 'Categoria',
    SUM(SalesQuantity) AS 'Qt. Vendida'
FROM
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
    INNER JOIN DimProductSubcategory
        ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
    INNER JOIN DimProductCategory
        ON DimProductSubcategory.ProductCategoryKey =
DimProductCategory.ProductCategoryKey
GROUP BY ProductCategoryName
/*

```

4.a) Você deve fazer uma consulta à tabela FactOnlineSales e descobrir qual é o nome completo do cliente que mais realizou compras online (de acordo com a coluna SalesQuantity).

```

*/
SELECT TOP(10) * FROM FactOnlineSales
SELECT TOP(10) * FROM DimCustomer

SELECT
    DimCustomer.CustomerKey,
    FirstName AS 'Nome',
    LastName AS 'Sobrenome',
    SUM(SalesQuantity) AS 'Qtd. Vendida'
FROM
    FactOnlineSales
INNER JOIN DimCustomer
    ON FactOnlineSales.CustomerKey = DimCustomer.CustomerKey
WHERE CustomerType = 'Person'
GROUP BY DimCustomer.CustomerKey, FirstName, LastName
ORDER BY SUM(SalesQuantity) DESC
/*

```

4.b) Feito isso, faça um agrupamento de produtos e descubra quais foram os top 10 produtos mais comprados pelo cliente da letra a, considerando o nome do produto.

```

*/
SELECT TOP(10)
    ProductName AS 'Sobrenome',
    SUM(SalesQuantity) AS 'Qtd. Vendida'
FROM
    FactOnlineSales

```

```

INNER JOIN DimProduct
    ON FactOnlineSales.ProductKey = DimProduct.ProductKey
WHERE CustomerKey = 7665
GROUP BY ProductName
ORDER BY SUM(SalesQuantity) DESC

```

/*

5. Faça um resumo mostrando o total de produtos comprados (Sales Quantity) de acordo com o sexo dos clientes.

FACTEXCHANGERATE

*/

```

SELECT
    Gender AS 'Sexo',
    SUM(SalesQuantity) AS 'Qtd. Vendida'

```

FROM

FactOnlineSales

INNER JOIN DimCustomer

ON FactOnlineSales.CustomerKey = DimCustomer.CustomerKey

WHERE Gender IS NOT NULL -- Sem essa condição vem as empresas

GROUP BY Gender

/*

6. Faça uma tabela resumo mostrando a taxa de câmbio média de acordo com cada CurrencyDescription. A tabela final deve conter apenas taxas entre 10 e 100.

FACTSTRATEGYPLAN

*/

SELECT TOP(10) * FROM FactExchangeRate

SELECT TOP(10) * FROM DimCurrency

SELECT

CurrencyDescription,
AVG(AverageRate) AS 'Taxa Média'

FROM

FactExchangeRate

INNER JOIN DimCurrency

ON FactExchangeRate.CurrencyKey = DimCurrency.CurrencyKey

GROUP BY CurrencyDescription

HAVING AVG(AverageRate) BETWEEN 10 AND 100

/*

7. Calcule a SOMA TOTAL de AMOUNT referente à tabela FactStrategyPlan destinado aos cenários: Actual e Budget.

Dica: A tabela DimScenario será importante para esse exercício.

*/

SELECT TOP(10) * FROM FactStrategyPlan

SELECT TOP(10) * FROM DimScenario

-- Usando <>

SELECT

ScenarioName AS 'Cenario',
SUM(Amount) AS 'Total'

FROM

FactStrategyPlan

INNER JOIN DimScenario

ON FactStrategyPlan.ScenarioKey = DimScenario.ScenarioKey

GROUP BY ScenarioName

HAVING ScenarioName <> 'Forecast'

-- Usando IN

SELECT

ScenarioName AS 'Cenario',
SUM(Amount) AS 'Total'

FROM

FactStrategyPlan

INNER JOIN DimScenario

```

        ON FactStrategyPlan.ScenarioKey = DimScenario.ScenarioKey
GROUP BY ScenarioName
HAVING ScenarioName IN ('Actual', 'Budget')
/*

```

8. Faça uma tabela resumo mostrando o resultado do planejamento estratégico por ano.
DIMPRODUCT/DIMPRODUCTSUBCATEGORY

```

*/
SELECT TOP(10) * FROM FactStrategyPlan
SELECT TOP(10) * FROM DimDate

SELECT
    CalendarYear AS 'Ano',
    SUM(Amount) AS 'Total'
FROM
    FactStrategyPlan
INNER JOIN DimDate
    ON FactStrategyPlan.Datekey = DimDate.Datekey
GROUP BY CalendarYear
/*

```

9. Faça um agrupamento de quantidade de produtos por ProductSubcategoryName. Leve em consideração em sua análise apenas a marca Contoso e a cor Silver.

```

*/
SELECT
    ProductSubcategoryName AS 'SubCategoria',
    COUNT(*) AS 'Qtd. Produtos'
FROM
    DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
GROUP BY ProductSubcategoryName
-- Com filtro WHERE
SELECT
    ProductSubcategoryName AS 'SubCategoria',
    COUNT(*) AS 'Qtd. Produtos'
FROM
    DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
WHERE BrandName = 'Contoso' AND ColorName = 'Silver'
GROUP BY ProductSubcategoryName

/*

```

10. Faça um agrupamento duplo de quantidade de produtos por BrandName e ProductSubcategoryName. A tabela final deverá ser ordenada de acordo com a coluna BrandName.

```

*/
SELECT
    BrandName AS 'Marca',
    ProductSubcategoryName AS 'SubCategoria',
    COUNT(*) AS 'Qtd. Produtos'
FROM
    DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
GROUP BY BrandName, ProductSubcategoryName
ORDER BY BrandName ASC

```

/*

MÓDULO 10: Variáveis

-- [SQL Server] Variáveis
-- Aula 2 de 23: Tipos de Dados

/*

Tipos de dados

O tipo de dado é a maneira como o SQL consegue diferenciar cada valor dentro de um banco de dados.

a) Inteiro

Exemplos: 1, 100, 569

Como o SQL entende um número inteiro: INT

b) Decimal

Exemplos: 10.33, 90.91, 410.787

Como o SQL entende um número decimal: FLOAT

Como o SQL entende um número decimal: DECIMAL(N, M)

N é a quantidade de dígitos que o número pode ter, incluindo casas decimais

M é o número máximo de casas decimais

c) Texto/String

Exemplos: 'Carla', 'Motorola', 'Pastel', '44'

Como o SQL entende um texto: VARCHAR(N)

N é o número de caracteres que o texto pode ter

d) Data

Exemplos: '01/01/2021', '23/03/2021'

Como o SQL entende uma data/hora: DATETIME

Como o SQL entende uma data: DATE

*/

-- Aula 3 de 23: Operações Básicas

-- Operações básicas

SELECT 10 AS 'Número'

SELECT 'Marcus' AS 'Nome'

SELECT '21/06/2021' AS 'Data'

-- Operações com números

SELECT 10+20

SELECT 20-5

SELECT 31*40

SELECT 431.0/23

-- Operações com textos

SELECT 'SQL' + ' ' + 'Impressionador'

-- Operações com datas

SELECT '21/03/2021' + 1

-- Aula 4 de 23: SQL_VARIANT_PROPERTY - Identificando o tipo de um dado

SELECT 10 AS 'Número'

SELECT 49.50 AS 'Decimal'

SELECT 'Marcus' AS 'Nome'

SELECT '20/06/2021' AS 'Data'

SELECT SQL_VARIANT_PROPERTY(10, 'BaseType')

```

SELECT SQL_VARIANT_PROPERTY(49.50, 'BaseType')
SELECT SQL_VARIANT_PROPERTY('Marcus', 'BaseType')
SELECT SQL_VARIANT_PROPERTY('20/06/2021', 'BaseType')
-- Aula 5 de 23: CAST - Especificando o tipo de um dado

-- 1) CAST: Função para especificar o tipo dos valores.
-- int: inteiro, float: decimal, varchar: string/texto, datetime: data e hora
SELECT CAST(21.45 AS int)

SELECT CAST(21.45 AS float)

SELECT CAST(18.7 AS varchar)

SELECT CAST('15.6' AS float)

SELECT CAST('31/05/2014' AS datetime)

-- Exemplo 1: Crie uma consulta juntando o texto 'O preço do produto é: ' com o valor 30.99
SELECT 'O preço do produto é: ' + CAST(30.99 AS VARCHAR(30))

-- Exemplo 2: Adicione 1 dia à data '20/06/2021'
SELECT CAST('20/06/2021' AS DATETIME) + 1

-- Aula 6 de 23: FORMAT - Formatação de dados personalizada

--FORMAT: Função para formatação de valores no SQL
-- a) Numéricos:
SELECT FORMAT(1000, 'N')
SELECT FORMAT(1000, 'G')

-- b) Personalizados:
SELECT FORMAT(123456789, '###-##-####')

-- c) Data:
SELECT FORMAT(CAST('21/03/2021' AS DATETIME), 'dd/MM/yyyy')

-- SQL_VARIANT_PROPERTY
SELECT SQL_VARIANT_PROPERTY('31/05/2014', 'BaseType')
SELECT SQL_VARIANT_PROPERTY(CAST('31/05/2014' AS datetime), 'BaseType')

-- Aula 7 de 23: ROUND, FLOOR e CEILING - Funções de Arredondamento

-- Operações com números
SELECT 10+20
SELECT 20-5
SELECT 31*40
SELECT 431.0/23

-- ROUND
SELECT ROUND(18.739130, 2)

-- ROUND (Truncar)
SELECT ROUND(18.739130, 2, 1)

-- FLOOR
SELECT FLOOR(18.739130)

-- CEILING
SELECT CEILING(18.739130)

-- Aula 8 de 23: DECLARE e SET Declarando uma variável

/*

```

Declarando Variáveis

1) O que é uma variável?

Uma variável é um objeto que armazena o valor de um dado.

2. Estrutura

```
DECLARE @var tipo  
SET @var = valor
```

```
DECLARE @var1 INT, @var2 INT,  
        @texto VARCHAR(MAX),  
        @data DATETIME
```

```
SET @var1 = 10  
SET @var1 = 45  
SET @texto = 'Um texto qualquer'  
SET @data = '18/02/2021'  
*/
```

```
SELECT 100 * 8.99 AS 'faturamento'
```

```
DECLARE @quantidade AS int, @preco AS float  
SET @quantidade = 100  
SET @preco = 8.99
```

```
SELECT @quantidade * @preco AS 'faturamento'  
-- Exemplo 1: Declare uma variável chamada 'idade' e armazene o valor 30
```

```
DECLARE @idade AS INT  
SET @idade = 30  
SELECT @idade
```

```
-- Exemplo 2: Declare uma variável chamada 'preco' e armazene o valor 10.89  
DECLARE @preco AS float  
SET @preco = 10.89  
SELECT @preco
```

```
DECLARE @preco2 AS decimal(5, 2)  
SET @preco2 = 10.89  
SELECT @preco2
```

```
-- Exemplo 3: Declare uma variável chamada 'nome' e armazene o valor 'Mateus'  
DECLARE @nome AS varchar(50)  
SET @nome = 'Mateus'  
SELECT @nome
```

```
-- Exemplo 4: Declare uma variável chamada 'data' e armazene a data de hoje.  
DECLARE @data AS datetime  
SET @data = '2021-10-28'  
SELECT @data
```

```
SELECT DAY(@data)
```

-- Aula 11 de 23: Utilizando uma variável em uma consulta (Parte 1)

-- Aplique um desconto de 10% em todos os preços dos produtos. Sua consulta final deve conter as colunas ProductKey, ProductName, UnitPrice e Preço com Desconto.

```
DECLARE @varDesconto FLOAT  
SET @varDesconto = 0.1
```

```
SELECT  
    ProductKey AS 'ID',  
    ProductName AS 'Nome do Produto',
```

```

        UnitPrice AS 'Preço',
        UnitPrice * (1 - @varDesconto) AS 'Preço com Desconto'
FROM
    DimProduct

```

-- Aula 12 de 23: Utilizando uma variável em uma consulta (Parte 2)

-- Crie uma variável de data para otimizar a consulta abaixo.

```

DECLARE @varData DATETIME
SET @varData = '01/01/1980'

```

```

SELECT
    FirstName AS 'Nome',
    LastName AS 'Sobrenome',
    BirthDate AS 'Nascimento',
    'Cliente' AS 'Tipo'
FROM
    DimCustomer
WHERE BirthDate >= @varData
UNION
SELECT
    FirstName AS 'Nome',
    LastName AS 'Sobrenome',
    BirthDate AS 'Nascimento',
    'Funcionário' AS 'Tipo'
FROM
    DimEmployee
WHERE BirthDate >= @varData
ORDER BY Nascimento

```

-- Aula 13 de 23: Armazenando o resultado de um SELECT em uma variável

-- Exemplo 1: Crie uma variável para armazenar a quantidade total de funcionários da tabela DimEmployee.

```

DECLARE @varTotalFuncionarios INT
SET @varTotalFuncionarios = (SELECT COUNT(*) FROM DimEmployee)
SELECT @varTotalFuncionarios

```

-- Exemplo 2: Crie uma variável para armazenar a quantidade total de lojas com o status Off.

```

DECLARE @varLojasOff INT
SET @varLojasOff = (SELECT COUNT(*) FROM DimStore WHERE Status = 'Off')
SELECT @varLojasOff

```

-- Aula 14 de 23: PRINT - Printando uma mensagem na tela

-- Exemplo 1: Printe na tela a quantidade de lojas On e a quantidade de lojas Off da tabela DimStore. Utilize variáveis para isso.

```

DECLARE @varLojasOn INT, @varLojasOff INT
SET @varLojasOn = (SELECT COUNT(*) FROM DimStore WHERE Status = 'On')
SET @varLojasOff = (SELECT COUNT(*) FROM DimStore WHERE Status = 'Off')

SELECT @varLojasOn AS 'Lojas Abertas', @varLojasOff AS 'Lojas Fechadas'

PRINT 'O total de lojas abertas é de ' + CAST(@varLojasOn AS VARCHAR(30))
PRINT 'O total de lojas fechadas é de ' + CAST(@varLojasOff AS VARCHAR(30))

```

-- Aula 15 de 23: Armazenando em uma variável um registro de uma consulta

-- Exemplo 1: Qual é o nome do produto que teve a maior quantidade vendida EM UMA ÚNICA VENDA da tabela FactSales?

```

DECLARE @varProdutoMaisVendido INT
DECLARE @varTotalMaisVendido INT

```

```

SELECT TOP(1)
    @varProdutoMaisVendido = ProductKey,
    @varTotalMaisVendido = SalesQuantity
FROM
    FactSales
ORDER BY SalesQuantity DESC

PRINT @varProdutoMaisVendido
PRINT @varTotalMaisVendido

SELECT
    ProductKey,
    ProductName
FROM
    DimProduct
WHERE ProductKey = @varProdutoMaisVendido

-- Aula 16 de 23: Acumulando valores dentro de uma variável

-- Exemplo 1: Retorne uma lista com os nomes dos funcionários do departamento de
Marketing
DECLARE @ListaNomes VARCHAR(MAX)
SET @ListaNomes = ''

SELECT
    @ListaNomes = @ListaNomes + FirstName + ', ' + CHAR(10)
FROM
    DimEmployee
WHERE DepartmentName = 'Marketing'

PRINT LEFT(@ListaNomes, DATALENGTH(@ListaNomes) - 3)

-- Aula 17 de 23: Variáveis Globais

SELECT @@SERVERNAME

SELECT @@VERSION

SELECT * FROM DimProduct

SELECT @@ROWCOUNT

```

/*

MÓDULO 11: EXERCÍCIOS

1. O setor de vendas decidiu aplicar um desconto aos produtos de acordo com a sua classe. O percentual aplicado deverá ser de:

Economy -> 5%
Regular -> 7%
Deluxe -> 9%

a) Faça uma consulta à tabela DimProduct que retorne as seguintes colunas: ProductKey, ProductName, e outras duas colunas que deverão retornar o % de Desconto e UnitPrice com desconto.

b) Faça uma adaptação no código para que os % de desconto de 5%, 7% e 9% sejam facilmente modificados (dica: utilize variáveis).

*/

```
SELECT TOP(10) * FROM DimProduct
```

```

DECLARE @varDescEco FLOAT, @varDescReg FLOAT, @varDescDe FLOAT
SET @varDescEco = 0.10
SET @varDescReg = 0.07
SET @varDescDe = 0.09
SELECT
    ProductKey AS 'ID Produto',
    ProductName AS 'Nome Produto',
    ClassName AS 'Classe',
    CASE
        WHEN ClassName = 'Economy' THEN (1 - @varDescEco) * UnitPrice
        WHEN ClassName = 'Regular' THEN (1 - @varDescReg) * UnitPrice
        ELSE (1 - @varDescDe) * UnitPrice
    END AS ' % Desconto'
FROM
    DimProduct

```

/*
2. Você ficou responsável pelo controle de produtos da empresa e deverá fazer uma análise da quantidade de produtos por Marca.

A divisão das marcas em categorias deverá ser a seguinte:

CATEGORIA A: Mais de 500 produtos
CATEGORIA B: Entre 100 e 500 produtos
CATEGORIA C: Menos de 100 produtos
Faça uma consulta à tabela DimProduct e retorne uma tabela com um agrupamento de Total de Produtos por Marca, além da coluna de Categoria, conforme a regra acima.

```

*/
SELECT TOP(10) * FROM DimProduct

SELECT
    BrandName AS 'Marca',
    COUNT(*) AS 'Qtd. Pruduto',
    CASE
        WHEN COUNT(*) >= 500 THEN 'CATEGORIA A'
        WHEN COUNT(*) >= 100 THEN 'CATEGORIA B'
        ELSE 'CATEGORIA C'
    END AS ' Categoria'
FROM
    DimProduct
GROUP BY BrandName

```

/*
3. Será necessário criar uma categorização de cada loja da empresa considerando a quantidade de funcionários de cada uma. A lógica a ser seguida será a lógica abaixo:

EmployeeCount >= 50; 'Acima de 50 funcionários'
EmployeeCount >= 40; 'Entre 40 e 50 funcionários'
EmployeeCount >= 30; 'Entre 30 e 40 funcionários'
EmployeeCount >= 20; 'Entre 20 e 30 funcionários'
EmployeeCount >= 40; 'Entre 10 e 20 funcionários'
Caso contrário: 'Abaixo de 10 funcionários'

Faça uma consulta à tabela DimStore que retorne as seguintes informações: StoreName, EmployeeCount e a coluna de categoria, seguindo a regra acima.

```

*/
SELECT TOP(10) * FROM DimStore

SELECT
    StoreName AS 'Nome da Loja',
    EmployeeCount AS 'Qtd. Funcionários',

```

```

CASE
    WHEN EmployeeCount >= 50 THEN 'Acima de 50 Funcionários'
    WHEN EmployeeCount >= 40 THEN 'Acima de 40 Funcionários'
    WHEN EmployeeCount >= 30 THEN 'Acima de 30 Funcionários'
    WHEN EmployeeCount >= 20 THEN 'Acima de 20 Funcionários'
    WHEN EmployeeCount >= 10 THEN 'Acima de 10 Funcionários'
    ELSE 'Abaixo de 10 Funcionários'
END AS 'Categoria'
FROM
    DimStore

```

/*

4. O setor de logística deverá realizar um transporte de carga dos produtos que estão no depósito

de Seattle para o depósito de Sunnyside.

Não se tem muitas informações sobre os produtos que estão no depósito, apenas se sabe que existem 100 exemplares de cada Subcategoria. Ou seja, 100 laptops, 100 câmeras digitais, 100 ventiladores, e assim vai.

O gerente de logística definiu que os produtos serão transportados por duas rotas distintas. Além

disso, a divisão dos produtos em cada uma das rotas será feita de acordo com as subcategorias (ou

seja, todos os produtos de uma mesma subcategoria serão transportados pela mesma rota):

Rota 1: As subcategorias que tiverem uma soma total menor que 1000 kg deverão ser transportados pela Rota 1.

Rota 2: As subcategorias que tiverem uma soma total maior ou igual a 1000 kg deverão ser transportados pela Rota 2.

Você deverá realizar uma consulta à tabela DimProduct e fazer essa divisão das subcategorias por

cada rota. Algumas dicas:

- Dica 1: A sua consulta deverá ter um total de 3 colunas: Nome da Subcategoria, Peso Total e Rota.

- Dica 2: Como não se sabe quais produtos existem no depósito, apenas que existem 100

exemplares de cada subcategoria, você deverá descobrir o peso médio de cada subcategoria e

multiplicar essa média por 100, de forma que você descubra aproximadamente qual é o peso total

dos produtos por subcategoria.

- Dica 3: Sua resposta final deverá ter um JOIN e um GROUP BY.

*/

```
SELECT TOP(10) * FROM DimProduct
```

```
SELECT
```

```

    ProductSubcategoryName AS 'Nome SubCategoria',
    ROUND(AVG(Weight) * 100, 2) AS 'Peso Total',
    CASE
        WHEN ROUND(AVG(Weight) * 100, 2) >= 1000 THEN 'Rota 2'
        ELSE 'Rota 1'
    END AS 'Categoria'

```

```
FROM
```

```
    DimProduct
```

```
INNER JOIN DimProductSubcategory
```

```
    ON DimProduct.ProductSubcategoryKey =
```

```
DimProductSubcategory.ProductSubcategoryKey
```

```
GROUP BY ProductSubcategoryName
```

/*

5. O setor de marketing está com algumas ideias de ações para alavancar as vendas em 2021. Uma

delas consiste em realizar sorteios entre os clientes da empresa.

Este sorteio será dividido em categorias:

‘Sorteio Mãe do Ano’: Nessa categoria vão participar todas as mulheres com filhos.

‘Sorteio Pai do Ano’: Nessa categoria vão participar todos os pais com filhos.

‘Caminhão de Prêmios’: Nessa categoria vão participar todas os demais clientes (homens e mulheres sem filhos).

Seu papel será realizar uma consulta à tabela DimCustomer e retornar 3 colunas:

- FirstName AS ‘Nome’

- Gender AS ‘Sexo’

- TotalChildren AS ‘Qtd. Filhos’

- EmailAddress AS ‘E-mail’

- Ação de Marketing: nessa coluna você deverá dividir os clientes de acordo com as categorias

‘Sorteio Mãe do Ano’, ‘Sorteio Pai do Ano’ e ‘Caminhão de Prêmios’.

*/

```
SELECT * FROM DimCustomer
```

```
SELECT
```

```
    FirstName AS 'Nome',
```

```
    Gender AS 'Sexo',
```

```
    TotalChildren AS 'Qtd. Filhos',
```

```
    EmailAddress AS 'E-mail',
```

```
    CASE
```

```
        WHEN Gender = 'F' AND TotalChildren > 0 THEN 'Sorteio Mãe do Ano'
```

```
        WHEN Gender = 'M' AND TotalChildren > 0 THEN 'Sorteio Pae do Ano'
```

```
        ELSE 'Sorteio caminhão de prêmios'
```

```
    END AS 'Acao de Marketing'
```

```
FROM
```

```
    DimCustomer
```

/*

6. Descubra qual é a loja que possui o maior tempo de atividade (em dias). Você deverá fazer essa

consulta na tabela DimStore, e considerar a coluna OpenDate como referência para esse cálculo.

Atenção: lembre-se que existem lojas que foram fechadas.

*/

```
SELECT TOP(10) * FROM DimStore
```

```
SELECT
```

```
    StoreName,
```

```
    OpenDate,
```

```
    CloseDate,
```

```
    CASE
```

```
        WHEN CloseDate IS NULL THEN DateDIFF(DAY, OpenDate, GETDATE())
```

```
        ELSE DateDIFF(DAY, OpenDate, CloseDate)
```

```
    END AS 'Dias de Atividade'
```

```
FROM
```

```
    DimStore
```

/*

Mod-12 – [SQL Server] Funções Condicionais

-- Aula 2 de 18: CASE WHEN... ELSE (Explicação)

-- Introdução: A função CASE permite tratar condições no SQL

/*

```
CASE
```

```
    WHEN teste_logico THEN 'resultado1'
```

```
    ELSE 'resultado2'
```

```
END
```

*/

-- Exemplo 1: Determine a situação do aluno. Se a Média for >= 6, então está aprovado. Caso contrário, reprovado.

```
DECLARE @varNota FLOAT
SET @varNota = 6
```

```
SELECT
    CASE
        WHEN @varNota >= 6 THEN 'Aprovado'
        ELSE 'Reprovado'
    END AS 'Situação'
```

-- Exemplo 2: A data de vencimento de um produto é no dia 10/03/2022. Faça um teste lógico para verificar se um produto passou da validade ou não.

```
DECLARE @varDataVencimento DATETIME, @varDataAtual DATETIME
SET @varDataVencimento = '10/03/2025'
SET @varDataAtual = '30/04/2022'
```

```
SELECT
    CASE
        WHEN @varDataAtual > @varDataVencimento THEN 'Produto Vencido'
        ELSE 'Na validade'
    END
```

-- Aula 3 de 18: CASE WHEN... ELSE (Exemplo)

-- Exemplo: Faça um SELECT das colunas CustomerKey, FirstName e Gender na tabela DimCustomer e utilize o CASE para criar uma 4ª coluna com a informação de 'Masculino' ou 'Feminino'.

```
SELECT * FROM DimCustomer
```

```
SELECT
    CustomerKey AS 'ID Cliente',
    FirstName AS 'Nome',
    Gender AS 'Sexo',
    CASE
        WHEN Gender = 'M' THEN 'Masculino'
        ELSE 'Feminino'
    END AS 'Sexo (CASE)'
FROM
    DimCustomer
```

-- [SQL Server] Funções Condicionais

-- Aula 4 de 18: CASE WHEN WHEN... ELSE (Explicação)

```
/*
CASE
    WHEN teste_logico1 THEN 'resultado1'
    WHEN teste_logico2 THEN 'resultado2'
    ELSE 'resultado3'
END
```

Exemplo 1:

Crie um código para verificar a nota do aluno e determinar a situação:

- Aprovado: nota maior ou igual a 6
- Prova final: nota entre 4 e 6
- Reprovado: nota abaixo de 4

```
DECLARE @varNota FLOAT
SET @varNota = 1
```

```

SELECT
CASE
    WHEN @varNota >= 6 THEN 'Aprovado'
    WHEN @varNota >= 4 THEN 'Prova final'
    ELSE 'Reprovado'
END

*/

```

```

-- Exemplo 2: Classifique o produto de acordo com o seu preço:
-- Preço >= 40000: Luxo
-- Preço >= 10000 e Preço < 40000: Econômico
-- Preço < 10000: Básico

```

```

DECLARE @varPreco FLOAT
SET @varPreco = 30000

```

```

SELECT
CASE
    WHEN @varPreco >= 40000 THEN 'Luxo'
    WHEN @varPreco >= 10000 THEN 'Econômico'
    ELSE 'Básico'
END

```

```

-- Aula 5 de 18: CASE WHEN WHEN... ELSE (Exemplo)
-- Exemplo: Crie uma coluna para substituir o 'M' por 'Masculino' e 'F' por
'Feminino'. Verifique se será necessário fazer alguma correção.

```

```

SELECT * FROM DimCustomer

```

```

SELECT
    CustomerKey,
    FirstName,
    Gender,
    CASE
        WHEN Gender = 'M' THEN 'Masculino'
        WHEN Gender = 'F' THEN 'Feminino'
        ELSE 'Empresa'
    END AS 'Sexo'
FROM
    DimCustomer

```

```

-- Aula 6 de 18: CASE com os operadores lógicos AND e OR
-- Faça uma consulta à tabela DimProduct e retorne as colunas ProductName, BrandName,
ColorName, UnitPrice e uma coluna de preço com desconto.
-- a) Caso o produto seja da marca Contoso E da cor Red, o desconto do produto será de
10%. Caso contrário, não terá nenhum desconto.

```

```

SELECT
    ProductName,
    BrandName,
    ColorName,
    UnitPrice,
    CASE
        WHEN BrandName = 'Contoso' AND ColorName = 'Red' THEN 0.1
        ELSE 0
    END AS 'Preço com desconto'
FROM
    DimProduct

```

-- b) Caso o produto seja da marca Litware OU Fabrikam, ele receberá um desconto de 5%. Caso contrário, não terá nenhum desconto.

```
SELECT
    ProductName,
    BrandName,
    ColorName,
    UnitPrice,
    CASE
        WHEN BrandName = 'Contoso' OR BrandName = 'Fabrikam' THEN 0.05
        ELSE 0
    END AS 'Preço com desconto'
FROM
    DimProduct
```

-- [SQL Server] Funções Condicionais
-- Aula 7 de 18: CASE Aninhado

-- DimEmployee

```
SELECT * FROM DimEmployee
```

-- 4 Cargos (Title):

-- Sales Group Manager

-- Sales Region Manager

-- Sales State Manager

-- Sales Store Manager

-- Assalariado (SalariedFlag)?

-- SalariedFlag = 0: não é assalariado

-- SalariedFlag = 1: é assalariado

-- Situação: Cálculo do bônus

-- Sales Group Manager: Se for assalariado, 20%; Se não, 15%.

-- Sales Region Manager: 15%

-- Sales State Manager: 7%

-- Sales Store Manager: 2%

```
SELECT
    FirstName,
    Title,
    SalariedFlag,
    CASE
        WHEN Title = 'Sales Group Manager' THEN
            CASE
                WHEN SalariedFlag = 1 THEN 0.3
                ELSE 0.2
            END
        WHEN Title = 'Sales Region Manager' THEN 0.15
        WHEN Title = 'Sales State Manager' THEN 0.07
        ELSE 0.02
    END AS 'Bônus'
FROM
    DimEmployee
```

-- Aula 8 de 18: CASE Aditivo

-- Os produtos da categoria 'TV and Video' terão um desconto de 10%

-- Se além de ser da categoria 'TV and Video', o produto for da subcategoria 'Televisions', receberá mais 5%. Total, 15%

```
SELECT
    ProductKey,
    ProductName,
```

```

        ProductCategoryName,
        ProductSubcategoryName,
        UnitPrice,
        CASE WHEN ProductCategoryName = 'TV and Video'
            THEN 0.10 ELSE 0.00 END
    + CASE WHEN ProductSubCategoryName = 'Televisions'
        THEN 0.05 ELSE 0.00 END
FROM DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey =
DimProductSubcategory.ProductSubcategoryKey
    INNER JOIN DimProductCategory
        ON DimProductSubcategory.ProductCategoryKey =
DimProductCategory.ProductCategoryKey

```

-- Aula 9 de 18: IIF (Alternativa ao CASE)

-- Exemplo 1: Qual é a categoria de risco do projeto abaixo, de acordo com a sua nota:
-- Risco Alto: Classificacao >= 5
-- Risco Baixo: Classificacao < 5

```

DECLARE @varClassificacao int
set @varClassificacao = 9

```

```

SELECT
    IIF(
        @varClassificacao >= 5,
        'Risco Alto',
        'Risco Baixo'
    )

```

-- Exemplo 2: Crie uma coluna única de 'Cliente', contendo o nome do cliente, seja ele uma pessoa ou uma empresa. Traga também a coluna de CustomerKey e CustomerType.

```

SELECT * FROM DimCustomer

```

```

SELECT
    CustomerKey,
    CustomerType,
    IIF(
        CustomerType = 'Person',
        FirstName,
        CompanyName) AS 'Cliente'
FROM
    DimCustomer

```

-- [SQL Server] Funções Condicionais
-- Aula 10 de 18: ISNULL – Tratando valores nulos

-- Exemplo: Existem 3 tipos de estoque: High, Mid e Low. Faça um SELECT contendo as colunas de ProductKey, ProductName, StockTypeName e Nome do responsável pelo produto, de acordo com o tipo de estoque. A regra deverá ser a seguinte:
-- João é responsável pelos produtos High
-- Maria é responsável pelos produtos Mid
-- Luis é responsável pelos produtos Low

```

SELECT
    ProductKey,
    ProductName,
    iif(
        StockTypeName = 'High',
        'João',
        IIF(
            StockTypeName = 'Mid',
            'Maria',

```

```

        'Luis')
    ) AS 'Responsável'

```

```

FROM
    DimProduct

```

-- Aula 11 de 18: ISNULL - Tratando valores nulos

-- Exemplo: Faça uma consulta que substitua todos os valores nulos de CityName da tabela DimGeography pelo texto 'Local desconhecido'.

```

SELECT
    GeographyKey,
    ContinentName,
    CityName,
    ISNULL(CityName, 'Local desconhecido')
FROM
    DimGeography

```

/*
MÓDULO 11: EXERCÍCIOS

1. O setor de vendas decidiu aplicar um desconto aos produtos de acordo com a sua classe. O percentual aplicado deverá ser de:

Economy -> 5%
Regular -> 7%
Deluxe -> 9%

a) Faça uma consulta à tabela DimProduct que retorne as seguintes colunas: ProductKey, ProductName, e outras duas colunas que deverão retornar o % de Desconto e UnitPrice com desconto.

b) Faça uma adaptação no código para que os % de desconto de 5%, 7% e 9% sejam facilmente modificados (dica: utilize variáveis).

```

*/
SELECT TOP(10) * FROM DimProduct

DECLARE @varDescEco FLOAT, @varDescReg FLOAT, @varDescDe FLOAT
SET @varDescEco = 0.10
SET @varDescReg = 0.07
SET @varDescDe = 0.09
SELECT

```

```

    ProductKey AS 'ID Produto',
    ProductName AS 'Nome Produto',
    ClassName AS 'Classe',
    CASE
        WHEN ClassName = 'Economy' THEN (1 - @varDescEco) * UnitPrice
        WHEN ClassName = 'Regular' THEN (1 - @varDescReg) * UnitPrice
        ELSE (1 - @varDescDe) * UnitPrice
    END AS ' % Desconto'

```

```

FROM
    DimProduct

```

/*
2. Você ficou responsável pelo controle de produtos da empresa e deverá fazer uma análise da quantidade de produtos por Marca.

A divisão das marcas em categorias deverá ser a seguinte:

CATEGORIA A: Mais de 500 produtos

CATEGORIA B: Entre 100 e 500 produtos

CATEGORIA C: Menos de 100 produtos

Faça uma consulta à tabela DimProduct e retorne uma tabela com um agrupamento de Total de Produtos por Marca, além da coluna de Categoria, conforme a regra acima.

*/

```
SELECT TOP(10) * FROM DimProduct
```

```
SELECT
```

```
    BrandName AS 'Marca',  
    COUNT(*) AS 'Qtd. Pruduto',  
    CASE  
        WHEN COUNT(*) >= 500 THEN 'CATEGORIA A'  
        WHEN COUNT(*) >= 100 THEN 'CATEGORIA B'  
        ELSE 'CATEGORIA C'  
    END AS 'Categoria'
```

```
FROM
```

```
    DimProduct
```

```
GROUP BY BrandName
```

/*

3. Será necessário criar uma categorização de cada loja da empresa considerando a quantidade de funcionários de cada uma. A lógica a ser seguida será a lógica abaixo:

```
EmployeeCount >= 50; 'Acima de 50 funcionários'  
EmployeeCount >= 40; 'Entre 40 e 50 funcionários'  
EmployeeCount >= 30; 'Entre 30 e 40 funcionários'  
EmployeeCount >= 20; 'Entre 20 e 30 funcionários'  
EmployeeCount >= 10; 'Entre 10 e 20 funcionários'  
Caso contrário: 'Abaixo de 10 funcionários'
```

Faça uma consulta à tabela DimStore que retorne as seguintes informações: StoreName, EmployeeCount e a coluna de categoria, seguindo a regra acima.

*/

```
SELECT TOP(10) * FROM DimStore
```

```
SELECT
```

```
    StoreName AS 'Nome da Loja',  
    EmployeeCount AS 'Qtd. Funcionários',  
    CASE  
        WHEN EmployeeCount >= 50 THEN 'Acima de 50 Funcionários'  
        WHEN EmployeeCount >= 40 THEN 'Acima de 40 Funcionários'  
        WHEN EmployeeCount >= 30 THEN 'Acima de 30 Funcionários'  
        WHEN EmployeeCount >= 20 THEN 'Acima de 20 Funcionários'  
        WHEN EmployeeCount >= 10 THEN 'Acima de 10 Funcionários'  
        ELSE 'Abaixo de 10 Funcionários'  
    END AS 'Categoria'
```

```
FROM
```

```
    DimStore
```

/*

4. O setor de logística deverá realizar um transporte de carga dos produtos que estão no depósito

de Seattle para o depósito de Sunnyside.

Não se tem muitas informações sobre os produtos que estão no depósito, apenas se sabe que existem 100 exemplares de cada Subcategoria. Ou seja, 100 laptops, 100 câmeras digitais, 100 ventiladores, e assim vai.

O gerente de logística definiu que os produtos serão transportados por duas rotas distintas. Além

disso, a divisão dos produtos em cada uma das rotas será feita de acordo com as subcategorias (ou

seja, todos os produtos de uma mesma subcategoria serão transportados pela mesma rota):

Rota 1: As subcategorias que tiverem uma soma total menor que 1000 kg deverão ser transportados pela Rota 1.

Rota 2: As subcategorias que tiverem uma soma total maior ou igual a 1000 kg deverão ser transportados pela Rota 2.

Você deverá realizar uma consulta à tabela DimProduct e fazer essa divisão das subcategorias por

cada rota. Algumas dicas:

- Dica 1: A sua consulta deverá ter um total de 3 colunas: Nome da Subcategoria, Peso Total e Rota.

- Dica 2: Como não se sabe quais produtos existem no depósito, apenas que existem 100

exemplares de cada subcategoria, você deverá descobrir o peso médio de cada subcategoria e

multiplicar essa média por 100, de forma que você descubra aproximadamente qual é o peso total

dos produtos por subcategoria.

- Dica 3: Sua resposta final deverá ter um JOIN e um GROUP BY.

*/

```
SELECT TOP(10) * FROM DimProduct
```

```
SELECT
```

```
    ProductSubcategoryName AS 'Nome SubCategoria',  
    ROUND(AVG(Weight) * 100, 2) AS 'Peso Total',  
    CASE  
        WHEN ROUND(AVG(Weight) * 100, 2) >= 1000 THEN 'Rota 2'  
        ELSE 'Rota 1'  
    END AS 'Categoria'
```

```
FROM
```

```
    DimProduct
```

```
INNER JOIN DimProductSubcategory
```

```
    ON DimProduct.ProductSubcategoryKey =
```

```
    DimProductSubcategory.ProductSubcategoryKey
```

```
GROUP BY ProductSubcategoryName
```

```
/*
```

5. O setor de marketing está com algumas ideias de ações para alavancar as vendas em 2021. Uma

delas consiste em realizar sorteios entre os clientes da empresa.

Este sorteio será dividido em categorias:

‘Sorteio Mãe do Ano’: Nessa categoria vão participar todas as mulheres com filhos.

‘Sorteio Pai do Ano’: Nessa categoria vão participar todos os pais com filhos.

‘Caminhão de Prêmios’: Nessa categoria vão participar todas os demais clientes (homens e mulheres sem filhos).

Seu papel será realizar uma consulta à tabela DimCustomer e retornar 3 colunas:

- FirstName AS ‘Nome’

- Gender AS ‘Sexo’

- TotalChildren AS ‘Qtd. Filhos’

- EmailAddress AS ‘E-mail’

- Ação de Marketing: nessa coluna você deverá dividir os clientes de acordo com as categorias

‘Sorteio Mãe do Ano’, ‘Sorteio Pai do Ano’ e ‘Caminhão de Prêmios’.

```
*/
```

```
SELECT * FROM DimCustomer
```

```
SELECT
```

```
    FirstName AS 'Nome',  
    Gender AS 'Sexo',  
    TotalChildren AS 'Qtd. Filhos',  
    EmailAddress AS 'E-mail',  
    CASE
```

```

        WHEN Gender = 'F' AND TotalChildren > 0 THEN 'Sorteio Mãe do Ano'
        WHEN Gender = 'M' AND TotalChildren > 0 THEN 'Sorteio Pae do Ano'
        ELSE 'Sorteio caminhão de prêmios'
    END AS 'Acao de Marketing'
FROM
    DimCustomer

```

/*
6. Descubra qual é a loja que possui o maior tempo de atividade (em dias). Você deverá fazer essa consulta na tabela DimStore, e considerar a coluna OpenDate como referência para esse cálculo.
Atenção: lembre-se que existem lojas que foram fechadas.

```

*/
SELECT TOP(10) * FROM DimStore

SELECT
    StoreName,
    OpenDate,
    CloseDate,
    CASE
        WHEN CloseDate IS NULL THEN DateDIFF(DAY, OpenDate, GETDATE())
        ELSE DateDIFF(DAY, OpenDate, CloseDate)
    END AS 'Dias de Atividade'
FROM
    DimStore

```

Mod-13 - Criando Banco de Dados

```

-- Criando nosso Banco de Dados, atualizar para vê a alteração
$ CREATE DATABASE TESTE

```

```

-- Deletando nosso Banco de Dados, atualizar para vê a alteração
$ DROP DATABASE TESTE
*/

```

```

-- ++++++
-- CRIAR NOSSO BANCO DE DADOS PARA ESSE MÓDULO
-- $ CREATE DATABASE BDImpressionador

```

```

-- ++++++
USE BDImpressionador

```

```

-- CREATE TABLE nome_tabela(
--         coluna1 tipo1,
--         coluna1 tipo1,
--         coluna1 tipo1
-- )

```

```

CREATE TABLE Produtos(
    id_produto int,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)

```

```

SELECT * FROM Produtos

```

```

-- ++++++
-- Adicionar Dados na Tabela Produtos

```

```

INSERT INTO Produtos(

```

```

        id_produto,
        nome_produto,
        data_validade,
        preco_produto
    )
SELECT
    ProductKey,
    ProductName,
    AvailableForSaleDate,
    UnitPrice
FROM
    ContosoRetailDW.dbo.DimProduct

-- Seleciona os primeiros linha da tabela Produtos
SELECT TOP(50) * FROM Produtos where id_produto = 1

-- Insere linhas na tabela Produtos
INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
VALUES
    (1, 'Arroz', '31/12/2021', 22.50),
    (2, 'Feijão', '20/11/2022', 8.99)

-- Seleciona os Produtos com indice 1(duplicados)
SELECT TOP(50) * FROM Produtos where id_produto = 1

-- Inserindo fora da ordem e sem informar campos
INSERT INTO Produtos(data_validade, id_produto, preco_produto)
VALUES
    ('31/12/2021', 3, 11.45)

-- Seleciona os Produtos com indice 1(duplicados)
SELECT TOP(50) * FROM Produtos where id_produto IN(1, 2, 3)

-- Alterando campo de uma tabela
UPDATE Produtos
    SET nome_produto = 'Macarrão'
WHERE
    id_produto = 3

-- Deletar registro
DELETE
FROM Produtos
WHERE id_produto = 3

```

/*

Mod-13 – Exercícios

1. a) A partir da tabela DimProduct, crie uma View contendo as informações de ProductName, ColorName, UnitPrice e UnitCost, da tabela DimProduct. Chame essa View de vwProdutos.

*/

```

CREATE VIEW vwProdutos AS
SELECT
    ProductName AS 'Nome do Produto',
    ColorName AS 'Cor',
    UnitPrice AS 'Preço',
    UnitCost AS 'Custo'
From
    DimProduct

GO
SELECT * FROM vwProdutos

```

/*

b) A partir da tabela DimEmployee, crie uma View mostrando FirstName, BirthDate, DepartmentName. Chame essa View de vwFuncionarios.

```
*/
GO
CREATE VIEW vwFuncionarios AS
SELECT
    FirstName AS 'Nome do Funcionário',
    BirthDate AS 'Data de Nascimento',
    DepartmentName AS 'Departamento'
From
    DimEmployee
```

```
GO
SELECT * FROM vwFuncionarios
```

/*
c) A partir da tabela DimStore, crie uma View mostrando StoreKey, StoreName e OpenDate. Chame essa View de vwLojas.

```
*/
GO
CREATE VIEW vwLojas AS
SELECT
    StoreKey,
    StoreName,
    OpenDate
FROM
    DimStore
```

```
GO
SELECT * FROM vwLojas
```

```
GO
/*
```

2. Crie uma View contendo as informações de Nome Completo (FirstName + LastName), Gênero (por extenso), E-mail e Renda Anual (formatada com R\$). Utilize a tabela DimCustomer. Chame essa View de vwClientes.

```
*/
CREATE VIEW vwClientes AS
SELECT
    CONCAT(FirstName, ' ', LastName) as 'Nome Completo' ,
    Replace(Replace(Gender, 'M', 'Masculino'), 'F', 'Feminino') AS 'Gênero',
    EmailAddress AS 'E-mail',
    FORMAT(YearlyIncome, 'C') AS 'Renda Anual'
FROM
    DimCustomer
```

```
GO
SELECT * FROM vwClientes
```

/*
3. a) A partir da tabela DimStore, crie uma View que considera apenas as lojas ativas. Faça um SELECT de todas as colunas. Chame essa View de vwLojasAtivas.

```
*/
GO
CREATE VIEW vwLojasAtivas AS
SELECT * FROM DimStore
WHERE Status = 'On'
```

```
GO
SELECT * FROM vwLojasAtivas
```

```
GO
/*
```

b) A partir da tabela DimEmployee, crie uma View de uma tabela que considera apenas os funcionários da área de Marketing. Faça um SELECT das colunas: FirstName, EmailAddress e DepartmentName. Chame essa de vwFuncionariosMkt.

```
*/
GO
CREATE VIEW vwFuncionariosMkt AS
SELECT
    FirstName,
    BirthDate,
    DepartmentName
From
    DimEmployee
WHERE DepartmentName = 'Marketing'

GO
SELECT * FROM vwFuncionariosMkt
```

/*
c) Crie uma View de uma tabela que considera apenas os produtos das marcas Contoso e Litware. Além disso, a sua View deve considerar apenas os produtos de cor Silver. Faça um SELECT de todas as colunas da tabela DimProduct. Chame essa View de vwContosoLitwareSilver.

```
*/
GO
CREATE VIEW vwContosoLitwareSilver AS
SELECT * From DimProduct
WHERE BrandName IN('Contoso', 'Litware') AND ColorName = 'Silver'

GO
SELECT * FROM vwContosoLitwareSilver
```

/*
4. Crie uma View que seja o resultado de um agrupamento da tabela FactSales. Este agrupamento deve considerar o SalesQuantity (Quantidade Total Vendida) por Nome do Produto. Chame esta View de vwTotalVendidoProdutos.

OBS: Para isso, você terá que utilizar um JOIN para relacionar as tabelas FactSales e DimProduct.

```
*/
GO
CREATE VIEW vwTotalVendidoProdutos AS
SELECT
    ProductName as 'Nome do Produto',
    Sum(SalesQuantity) AS 'Qtd. Vendida'
From
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
GROUP BY ProductName

GO
SELECT * FROM vwTotalVendidoProdutos
```

/*
5. Faça as seguintes alterações nas tabelas da questão 1.
a. Na View criada na letra a da questão 1, adicione a coluna de BrandName.

```
*/
GO
ALTER VIEW vwProdutos AS
SELECT
    ProductName AS 'Nome do Produto',
    BrandName AS 'Marca',
    ColorName AS 'Color',
    UnitCost AS 'Custo Unitário'
```

```
FROM
    DimProduct
```

```
GO
SELECT * FROM vwProdutos
```

```
/*
b. Na View criada na letra b da questão 1, faça um filtro e considere apenas os
funcionários do sexo feminino.
*/
```

```
GO
ALTER VIEW vwFuncionarios AS
SELECT
    FirstName AS 'Nome',
    BirthDate AS 'Data de Nascimento',
    DepartmentName AS 'Departamento'
From
    DimEmployee
WHERE Gender = 'F'
```

```
GO
SELECT * FROM vwFuncionarios
```

```
/*
c. Na View criada na letra c da questão 1, faça uma alteração e filtre apenas as lojas
ativas.
*/
```

```
GO
ALTER VIEW vwLojas AS
SELECT
    StoreKey AS 'ID Loja',
    StoreName AS 'Nome da Loja',
    OpenDate AS 'Data de Abertura'
FROM
    DimStore
WHERE Status = 'On'
```

```
GO
SELECT * FROM vwLojas
```

```
/*
6. a) Crie uma View que seja o resultado de um agrupamento da tabela DimProduct. O
resultado esperado da consulta deverá ser o total de produtos por marca. Chame essa
View de vw_6a.
*/
```

```
GO
CREATE VIEW vw_6A AS
SELECT
    BrandName AS 'Marca',
    COUNT(*) AS 'Qtd. Produtos'
FROM
    DimProduct
GROUP BY BrandName
```

```
GO
SELECT * FROM vw_6A
```

```
/*
b) Altere a View criada no exercício anterior, adicionando o peso total por marca.
Atenção:
sua View final deverá ter então 3 colunas: Nome da Marca, Total de Produtos e Peso
Total.
*/
```

```
GO
ALTER VIEW vw_6A AS
```

```

SELECT
    BrandName AS 'Marca',
    COUNT(*) AS 'Qtd. Produtos',
    SUM(Weight) AS 'Peso Total'
FROM
    DimProduct
GROUP BY BrandName

```

```

GO
SELECT * FROM vw_6A
/*
c) Exclua a View vw_6a.
*/
GO

```

```

DROP VIEW vw_6A

```

``` /* MOD-14- CRUD ```

```

-- Criando nosso Banco de Dados, atualizar para vê a alteração
$ CREATE DATABASE TESTE

```

```

-- Deletando nosso Banco de Dados, atualizar para vê a alteração
$ DROP DATABASE TESTE
*/

```

```

-- ++++++
-- CRIAR NOSSO BANCO DE DADOS PARA ESSE MÓDULO
-- $ CREATE DATABASE BDImpressionador

```

```

-- ++++++
USE BDImpressionador

```

```

-- CREATE TABLE nome_tabela(
--         coluna1 tipo1,
--         coluna1 tipo1,
--         coluna1 tipo1
-- )

```

```

CREATE TABLE Produtos(
    id_produto int,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)

```

```

SELECT * FROM Produtos

```

```

-- ++++++
-- Adicionar Dados na Tabela Produtos

```

```

INSERT INTO Produtos(
    id_produto,
    nome_produto,
    data_validade,
    preco_produto
)

```

```

SELECT
    ProductKey,
    ProductName,
    AvailableForSaleDate,
    UnitPrice

```

```

FROM
    ContosoRetailDW.dbo.DimProduct

-- Seleciona os primeiros linha da tabela Produtos
SELECT TOP(50) * FROM Produtos where id_produto = 1

-- Insere linhas na tabela Produtos
INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
VALUES
    (1, 'Arroz', '31/12/2021', 22.50),
    (2, 'Feijão', '20/11/2022', 8.99)

-- Seleciona os Produtos com indice 1(duplicados)
SELECT TOP(50) * FROM Produtos where id_produto = 1

-- Inserindo fora da ordem e sem informar campos
INSERT INTO Produtos(data_validade, id_produto, preco_produto)
VALUES
    ('31/12/2021', 3, 11.45)

-- Seleciona os Produtos com indice 1(duplicados)
SELECT TOP(50) * FROM Produtos where id_produto IN(1, 2, 3)

-- Alterando campo de uma tabela
UPDATE Produtos
SET nome_produto = 'Macarrão'
WHERE
    id_produto = 3

-- Deletar registro
DELETE
FROM Produtos
WHERE id_produto = 3

-- [SQL Server] CRUD
-- Aula 6 de 21 - CREATE TABLE - Criando a primeira tabela

-- Crie uma tabela chamada 'Produtos'
-- Essa abela deve conter 4 colunas: id_produ, nome_produto, data_validade e
preco_produto
-- Certifique-se de que o tipo das colunas está correto.

CREATE DATABASE BDImpressionador    -- Obs: Se você já criou o BD Impressionador não
precisa executar esta linha

USE BDImpressionador

CREATE TABLE Produtos(
    id_produto INT,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)

SELECT * FROM Produtos

-- [SQL Server] CRUD
-- Aula 7 de 21 - INSERT SELECT - Adicionando dados de outra tabela

-- Crie uma tabela chamada 'Produtos'
-- Essa abela deve conter 4 colunas: id_produ, nome_produto, data_validade e
preco_produto
-- Certifique-se de que o tipo das colunas está correto.

```

```

CREATE DATABASE BDImpressionador    -- Obs: Se você já criou o BD Impressionador não
precisa executar esta linha

USE BDImpressionador

CREATE TABLE Produtos(
    id_produto INT,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)

SELECT * FROM Produtos

-- Adicionando valores de outra tabela

INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
SELECT
    ProductKey,
    ProductName,
    AvailableForSaleDate,
    UnitPrice
FROM
    ContosoRetailDW.dbo.DimProduct

-- [SQL Server] CRUD
-- Aula 8 de 21 - INSERT SELECT - Adicionando novos valores na tabela

-- Crie uma tabela chamada 'Produtos'
-- Essa tabela deve conter 4 colunas: id_produto, nome_produto, data_validade e
preco_produto
-- Certifique-se de que o tipo das colunas está correto.

CREATE DATABASE BDImpressionador    -- Obs: Se você já criou o BD Impressionador não
precisa executar esta linha

USE BDImpressionador

CREATE TABLE Produtos(
    id_produto INT,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)

SELECT * FROM Produtos

-- Adicionando valores de outra tabela

INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
SELECT
    ProductKey,
    ProductName,
    AvailableForSaleDate,
    UnitPrice
FROM
    ContosoRetailDW.dbo.DimProduct

-- Adicionando novos valores na tabela

INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
VALUES
    (1, 'Arroz', '2021-12-31', 22.50),
    (2, 'Feijão', '2021-12-31', 8.99)

```

```

-- [SQL Server] CRUD
-- Aula 10 de 21 - UPDATE - Atualizando o dado de uma tabela

-- Crie uma tabela chamada 'Produtos'
-- Essa tabela deve conter 4 colunas: id_produto, nome_produto, data_validade e
preco_produto
-- Certifique-se de que o tipo das colunas está correto.

CREATE DATABASE BDImpressionador    -- Obs: Se você já criou o BD Impressionador não
precisa executar esta linha

USE BDImpressionador

CREATE TABLE Produtos(
    id_produto INT,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)

SELECT * FROM Produtos

-- Adicionando valores de outra tabela

INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
SELECT
    ProductKey,
    ProductName,
    AvailableForSaleDate,
    UnitPrice
FROM
    ContosoRetailDW.dbo.DimProduct

-- Adicionando novos valores na tabela

INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
VALUES
    (1, 'Arroz', '2021-12-31', 22.50),
    (2, 'Feijão', '2021-12-31', 8.99)

-- Atualizando o dado de uma tabela

UPDATE Produtos
SET nome_produto = 'Macarrão'
WHERE id_produto = 3

-- [SQL Server] CRUD
-- Aula 11 de 21 - DELETE - Deletando dados

-- Crie uma tabela chamada 'Produtos'
-- Essa tabela deve conter 4 colunas: id_produto, nome_produto, data_validade e
preco_produto
-- Certifique-se de que o tipo das colunas está correto.

CREATE DATABASE BDImpressionador    -- Obs: Se você já criou o BD Impressionador não
precisa executar esta linha

USE BDImpressionador

CREATE TABLE Produtos(
    id_produto INT,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)

```

```
SELECT * FROM Produtos
```

```
-- Adicionando valores de outra tabela
```

```
INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
SELECT
```

```
    ProductKey,
    ProductName,
    AvailableForSaleDate,
    UnitPrice
```

```
FROM
```

```
    ContosoRetailDW.dbo.DimProduct
```

```
-- Adicionando novos valores na tabela
```

```
INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
VALUES
```

```
    (1, 'Arroz', '2021-12-31', 22.50),
    (2, 'Feijão', '2021-12-31', 8.99)
```

```
-- Atualizando o dado de uma tabela
```

```
UPDATE Produtos
```

```
SET nome_produto = 'Macarrão'
```

```
WHERE id_produto = 3
```

```
-- Deletando dados de uma tabela
```

```
DELETE
```

```
FROM Produtos
```

```
WHERE id_produto = 3
```

```
-- [SQL Server] CRUD
```

```
-- Aula 12 de 21 - DROP TABLE - Excluindo uma tabela
```

```
-- Crie uma tabela chamada 'Produtos'
```

```
-- Essa tabela deve conter 4 colunas: id_produto, nome_produto, data_validade e
preco_produto
```

```
-- Certifique-se de que o tipo das colunas está correto.
```

```
CREATE DATABASE BDImpressionador    -- Obs: Se você já criou o BD Impressionador não
precisa executar esta linha
```

```
USE BDImpressionador
```

```
CREATE TABLE Produtos(
```

```
    id_produto INT,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
```

```
)
```

```
SELECT * FROM Produtos
```

```
-- Adicionando valores de outra tabela
```

```
INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
SELECT
```

```
    ProductKey,
    ProductName,
    AvailableForSaleDate,
    UnitPrice
```

```
FROM
```

ContosoRetailDW.dbo.DimProduct

-- Adicionando novos valores na tabela

```
INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
VALUES
    (1, 'Arroz', '2021-12-31', 22.50),
    (2, 'Feijão', '2021-12-31', 8.99)
```

-- Atualizando o dado de uma tabela

```
UPDATE Produtos
SET nome_produto = 'Macarrão'
WHERE id_produto = 3
```

-- Deletando dados de uma tabela

```
DELETE
FROM Produtos
WHERE id_produto = 3
```

-- Excluindo uma tabela

```
DROP TABLE Produtos
```

-- [SQL Server] CRUD

-- Aula 13 de 21 - Código da tabela utilizada nos próximos exemplos

-- Usar o Banco

```
USE BDImpressionador
```

-- Criar tabela Funcionários

```
CREATE TABLE Funcionarios(
    id_funcionario int,
    nome_funcionario varchar(100),
    salario float,
    data_nascimento date
)
```

--- Atualiza Banco de Dados - Atualizar(F5)

-- Inserir dados na tabela Funcionários

```
INSERT INTO Funcionarios(id_funcionario, nome_funcionario, salario, data_nascimento)
VALUES
    (1, 'Lucas', 1500, '20/03/1990'),
    (2, 'Andressa', 2300, '07/12/1988'),
    (3, 'Felipe', 4000, '13/02/1993'),
    (4, 'Marcelo', 7100, '10/04/1993'),
    (5, 'Carla', 3200, '02/09/1986'),
    (6, 'Juliana', 5500, '21/01/1989'),
    (7, 'Mateus', 1900, '02/11/1993'),
    (8, 'Sandra', 3900, '09/05/1990'),
    (9, 'André', 1000, '13/03/1994'),
    (10, 'Julio', 4700, '05/07/1992')
```

```
SELECT * FROM Funcionarios
```

-- [SQL Server] CRUD

-- Aula 14 de 21 - ALTER TABLE

-- ALTER TABLE: Adicionando, deletar ou modificar tipo de dados de uma coluna

-- Adicionar coluna:

```
ALTER TABLE Funcionarios
ADD cargo varchar(100), bonus decimal(10,2)
```

```
UPDATE Funcionarios
SET cargo = 'Analista'
WHERE id_funcionario = 1
SELECT * FROM Funcionarios
```

-- Alterar tipo de dados de uma coluna:

```
ALTER TABLE Funcionarios
ALTER COLUMN salario int
```

-- Deletar coluna:

```
ALTER TABLE Funcionarios
DROP COLUMN cargo, bônus
```

/*

MÓDULO 14: EXERCÍCIOS

1. a) Crie um banco de dados chamado BD_Testes.
- b) Exclua o banco de dados criado no item anterior.
- c) Crie um banco de dados chamado Exercicios.

*/

```
CREATE DATABASE BD_Testes
-- Atualizar(F5)
DROP DATABASE BD_Testes
-- Atualizar(F5)
CREATE DATABASE Exercicios
-- Atualizar(F5)
```

GO

/*

2. No banco de dados criado no exercício anterior(Exercicios), crie 3 tabelas, cada uma contendo as seguintes colunas:

Tabela 1: dCliente

- ID_Cliente
- Nome_Cliente
- Data_de _Nascimento

Tabela 2: dGerente

- ID_Gerente
- Nome_Gerente
- Data_de_Contratacao
- Salario

Tabela 3: fContratos

- ID_Contrato
- Data_de_Assinatura
- ID_Cliente
- ID_Gerente
- Valor_do_Contrato

Lembre-se dos seguintes pontos:

- a) Garantir que o Banco de Dados Exercicios está selecionado.
 - b) Definir qual será o tipo de dados mais adequado para cada coluna das tabelas. Lembrando que os tipos de dados mais comuns são: INT, FLOAT, VARCHAR e DATETIME.
- Por fim, faça um SELECT para visualizar cada tabela.

*/

```
USE Exercicios
```

GO

```
CREATE TABLE dCliente(
    ID_Cliente INT,
    Nome_Cliente VARCHAR(100),
```

```

        Data_de_Nascimento DATETIME
    )
-- Atualizar(F5)
SELECT * FROM dCliente

GO

CREATE TABLE dGerente(
    ID_Gerente INT,
    Nome_Gerente VARCHAR(100),
    Data_de_Contratacao DATETIME,
    Salario FLOAT
)
-- Atualizar(F5)
SELECT * FROM dGerente

GO

CREATE TABLE fContratos(
    ID_Contrato INT,
    Data_de_Assinatura DATETIME,
    ID_Cliente INT,
    ID_Gerente INT,
    Valor_do_Contrato FLOAT
)
-- Atualizar(F5)
SELECT * FROM fContratos

/*
3. Em cada uma das 3 tabelas, adicione os seguintes valores:
TABELA dCliente
TABELA dGerente
TABELA fContratos
*/
INSERT INTO dCliente(ID_Cliente, Nome_Cliente, Data_de_Nascimento)
VALUES
    (1, 'André Martins', '12/02/1989'),
    (2, 'Bárbara Campos', '07/05/1992'),
    (3, 'Carol Freitas', '23/04/1985'),
    (4, 'Diego Cardoso', '11/10/1984'),
    (5, 'Eduardo Pereira', '09/11/1988'),
    (6, 'Fabiana Silva', '02/09/1989'),
    (7, 'Gustavo Brabosa', '27/06/1993'),
    (8, 'Helen Viana', '11/02/1990'),
    (9, 'Igor Castro', '21/08/1989'),
    (10, 'Juliana Pires', '13/01/1991')

-- F5

INSERT INTO dGerente(ID_Gerente, Nome_Gerente, Data_de_Contratacao, Salario)
VALUES
    (1, 'Lucas Sampaio', '21/03/2015', 6700),
    (2, 'Mariana Padilha', '10/01/2011', 9900),
    (3, 'Natália Santos', '03/10/2018', 7200),
    (4, 'Otávio Costa', '18/04/2017', 11000 )

-- F5

INSERT INTO fContratos(ID_Contrato, Data_de_Assinatura, ID_Cliente, ID_Gerente,
Valor_do_Contrato)
VALUES
    (1, '12/01/2019', 8, 1, 6700),
    (2, '10/02/2019', 3, 2, 15500),
    (3, '07/03/2019', 7, 2, 6500),
    (4, '15/03/2019', 1, 3, 33000),

```

```
(5, '21/03/2019', 5, 4, 11100),
(6, '23/03/2019', 4, 2, 5500),
(7, '28/03/2019', 9, 3, 55000),
(8, '04/04/2019', 2, 1, 31000),
(9, '05/04/2019', 10, 4, 34000),
(10, '05/04/2019', 6, 2, 9200)
```

-- F5

```
SELECT * FROM dCliente
SELECT * FROM dGerente
SELECT * FROM fContratos
```

/*

4. Novos dados deverão ser adicionados nas tabelas dCliente, dGerente e fContratos.

Fique livre para adicionar uma nova linha em cada tabela contendo, respectivamente,

(1) um novo cliente (id cliente, nome e data de nascimento)

(2) um novo gerente (id gerente, nome, data de contratação e salário)

(3) um novo contrato (id, data assinatura, id cliente, id gerente, valor do contrato)

*/

```
INSERT INTO dCliente(ID_Cliente, Nome_Cliente, Data_de_Nascimento)
VALUES
```

```
(11, 'Pedro Cavalcante', '10/02/1992')
```

```
SELECT * FROM dCliente
```

--

```
INSERT INTO dGerente(ID_Gerente, Nome_Gerente, Data_de_Contratacao, Salario)
VALUES
```

```
(5, 'Rita Correa', '01/01/2020', 5000)
```

```
SELECT * FROM dGerente
```

--

```
INSERT INTO fContratos(ID_Contrato, Data_de_Assinatura, ID_Cliente, ID_Gerente,
Valor_do_Contrato)
VALUES
```

```
(11, '05/03/2020', 11, 2, 10000)
```

```
SELECT * FROM fContratos
```

/*

5. O contrato de ID igual a 4 foi registrado com alguns erros na tabela fContratos.

Faça uma alteração na tabela atualizando os seguintes valores:

Data_de_Assinatura: 17/03/2019

ID_Gerente: 2

Valor_do_Contrato: 33500

*/

```
SELECT * FROM fContratos
```

```
UPDATE fContratos
```

```
SET Data_de_Assinatura = '17/03/2019',
    ID_Gerente = 2,
    Valor_do_Contrato = 33500
```

```
WHERE ID_Contrato = 4
```

/*

6. Delete a linha da tabela fContratos que você criou na questão 4.

*/

```
SELECT * FROM fContratos
```

```
DELETE
```

```
FROM fContratos
```

```
WHERE ID_Contrato = 11
```

-- MOD-15-[SQL Server] Subqueries e CTE's

-- Aula 2 de 28: O que é uma Subquery e onde usamos

/*

1. O que é uma Subquery?

Uma Subconsulta (ou Subquery ou SubSELECT) nada mais é do que uma consulta dentro de outra consulta. Ou seja, com uma subquery conseguimos utilizar o resultado de uma query (consulta) dentro de outra consulta.

*/

/*

2. Onde utilizamos uma Subquery

Subqueries podem ser utilizadas em 3 situações possíveis:

Junto com o Where, funcionando como um filtro variável

Junto com o SELECT, como uma nova coluna na tabela

Junto com o FROM, como uma nova tabela

Ex: Imagina que eu quero selecionar os produtos com o Preço maior do que a média.

*/

SELECT

*

FROM

Produtos

WHERE Preço > 10.5

SELECT AVG(Preço) FROM Produtos

/*

3. Onde usamos uma subquery?

Em 4 situações possíveis:

1- Subquery junto com um WHERE, como um filtro dinâmico e escalar (valor único)

*/

SELECT

Coluna1,

Coluna2

FROM

Tabela

WHERE Coluna1 = (SELECT)

/*

2- Subquery junto com um WHERE, como um filtro dinâmico e em lista (vários valores)

*/

SELECT

Coluna1,

Coluna2

FROM

Tabela

WHERE Coluna1 IN (SELECT)

/*

3- Subquery junto com um SELECT, como uma nova coluna

*/

SELECT

Coluna1,

Coluna2,

(SELECT)

FROM

Tabela

/*

4- Subquery junto com um FROM, como uma nova tabela

```

*/
SELECT
    Coluna1,
    Coluna2
FROM
    (SELECT) AS T

```

-- [SQL Server] Subqueries e CTE's

-- Aula 2 de 28: O que é uma Subquery e onde usamos

```

/*
1. O que é uma Subquery?

```

Uma Subconsulta (ou Subquery ou SubSELECT) nada mais é do que uma consulta dentro de outra consulta. Ou seja, com uma subquery conseguimos utilizar o resultado de uma query (consulta) dentro de outra consulta.

```

*/

```

```

/*
2. Onde utilizamos uma Subquery
Subqueries podem ser utilizadas em 3 situações possíveis:
Junto com o Where, funcionando como um filtro variável
Junto com o SELECT, como uma nova coluna na tabela
Junto com o FROM, como uma nova tabela

```

Ex: Imagina que eu quero selecionar os produtos com o Preço maior do que a média.

```

*/
SELECT
    *
FROM
    Produtos
WHERE Preço > 10.5

SELECT AVG(Preço) FROM Produtos

```

```

/*
3. Onde usamos uma subquery?

```

Em 4 situações possíveis:

1- Subquery junto com um WHERE, como um filtro dinâmico e escalar (valor único)

```

*/
SELECT
    Coluna1,
    Coluna2
FROM
    Tabela
WHERE Coluna1 = (SELECT)

```

```

/*
2- Subquery junto com um WHERE, como um filtro dinâmico e em lista (vários valores)
*/

```

```

SELECT
    Coluna1,
    Coluna2
FROM
    Tabela
WHERE Coluna1 IN (SELECT)

```

```

/*
3- Subquery junto com um SELECT, como uma nova coluna
*/

```

```

SELECT
    Coluna1,
    Coluna2,
    (SELECT)

```

```

FROM
    Tabela
/*
4- Subquery junto com um FROM, como uma nova tabela
*/
SELECT
    Coluna1,
    Coluna2
FROM
    (SELECT) AS T

```

-- [SQL Server] [Subqueries e CTE's] Aula 3 de 28: Subquery na prática - Aplicação com o Where (Exemplo 1)

-- Para entender a ideia por trás das subqueries, vamos começar fazendo 3 exemplos com a aplicação WHERE.

-- Exemplo 1: Quais produtos da tabela DimProduct possuem custos acima da média?

```

SELECT AVG(UnitCost) FROM DimProduct      -- 147.6555

SELECT
    *
FROM
    DimProduct
WHERE UnitCost >= (SELECT AVG(UnitCost) FROM DimProduct)

```

-- [SQL Server] [Subqueries e CTE's] Aula 4 de 28: Subquery na prática - Aplicação com o Where (Exemplo 2)

-- Para entender a ideia por trás das subqueries, vamos começar fazendo 3 exemplos com a aplicação WHERE.

-- Exemplo 2: Faça uma consulta para retornar os produtos da categoria 'Televisions'. Tome cuidado pois não temos a informação de Nome da Subcategoria na tabela DimProduct. Dessa forma, precisaremos criar um SELECT que descubra o ID da categoria 'Televisions' e passar esse resultado como o valor que queremos filtrar dentro do WHERE.

```

SELECT * FROM DimProduct
WHERE ProductSubcategoryKey =
    (SELECT ProductSubcategoryKey FROM DimProductSubcategory
     WHERE ProductSubcategoryName = 'Televisions')

```

-- [SQL Server] [Subqueries e CTE's] Aula 5 de 28: Subquery na prática - Aplicação com o Where (Exemplo 3)

-- Para entender a ideia por trás das subqueries, vamos começar fazendo 3 exemplos com a aplicação WHERE.

-- Exemplo 3: Filtre a tabela FactSales e mostre apenas as vendas referentes às lojas com 100 ou mais funcionários

```

SELECT * FROM FactSales
WHERE StoreKey IN (
    SELECT StoreKey
    FROM DimStore
    WHERE EmployeeCount >= 100
)

```

-- [SQL Server] [Subqueries e CTE's] Aula 6 de 28: ANY, SOME e ALL

```

CREATE TABLE funcionarios(
    id_funcionario INT,
    nome VARCHAR(50),
    idade INT,

```

```
sexo VARCHAR(50))
```

```
INSERT INTO funcionarios(id_funcionario, nome, idade, sexo)
VALUES
```

```
(1, 'Julia', 20, 'F'),
(2, 'Daniel', 21, 'M'),
(3, 'Amanda', 22, 'F'),
(4, 'Pedro', 23, 'M'),
(5, 'André', 24, 'M'),
(6, 'Luisa', 25, 'F')
```

```
SELECT * FROM funcionarios
```

```
-- Selecione os funcionários do sexo masculino (MAS, utilizando a coluna de IDADE para
isso)
```

```
SELECT * FROM funcionarios
WHERE idade IN (21, 23, 24)
```

```
SELECT * FROM funcionarios
WHERE idade IN (SELECT idade FROM funcionarios WHERE sexo = 'M')
```

```
/*
= ANY(valor1, valor2, valor3) :
Equivalente ao IN, retorna as linhas da tabela que sejam iguais ao valor1, OU valor2,
OU valor3
*/
```

```
SELECT * FROM funcionarios
WHERE idade = ANY (SELECT idade FROM funcionarios WHERE sexo = 'M')
```

```
/*
> ANY(valor1, valor2, valor3) :
Retorna as linhas da tabela com valores maiores que o valor1, OU valor2, OU valor3. Ou
seja, maior que o mínimo dos valores
*/
```

```
SELECT * FROM funcionarios
WHERE idade > ANY (SELECT idade FROM funcionarios WHERE sexo = 'M')
```

```
/*
< ANY(valor1, valor2, valor3) :
Retorna as linhas da tabela com valores maiores que o valor1, OU valor2, OU valor3. Ou
seja, maior que o máximo dos valores
*/
```

```
SELECT * FROM funcionarios
WHERE idade < ANY (SELECT idade FROM funcionarios WHERE sexo = 'M')
```

```
/*
> ALL(valor1, valor2, valor3) :
Retorna as linhas da tabela com valores maiores que o valor1, E valor2, E valor3. Ou
seja, maior que o máximo dos valores
*/
```

```
SELECT * FROM funcionarios
WHERE idade > ALL (SELECT idade FROM funcionarios WHERE sexo = 'M')
```

```
/*
< ALL(valor1, valor2, valor3) :
Retorna as linhas da tabela com valores menores que o valor1, E valor2, E valor3. Ou
seja, menor que o mínimo dos valores
*/
```

```
SELECT * FROM funcionarios
WHERE idade < ALL (SELECT idade FROM funcionarios WHERE sexo = 'M')
```

-- [SQL Server] [Subqueries e CTE's] Aulas 7 e 8 de 28: EXISTS

-- Exemplo: Retornar uma tabela com todos os produtos (ID Produto e Nome Produto) que possuem alguma venda no dia 01/01/2007

-- 2517 - 1724 = 793

-- Conta a quantidade de Produtos

```
SELECT COUNT(*) FROM DimProduct
```

-- Retorna uma tabela com todos os produtos que possuem alguma venda

```
SELECT
    ProductKey,
    ProductName
FROM
    DimProduct
WHERE EXISTS(
    SELECT
        ProductKey
    FROM
        factSales
    WHERE
        DateKey = '01/01/2007'
        AND factSales.ProductKey = DimProduct.ProductKey
)
```

-- Solução alternativa com o ANY

```
SELECT
    ProductKey,
    ProductName
FROM
    DimProduct
WHERE ProductKey = ANY(
    SELECT
        ProductKey
    FROM
        factSales
    WHERE
        DateKey = '01/01/2007'
        -- AND factSales.ProductKey = DimProduct.ProductKey
)
```

-- [SQL Server] [Subqueries e CTE's] Aula 17 de 28: Criando múltiplas CTE's

-- Exemplo: Crie 2 CTE's:

-- 1. A primeira, chamada produtos_contoso, deve conter as seguintes colunas (DimProduct): ProductKey, ProductName, BrandName

-- 2. A segunda, chamada vendas_top100, deve ser um top 100 vendas mais recentes, considerando as seguintes colunas da tabela FactSales: SalesKey, ProductKey, DateKey, SalesQuantity

-- Por fim, faça um INNER JOIN dessas tabelas

```
WITH produtos_contoso AS (
    SELECT
        ProductKey,
        ProductName,
        BrandName
    FROM
```

```

        DimProduct
    WHERE
        BrandName = 'Contoso'
),
vendas_top100 AS (
    SELECT TOP(100)
        SalesKey,
        ProductKey,
        DateKey,
        SalesQuantity
    FROM
        FactSales
    ORDER BY
        DateKey DESC
)

```

```

SELECT * FROM vendas_top100
INNER JOIN produtos_contoso
    ON vendas_top100.ProductKey = produtos_contoso.ProductKey

```

-- [SQL Server] [Subqueries e CTE's] Aula 9 de 28: Subquery na prática - Aplicação com o SELECT

-- Exemplo: Retornar uma tabela com todos os produtos (ID Produto e Nome Produto) e também o total de vendas para cada produto

```

SELECT
    ProductKey,
    ProductName,
    (SELECT COUNT(ProductKey) FROM FactSales WHERE FactSales.ProductKey =
DimProduct.ProductKey) AS 'Qtd. Vendas'
FROM
    DimProduct

```

-- [SQL Server] [Subqueries e CTE's] Aula 10 de 28: Subquery na prática - Aplicação com o FROM

-- Exemplo: Retornar a quantidade total de produtos da marca Contoso.

```

SELECT
    COUNT(*)
FROM DimProduct
WHERE BrandName = 'Contoso'

SELECT
    COUNT(*)
FROM (SELECT * FROM DimProduct WHERE BrandName = 'Contoso') AS T

```

-- [SQL Server] [Subqueries e CTE's] Aula 11 de 28: Plano de Execução Estimado

-- Exemplo: Retornar a quantidade total de produtos da marca Contoso.

```

SELECT
    COUNT(*)
FROM DimProduct
WHERE BrandName = 'Contoso'

SELECT
    COUNT(*)
FROM (SELECT * FROM DimProduct WHERE BrandName = 'Contoso') AS T

```

-- [SQL Server] [Subqueries e CTE's] Aula 12 e 13 de 28: Subquery aninhada

-- Exemplo: Descubra os nomes dos clientes que ganham o segundo maior salário.

```
SELECT * FROM DimCustomer
WHERE CustomerType = 'Person'
ORDER BY YearlyIncome DESC
```

```
SELECT DISTINCT TOP(2) YearlyIncome FROM DimCustomer
WHERE CustomerType = 'Person'
ORDER BY YearlyIncome DESC
```

```
SELECT
    CustomerKey,
    FirstName,
    LastName,
    YearlyIncome
FROM DimCustomer
WHERE YearlyIncome = 160000
```

```
--1. Descobrir o maior salário
--2. Descobrir o segundo maior salário
--3. Descobrir os nomes dos clientes que ganham o segundo maior salário
```

```
SELECT
    CustomerKey,
    FirstName,
    LastName,
    YearlyIncome
FROM DimCustomer
WHERE YearlyIncome = (
    SELECT
        MAX(YearlyIncome)
    FROM DimCustomer
    WHERE
        YearlyIncome < (
            SELECT
                MAX(YearlyIncome)
            FROM DimCustomer
            WHERE CustomerType = 'Person'
        )
)
```

```
-- [SQL Server] [Subqueries e CTE's] Aula 14 de 28: CTE - O que é e como criar
```

```
-- Exemplo: Crie uma CTE para armazenar o resultado de uma consulta que contenha:
ProductKey, ProductName, BrandName, ColorName e UnitPrice, apenas para a marca
Contoso.
```

```
WITH cte AS (
SELECT
    ProductKey,
    ProductName,
    BrandName,
    ColorName,
    UnitPrice
FROM DimProduct
WHERE BrandName = 'Contoso'
)

SELECT
    COUNT(*)
FROM cte
```

```
-- [SQL Server] [Subqueries e CTE's] Aula 15 de 28: Calculando agregações com CTE
```

```
-- Exemplo: Crie uma CTE que seja o resultado do agrupamento de total de produtos por
marca. Faça uma média de produtos por marca.
```

```

WITH cte AS (
    SELECT
        BrandName,
        COUNT(*) AS 'Total'
    FROM
        DimProduct
    GROUP BY
        BrandName
)

```

```

SELECT
    AVG(Total)
FROM
    cte

```

-- [SQL Server] [Subqueries e CTE's] Aula 16 de 28: Nomeando colunas de uma CTE

-- Exemplo: Crie uma CTE que seja o resultado do agrupamento de total de produtos por marca. Faça uma média de produtos por marca.

```

WITH cte(Marca, Total) AS (
    SELECT
        BrandName,
        COUNT(*)
    FROM
        DimProduct
    GROUP BY
        BrandName
)

```

```

SELECT
    Marca,
    Total
FROM
    cte

```

-- [SQL Server] [Subqueries e CTE's] Aula 17 de 28: Criando múltiplas CTE's

-- Exemplo: Crie 2 CTE's:

-- 1. A primeira, chamada produtos_contoso, deve conter as seguintes colunas (DimProduct): ProductKey, ProductName, BrandName

-- 2. A segunda, chamada vendas_top100, deve ser um top 100 vendas mais recentes, considerando as seguintes colunas da tabela FactSales: SalesKey, ProductKey, DateKey, SalesQuantity

-- Por fim, faça um INNER JOIN dessas tabelas

-- USAR o BD Contoso

-- USE ContosoRetailDW

-- SELECT TOP(5) * FROM DimProduct

GO

```

WITH produtos_contoso AS (
    SELECT
        ProductKey,
        ProductName,
        BrandName
    FROM
        DimProduct
    WHERE
        BrandName = 'Contoso'
),
vendas_top100 AS (

```

```

SELECT TOP(100)
    SalesKey,
    ProductKey,
    DateKey,
    SalesQuantity
FROM
    FactSales
ORDER BY
    DateKey DESC
)

SELECT * FROM vendas_top100
INNER JOIN produtos_contoso
    ON vendas_top100.ProductKey = produtos_contoso.ProductKey

```

/*

Mod-15 – EXERCÍCIOS SUBQUERIES e CTEs

1. Para fins fiscais, a contabilidade da empresa precisa de uma tabela contendo todas as vendas referentes à loja 'Contoso Orlando Store'. Isso porque essa loja encontra-se em uma região onde a tributação foi modificada recente. Portanto, crie uma consulta ao Banco de Dados para obter uma tabela FactSales contendo todas as vendas desta loja.

```

*/
SELECT *
FROM FactSales
WHERE StoreKey = (
    SELECT
        StoreKey
    FROM
        DimStore
    WHERE
        StoreName = 'Contoso Orlando Store'
)
/*

```

2. O setor de controle de produtos quer fazer uma análise para descobrir quais são os produtos que possuem um UnitPrice maior que o UnitPrice do produto de ID igual a 1893.

a) A sua consulta resultante deve conter as colunas ProductKey, ProductName e UnitPrice da tabela DimProduct.

b) Nessa query você também deve retornar uma coluna extra, que informe o UnitPrice do produto 1893.

```

*/
SELECT
    ProductKey,
    ProductName,
    UnitPrice,
    (SELECT
        UnitPrice
    FROM DimProduct
    WHERE ProductKey = 1893)
FROM DimProduct
WHERE UnitPrice > (
    SELECT
        UnitPrice
    FROM DimProduct
    WHERE ProductKey = 1893
)
/*

```

3. A empresa Contoso criou um programa de bonificação chamado "Todos por 1". Este programa consistia no seguinte: 1 funcionário seria escolhido ao final do ano como o funcionário destaque, só que a bonificação seria recebida por todos da área daquele funcionário em particular. O objetivo desse programa seria o de incentivar a colaboração coletiva entre os funcionários de uma mesma área. Desta forma, o

funcionário destaque beneficiaria não só a si, mas também a todos os colegas de sua área.

Ao final do ano, o funcionário escolhido como destaque foi o Miguel Severino. Isso significa que todos os funcionários da área do Miguel seriam beneficiados com o programa.

O seu objetivo é realizar uma consulta à tabela DimEmployee e retornar todos os funcionários da área “vencedora” para que o setor Financeiro possa realizar os pagamentos das bonificações.

*/

```
SELECT *
FROM DimEmployee
WHERE DepartmentName = (
    SELECT
        DepartmentName
    FROM DimEmployee
    WHERE FirstName = 'Miguel'
    AND LastName = 'Severino'
)
/*
```

4. Faça uma query que retorne os clientes que recebem um salário anual acima da média. A sua query deve retornar as colunas CustomerKey, FirstName, LastName, EmailAddress e YearlyIncome.

Obs: considere apenas os clientes que são 'Pessoas Físicas'.

*/

```
SELECT
    CustomerKey,
    FirstName,
    LastName,
    EmailAddress,
    YearlyIncome
FROM DimCustomer
WHERE YearlyIncome > (
    SELECT
        AVG(YearlyIncome)
    FROM DimCustomer
    WHERE CustomerType = 'Person'
)
AND CustomerType = 'Person'
ORDER BY YearlyIncome DESC
/*
```

5. A ação de desconto da Asian Holiday Promotion foi uma das mais bem sucedidas da empresa. Agora, a Contoso quer entender um pouco melhor sobre o perfil dos clientes que compraram produtos com essa promoção.

Seu trabalho é criar uma query que retorne a lista de clientes que compraram nessa promoção.

*/

```
SELECT
    CustomerKey,
    FirstName,
    LastName,
    EmailAddress
FROM DimCustomer
WHERE CustomerKey IN(
    SELECT
        CustomerKey
    FROM FactOnlineSales
    WHERE PromotionKey IN (
        SELECT
            PromotionKey
        FROM DimPromotion
        WHERE PromotionName = 'Asian Holiday Promotion'
    )
)
/*
```

6. A empresa implementou um programa de fidelização de clientes empresariais. Todos aqueles que comprarem mais de 3000 unidades de um mesmo produto receberá descontos em outras compras.

Você deverá descobrir as informações de CustomerKey e CompanyName destes clientes.

```
*/
SELECT
    CustomerKey,
    CompanyName
FROM DimCustomer
WHERE CustomerKey IN (
    SELECT
        CustomerKey
    FROM FactOnlineSales
    GROUP BY CustomerKey, ProductKey
    HAVING COUNT(*) >= 3000
)
/*
```

7. Você deverá criar uma consulta para o setor de vendas que mostre as seguintes colunas da tabela DimProduct:

ProductKey,
ProductName,
BrandName,
UnitPrice
Média de UnitPrice.

```
*/
SELECT
    ProductKey,
    ProductName,
    BrandName,
    UnitPrice,
    (SELECT AVG(UnitPrice) FROM DimProduct) AS 'Média de UnitPrice'
FROM DimProduct
/*
```

8. Faça uma consulta para descobrir os seguintes indicadores dos seus produtos:

Maior quantidade de produtos por marca
Menor quantidade de produtos por marca
Média de produtos por marca

```
*/
SELECT
    MAX(Quantidade) AS 'Máximo',
    MIN(Quantidade) AS 'Mínimo',
    AVG(Quantidade) AS 'Média'
FROM(
    SELECT
        BrandName,
        COUNT(*) AS 'Quantidade'
    FROM DimProduct
    GROUP BY BrandName
) AS T
/*
```

9. Crie uma CTE que seja o agrupamento da tabela DimProduct, armazenando o total de produtos por marca. Em seguida, faça um SELECT nesta CTE, descobrindo qual é a quantidade máxima de produtos para uma marca. Chame esta CTE de CTE_QtdProdutosPorMarca.

```
*/
GO
WITH CTE_QtdProdutosPorMarca AS(
    SELECT
        BrandName,
        COUNT(*) AS 'Quantidade'
    FROM DimProduct
    GROUP BY BrandName
)
SELECT MAX(Quantidade) FROM CTE_QtdProdutosPorMarca
/*
```

10. Crie duas CTEs:

(i) a primeira deve conter as colunas ProductKey, ProductName, ProductSubcategoryKey, BrandName e UnitPrice, da tabela DimProduct, mas apenas os produtos da marca Adventure Works. Chame essa CTE de CTE_ProdutosAdventureWorks.

(ii) a segunda deve conter as colunas ProductSubcategoryKey, ProductSubcategoryName, da tabela DimProductSubcategory mas apenas para as subcategorias 'Televisions' e 'Monitors'. Chame essa CTE de CTE_CategoriaTelevisionsERadio.

Faça um Join entre essas duas CTEs, e o resultado deve ser uma query contendo todas as colunas das duas tabelas. Observe nesse exemplo a diferença entre o LEFT JOIN e o INNER JOIN.

```
*/
GO
WITH CTE_ProdutosAdventureWorks AS(
    SELECT
        ProductKey,
        ProductName,
        ProductSubcategoryKey,
        BrandName,
        UnitPrice
    FROM DimProduct
    WHERE BrandName = 'Adventure Works'
),
CTE_CategoriaTelevisionsERadio AS(
    SELECT
        ProductSubcategoryKey,
        ProductSubcategoryName
    FROM DimProductSubcategory
    WHERE ProductSubcategoryName IN ('Televisions', 'Monitors')
)
SELECT
    CTE_ProdutosAdventureWorks.*,
    CTE_CategoriaTelevisionsERadio.ProductSubcategoryName
FROM CTE_ProdutosAdventureWorks
    LEFT JOIN CTE_CategoriaTelevisionsERadio
        ON CTE_ProdutosAdventureWorks.ProductSubcategoryKey =
CTE_CategoriaTelevisionsERadio.ProductSubcategoryKey
```

-- MOD-16 - LOOPs no SQL

-- [SQL Server] Loops no SQL

-- Loop While

-- 1. Crie um contador que faça uma contagem de 1 até 10 utilizando a estrutura de repetição WHILE

```
DECLARE @vContador INT
SET @vContador = 1

WHILE @vContador <= 10
BEGIN
    PRINT 'O valor do contador é: ' + CONVERT(VARCHAR, @vContador)
    SET @vContador = @vContador + 1
END
```

-- 2. Cuidado com loops infinitos!!!

-- SOLUÇÃO

```
DECLARE @vContador INT
SET @vContador = 1

WHILE @vContador <= 5
BEGIN
    PRINT 'Alguém pare esse contador!!!'
END
```

```

-- SOLUÇÃO
DECLARE @vContador INT
SET @vContador = 1

WHILE @vContador <= 5
BEGIN
    PRINT 'Alguém pare esse contador!!!'
    SET @vContador = @vContador + 1
END

-- 3. Faça um contador de 1 a 100. OBS: Se o valor do contador for igual a 15, então o
loop WHILE deve ser encerrado

DECLARE @vContador INT
SET @vContador = 1

WHILE @vContador <= 100
BEGIN
    PRINT 'O valor do contador é: ' + CONVERT(VARCHAR, @vContador)
    IF @vContador = 15
        BREAK
    SET @vContador = @vContador + 1
END

-- 4. Faça um contador de 1 a 100. OBS: Os números 3 ou 6 não podem ser printados na
tela

DECLARE @varContador INT
SET @varContador = 0

WHILE @varContador <= 10
BEGIN
    SET @varContador += 1
    IF @varContador = 3 OR @varContador = 6
        CONTINUE
    PRINT @varContador
END

```

/*

MOD-16 – EXERCÍCIOS LOOPS

1. Utilize o Loop While para criar um contador que comece em um valor inicial @ValorInicial e termine em um valor final @ValorFinal. Você deverá printar na tela a seguinte frase:

“O valor do contador é: “ + ____

*/

```

DECLARE @valorInicial INT
DECLARE @valorFinal INT

```

```

SET @valorInicial = 1
SET @valorFinal = 100

```

```

WHILE @valorInicial <= @valorFinal
BEGIN
    PRINT CONCAT('O valor do contador é: ', @valorInicial)
    SET @valorInicial = @valorInicial + 1
END

```

/*

2. Você deverá criar uma estrutura de repetição que printe na tela a quantidade de contratações para cada ano, desde 1996 até 2003. A informação de data de contratação encontra-se na coluna HireDate da tabela DimEmployee. Utilize o formato:

X contratações em 1996

Y contratações em 1997

```

Z contratações em 1998
...
...
N contratações em 2003
Obs: a coluna HireDate contém a data completa (dd/mm/aaaa). Lembrando que você deverá
printar a quantidade de contratações por ano.
*/
USE ContosoRetailDW
DECLARE @anoInicial INT = 1996
DECLARE @anoFinal INT = 2003

WHILE @anoInicial <= @anoFinal
BEGIN
    DECLARE @QtdFuncionarios INT = (SELECT COUNT(*) FROM DimEmployee
                                     WHERE YEAR(HireDate) =
@anoInicial)
    PRINT CONCAT(@QtdFuncionarios, ' contratações em: ', @anoInicial)
    SET @anoInicial = @anoInicial +1
END
/*
3. Utilize um Loop While para criar uma tabela chamada Calendario, contendo uma coluna
que comece com a data 01/01/2021 e vá até 31/12/2021.
*/
USE Exercicios

CREATE TABLE Calendario (
    Data DATE
)

DECLARE @DataInicio DATE = '01/01/2021'
DECLARE @DataFim DATE = '31/12/2021'

WHILE @DataInicio <= @DataFim
BEGIN
    INSERT INTO Calendario (Data) VALUES(@DataInicio)
    SET @DataInicio = DATEADD(DAY, 1, @DataInicio)
END

SELECT * FROM Calendario

/*
MOD-17 - Funcoes_de_Janela
*/

```

```

-- Funções de Janela
-- CÁLCULO DO PERCENTUAL TOTAL
-- a) Calcule o % de participação de cada loja em relação ao total de vendas de todas
as lojas.
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) OVER() AS 'Total Vendido',
    FORMAT(Qtd_Vendida/SUM(Qtd_Vendida) OVER(), '0.00%') AS '% Por Região'
FROM Lojas
ORDER BY ID_Loja

-- b) Calcule o % de participacao de cada loja em relação ao total de vendas de cada
região.
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,

```

```

        Qtd_Vendida,
        SUM(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Total Vendido',
        FORMAT(Qtd_Vendida/SUM(Qtd_Vendida) OVER(PARTITION BY Regiao), '0.00%') AS '%'
Por Região'
FROM Lojas
ORDER BY ID_Loja

-- Funções de Janela
-- Cálculo de soma móvel e média móvel
USE WF

CREATE TABLE Resultado(
Data_Fechamento DATETIME,
Faturamento_MM FLOAT)

INSERT INTO Resultado(Data_Fechamento, Faturamento_MM)
VALUES
('01/01/2020', 8),
('01/02/2020', 10),
('01/03/2020', 6),
('01/04/2020', 9),
('01/05/2020', 5),
('01/06/2020', 4),
('01/07/2020', 7),
('01/08/2020', 11),
('01/09/2020', 9),
('01/10/2020', 12),
('01/11/2020', 11),
('01/12/2020', 10)

SELECT * FROM Resultado

-- Soma acumulada
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS 'Fat. Acumulado (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]

-- Média acumulada
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    AVG(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS 'Fat. Acumulado (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]

-- Acumulado
SELECT
    Data_Fechamento,
    -- Mes_Ano,
    Faturamento_MM,
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 3 PRECEDING AND 3
FOLLOWING) AS 'Soma Móvel'
FROM Resultado

-- Funções de Janela
-- Cálculo de soma móvel e média móvel
CREATE TABLE Resultado(
Data_Fechamento DATETIME,
Faturamento_MM FLOAT)

```

```

INSERT INTO Resultado(Data_Fechamento, Faturamento_MM)
VALUES
    ('01/01/2020', 8),
    ('01/02/2020', 10),
    ('01/03/2020', 6),
    ('01/04/2020', 9),
    ('01/05/2020', 5),
    ('01/06/2020', 4),
    ('01/07/2020', 7),
    ('01/08/2020', 11),
    ('01/09/2020', 9),
    ('01/10/2020', 12),
    ('01/11/2020', 11),
    ('01/12/2020', 10)

SELECT * FROM Resultado

-- Soma móvel
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 1 PRECEDING AND
CURRENT ROW) AS 'Fat. Acumulado (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]

-- Média móvel
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    AVG(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 1 PRECEDING AND
CURRENT ROW) AS 'Fat. Acumulado (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]

-- Funções de Janela
-- FUNÇÕES DE OFFSET (Deslocamento): LAG e LEAD
-- Cálculo MoM
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    FORMAT(((Faturamento_MM/NULLIF(LAG(Faturamento_MM, 1, 0) OVER(ORDER BY
Data_Fechamento), 0)) - 1, '0.00%') AS 'Crescimento MoM'
FROM Resultado
ORDER BY [Data do Fechamento]

-- Funções de Janela
-- Funções de Agregação: SUM, COUNT, AVG, MIN, MAX

-- 1. Crie uma coluna contendo a SOMA total das vendas da tabela lojas
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) OVER() AS 'Total Vendido'
FROM Lojas
ORDER BY ID_Loja

SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,

```

```

        Qtd_Vendida,
        SUM(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Total Vendido'
FROM Lojas
ORDER BY ID_Loja

```

-- 2. Crie uma coluna contendo a CONTAGEM das vendas da tabela lojas

```

SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    COUNT(*) OVER() AS 'Qtd. Lojas'
FROM Lojas
ORDER BY ID_Loja

```

```

SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    COUNT(*) OVER(PARTITION BY Regiao) AS 'Qtd. Lojas'
FROM Lojas
ORDER BY ID_Loja

```

-- 3. Crie uma coluna contendo a MÉDIA das vendas da tabela lojas

```

SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    AVG(Qtd_Vendida) OVER() AS 'Qtd. Lojas'
FROM Lojas
ORDER BY ID_Loja

```

```

SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    AVG(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Qtd. Lojas'
FROM Lojas
ORDER BY ID_Loja

```

-- 4. Crie uma coluna contendo o MIN/MAX das vendas da tabela lojas

```

SELECT
    *,
    MIN(Qtd_Vendida) OVER() AS 'Qtd. Lojas'
FROM Lojas
ORDER BY ID_Loja

```

```

SELECT
    *,
    MIN(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Qtd. Lojas'
FROM Lojas
ORDER BY ID_Loja

```

-- Funções de Janela

-- Funções de Classificação: ROW_NUMBER, RANK, DENSE_RANK, NTILE

```

SELECT
    *,
    ROW_NUMBER() OVER(ORDER BY Qtd_Vendida DESC) AS 'rownumber',
    RANK() OVER(ORDER BY Qtd_Vendida DESC) AS 'rank',
    DENSE_RANK() OVER(ORDER BY Qtd_Vendida DESC) AS 'dense',
    NTILE(3) OVER(ORDER BY Qtd_Vendida DESC) AS 'ntile'
FROM

```

```

        Lojas
WHERE Regiao = 'Sudeste'
--ORDER BY Qtd_Vendida DESC

-- Funções de Janela
-- Funções de Classificação: ROW_NUMBER, RANK, DENSE_RANK, NTILE
SELECT
    *,
    ROW_NUMBER() OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'rownumber',
    RANK() OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'rank',
    DENSE_RANK() OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'dense',
    NTILE(3) OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'ntile'
FROM
    Lojas
--ORDER BY Qtd_Vendida DESC

-- Funções de Janela
-- FUNÇÕES DE OFFSET (Deslocamento): FIRST_VALUE e LAST_VALUE
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    FIRST_VALUE(Faturamento_MM) OVER(ORDER BY Data_Fechamento) AS 'Primeiro valor',
    LAST_VALUE(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN UNBOUNDED
PRECEDING AND UNBOUNDED FOLLOWING) AS 'Último valor'
FROM Resultado
ORDER BY [Data do Fechamento]

-- Funções de Janela
-- FUNÇÕES DE OFFSET (Deslocamento): LAG e LEAD
USE WF

SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Fat. Mês Anterior',
    LEAD(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Fat. Mês Seguinte'
FROM Resultado
ORDER BY [Data do Fechamento]

-- Funções de Janela
-- Funções de Classificação: Ranking com GROUP BY
-- Crie uma tabela com o total de vendas por região e adicione uma coluna de ranking
nessa tabela
USE WF

SELECT
    Regiao AS 'Região',
    SUM(Qtd_Vendida) AS 'Total Vendido',
    RANK() OVER(ORDER BY SUM(Qtd_Vendida) DESC) AS 'Rank'
FROM
    Lojas
GROUP BY Regiao
ORDER BY Rank

-- Criar Banco de Dados WF
CREATE DATABASE WF

-- Usar o Banco WF
USE WF

--Criar a Tabela Lojas
CREATE TABLE Lojas(
ID_Loja INT,
Nome_Loja VARCHAR(100),
Regiao VARCHAR(100),

```

Qtd_Vendida FLOAT)

-- Inserir registros na Tabela Lojas

```
INSERT INTO Lojas(ID_Loja, Nome_Loja, Regiao, Qtd_Vendida)
VALUES
```

```
(1, 'Botafogo Praia&Mar', 'Sudeste', 1800),
(2, 'Lojas Vitoria', 'Sudeste', 800),
(3, 'Emporio Mineirinho', 'Sudeste', 2300),
(4, 'Central Paulista', 'Sudeste', 1800),
(5, 'Rio 90 graus', 'Sudeste', 700),
(6, 'Casa Flor & Anópolis', 'Sul', 2100),
(7, 'Pampas & Co', 'Sul', 990),
(8, 'Paraná Papéis', 'Sul', 2800),
(9, 'Amazonas Prime', 'Norte', 4200),
(10, 'Pará Bens', 'Norte', 3200),
(11, 'Tintas Rio Branco', 'Norte', 1500),
(12, 'Nordestemido Hall', 'Nordeste', 1910),
(13, 'Cachoerinha Loft', 'Nordeste', 2380)
```

```
SELECT * FROM Lojas
```

```
SELECT SUM(Qtd_Vendida) FROM Lojas
```

/*

MOD-17-Exercicios-Funcoes_de_Janela

EXERCÍCIOS WINDOW FUNCTIONS

Para resolver os exercícios 1 a 4, crie uma View chamada vwProdutos, que contenha o agrupamento das colunas BrandName, ColorName e os totais de quantidade vendida por marca/cor e também o total de receita por marca/cor.

*/

```
DROP VIEW vwProdutos
```

```
CREATE VIEW vwProdutos AS
```

```
SELECT
```

```
    BrandName AS 'Marca',
    ColorName AS 'Cor',
    COUNT(*) AS 'Quantidade_Vendida',
    ROUND(SUM(SalesAmount), 2) AS 'Receita_Total'
```

```
FROM DimProduct
```

```
    INNER JOIN FactSales
```

```
        ON DimProduct.ProductKey = FactSales.ProductKey
```

```
GROUP BY BrandName, ColorName
```

```
SELECT * FROM vwProdutos
```

```
ORDER BY Marca
```

/*

1. Utilize a View vwProdutos para criar uma coluna extra calculando a quantidade total vendida dos produtos.

*/

```
SELECT
```

```
    *,
    SUM(Quantidade_Vendida) OVER() AS 'Qtd. Total Vendida'
```

```
FROM vwProdutos
```

```
ORDER BY Marca
```

/*

2. Crie mais uma coluna na consulta anterior, incluindo o total de produtos vendidos para cada marca.

*/

```
SELECT
```

```
    *,
    SUM(Quantidade_Vendida) OVER() AS 'Qtd. Total Vendida',
```

```

SUM(Quantidade_Vendida) OVER(PARTITION BY Marca) AS 'Qtd. Total Vendida(Por
Marca)'
FROM vwProdutos
ORDER BY Marca
/*

```

3. Calcule o % de participação do total de vendas de produtos por marca.

Ex: A marca A. Datum teve uma quantidade total de vendas de 199.041 de um total de 3.406.089 de vendas. Isso significa que a da marca A. Datum é $199.041/3.406.089 = 5,84\%$.

```

*/
SELECT
    *,
    SUM(Quantidade_Vendida) OVER() AS 'Qtd. Total Vendida',
    SUM(Quantidade_Vendida) OVER(PARTITION BY Marca) AS 'Qtd. Total Vendida(Por
Marca)',
    SUM(Quantidade_Vendida) OVER(PARTITION BY Marca)/SUM(Quantidade_Vendida) OVER()
AS '% Participação'
FROM vwProdutos
ORDER BY Marca
-- Preencher a participação
SELECT
    *,
    SUM(Quantidade_Vendida) OVER() AS 'Qtd. Total Vendida',
    SUM(Quantidade_Vendida) OVER(PARTITION BY Marca) AS 'Qtd. Total Vendida(Por
Marca)',
    FORMAT(1.0*SUM(Quantidade_Vendida) OVER(PARTITION BY
Marca)/SUM(Quantidade_Vendida) OVER(), '0.00%') AS '% Participação'
FROM vwProdutos
ORDER BY Marca

```

4. Crie uma consulta à View vwProdutos, selecionando as colunas Marca, Cor, Quantidade_Vendida e também criando uma coluna extra de Rank para descobrir a posição de cada Marca/Cor. Você deve obter o resultado abaixo. Obs: Sua consulta deve ser filtrada para que seja mostrada apenas a marca Contoso.

```

*/
SELECT
    Marca,
    Cor,
    Quantidade_Vendida,
    RANK() OVER(ORDER BY Quantidade_Vendida DESC) AS 'Rank'
FROM vwProdutos
WHERE Marca = 'Contoso'

```

5. A partir da view criada no exercício anterior, você deverá fazer uma soma móvel considerando sempre o mês atual + 2 meses para trás.

```

*/
SELECT
    *,
    SUM(Qtd_Lojas) OVER(ORDER BY ID ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
FROM vwHistoricoLojas
/*

```

6. Utilize a vwHistoricoLojas para calcular o acumulado de lojas abertas a cada ano/mês.

```

*/
SELECT
    *,
    SUM(Qtd_Lojas) OVER(ORDER BY ID ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS 'Acumulado'
FROM vwHistoricoLojas

```

/*

7.

a) Faça um cálculo de soma móvel de novos clientes nos últimos 2 meses.

b) Faça um cálculo de média móvel de novos clientes nos últimos 2 meses.
c) Faça um cálculo de acumulado dos novos clientes ao longo do tempo.
d) Faça um cálculo de acumulado intra-ano, ou seja, um acumulado que vai de janeiro a dezembro de cada ano, e volta a fazer o cálculo de acumulado no ano seguinte.
*/

/*

8. Faça os cálculos de MoM e YoY para avaliar o percentual de crescimento de novos clientes, entre o mês atual e o mês anterior, e entre um mês atual e o mesmo mês do ano anterior.

Solução

*/

-- 1.

SELECT

```
*,
SUM(Quantidade_Por_Produto) OVER() AS 'Qtd Total Produtos'
FROM vwProdutos
ORDER BY Marca
```

-- 2.

SELECT

```
*,
SUM(Quantidade_Por_Produto) OVER() AS 'Qtd Total Produtos',
SUM(Quantidade_Por_Produto) OVER(PARTITION BY Marca) AS 'Qtd Total Por Marca'
FROM vwProdutos
ORDER BY Marca
```

-- 3.

SELECT

```
*,
SUM(Quantidade_Por_Produto) OVER() AS 'Qtd1',
SUM(Quantidade_Por_Produto) OVER(PARTITION BY Marca) AS 'Qtd2',
FORMAT(1.0*(SUM(Quantidade_Por_Produto) OVER(PARTITION BY
Marca))/SUM(Quantidade_Por_Produto) OVER(), '0.00%')
FROM vwProdutos
ORDER BY Marca
```

-- 4.

SELECT

```
*,
RANK() OVER(ORDER BY Quantidade_Por_Produto DESC) AS 'Rank'
FROM vwProdutos
WHERE Marca = 'Contoso'
```

-- Exercício Desafio 1

CREATE VIEW vwHistoricoLojas AS

SELECT

```
ROW_NUMBER() OVER(ORDER BY CalendarMonth) AS 'ID',
CalendarYear AS 'Ano',
CalendarMonthLabel AS 'Mês',
COUNT(DimStore.OpenDate) AS 'Qtd_Lojas'
FROM DimDate
LEFT JOIN DimStore
ON DimDate.Datekey = DimStore.OpenDate
GROUP BY CalendarYear, CalendarMonthLabel, CalendarMonth
```

-- 5.

SELECT

```
*,
SUM(Qtd_Lojas) OVER(ORDER BY ID ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
FROM vwHistoricoLojas
```

-- 6.

SELECT

*,

```
SUM(Qtd_Lojas) OVER(ORDER BY ID ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
FROM vwHistoricoLojas
```

-- Exercício Desafio 2

--1) Passo 1: Criar o Banco de Dados Desafio

```
CREATE DATABASE Desafio
```

```
USE Desafio
```

--2) Passo 2: Criar uma tabela de datas entre o dia 1 de janeiro do ano com a primeira compra de cliente mais antiga até o dia 31 de dezembro do ano com a última compra

```
CREATE TABLE Calendario (
data DATE
)
```

```
DECLARE @varAnoInicial INT = YEAR((SELECT MIN(DateFirstPurchase) FROM
ContosoRetailDW.dbo.DimCustomer))
```

```
DECLARE @varAnoFinal INT = YEAR((SELECT MAX(DateFirstPurchase) FROM
ContosoRetailDW.dbo.DimCustomer))
```

```
DECLARE @varDataInicial DATE = DATEFROMPARTS(@varAnoInicial, 1, 1)
```

```
DECLARE @varDataFinal DATE = DATEFROMPARTS(@varAnoFinal, 12, 31)
```

```
WHILE @varDataInicial <= @varDataFinal
```

```
BEGIN
```

```
INSERT INTO Calendario(data) VALUES(@varDataInicial)
```

```
SET @varDataInicial = DATEADD(DAY, 1, @varDataInicial)
```

```
END
```

```
SELECT * FROM Calendario
```

--3) Passo 3: Criar colunas auxiliares na tabela Calendario

```
ALTER TABLE Calendario
```

```
ADD Ano INT,
```

```
Mes INT,
```

```
Dia INT,
```

```
AnoMes INT,
```

```
NomeMes VARCHAR(50)
```

--4) Passo 4: Adicionar os valores às colunas

```
UPDATE Calendario SET Ano = YEAR(data)
```

```
UPDATE Calendario SET Mes = MONTH(data)
```

```
UPDATE Calendario SET Dia = DAY(data)
```

```
UPDATE Calendario SET AnoMes = CONCAT(YEAR(data), FORMAT(MONTH(data), '00'))
```

```
UPDATE Calendario SET NomeMes =
```

```
CASE
```

```
WHEN MONTH(data) = 1 THEN 'Janeiro'
```

```
WHEN MONTH(data) = 2 THEN 'Fevereiro'
```

```
WHEN MONTH(data) = 3 THEN 'Março'
```

```
WHEN MONTH(data) = 4 THEN 'Abril'
```

```
WHEN MONTH(data) = 5 THEN 'Maio'
```

```
WHEN MONTH(data) = 6 THEN 'Junho'
```

```
WHEN MONTH(data) = 7 THEN 'Julho'
```

```
WHEN MONTH(data) = 8 THEN 'Agosto'
```

```
WHEN MONTH(data) = 9 THEN 'Setembro'
```

```
WHEN MONTH(data) = 10 THEN 'Outubro'
```

```
WHEN MONTH(data) = 11 THEN 'Novembro'
```

```
WHEN MONTH(data) = 12 THEN 'Dezembro'
```

```
END
```

--5) Passo 5: Crie uma View

```
CREATE VIEW vwNovosClientes AS
```

```
SELECT
```

```
ROW_NUMBER() OVER(ORDER BY AnoMes) AS 'ID',
```

```
Ano,
```

```
NomeMes,
```

```
COUNT(DimCustomer.DateFirstPurchase) AS 'Novos_Clientes'
```

```
FROM Calendario
```

```
LEFT JOIN ContosoRetailDW.dbo.DimCustomer
```

```
ON Calendario.data = DimCustomer.DateFirstPurchase
```

```
GROUP BY Ano, NomeMes, AnoMes
SELECT * FROM vwNovosClientes
```

-- 7.

```
SELECT
*,
SUM(Novos_Clientes) OVER(ORDER BY ID ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
'Soma Móvel (2 meses)',
AVG(Novos_Clientes) OVER(ORDER BY ID ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
'Média Móvel (2 meses)',
SUM(Novos_Clientes) OVER(ORDER BY ID ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AS 'Acumulado Total',
SUM(Novos_Clientes) OVER(PARTITION BY Ano ORDER BY ID ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW) AS 'Acumulado (YTD)'
FROM vwNovosClientes
```

-- 8. Vai ter que dividir em duas partes para corrigir o problema do CONVERT

```
SELECT
*,
FORMAT(CONVERT(FLOAT, Novos_Clientes)/NULLIF(LAG(Novos_Clientes, 1) OVER(ORDER BY ID),
0) - 1, '0.00%') AS '% MoM',
FORMAT(CONVERT(FLOAT, Novos_Clientes)/NULLIF(LAG(Novos_Clientes, 12) OVER(ORDER BY
ID), 0) - 1, '0.00%') AS '% YoY'
FROM vwNovosClientes
```

/*

MOD-17-Desafios_1_e_2-Funcoes_de_Janela

Exercício Desafio 1.

Para responder os próximos 2 exercícios, você precisará criar uma View auxiliar. Diferente do que foi feito anteriormente, você não terá acesso ao código dessa view antes do gabarito.

A sua view deve se chamar vwHistoricoLojas e deve conter um histórico com a quantidade de lojas abertas a cada Ano/Mês. Os desafios são:

- (1) Criar uma coluna de ID para essa View
- (2) Relacionar as tabelas DimDate e DimStore

Dicas:

1- A coluna de ID será criada a partir de uma função de janela. Você deverá se atentar a forma como essa coluna deverá ser ordenada, pensando que queremos visualizar uma ordem de Ano/Mês que seja: 2005/january, 2005/February... e não 2005/April, 2005/August...

2- As colunas Ano, Mês e Qtd_Lojas correspondem, respectivamente, às seguintes colunas: CalendarYear e CalendarMonthLabel da tabela DimDate e uma contagem da coluna OpenDate da tabela DimStore.

*/

```
SELECT * FROM DimDate
SELECT * FROM DimStore ORDER BY OpenDate
```

CREATE VIEW vwHistoricoLojas AS

```
SELECT
    ROW_NUMBER() OVER(ORDER BY CalendarMonth) AS 'ID',
    CalendarYear AS 'Ano',
    CalendarMonthLabel AS 'Mês',
    COUNT(OpenDate) AS 'Qtd_Lojas'
FROM DimDate
LEFT JOIN DimStore -- Poderia ser usado INNER mas estaria errado
    ON DimDate.DateKey = DimStore.OpenDate
GROUP BY CalendarYear, CalendarMonthLabel, CalendarMonth
```

```
SELECT * FROM vwHistoricoLojas
```

/*

Exercício Desafio 2

Neste desafio, você terá que criar suas próprias tabelas e views para conseguir resolver os exercícios 7 e 8. Os próximos exercícios envolverão análises de novos clientes. Para isso, será necessário criar uma nova tabela e uma nova view. Abaixo, temos um passo a passo para resolver o problema por partes.

PASSO 1: Crie um novo banco de dados chamado Desafio e selecione esse banco de dados criado.

```
*/
-- Criar o Banco de Dados(Desafio)
CREATE DATABASE Desafio
USE Desafio

/*
PASSO 2: Crie uma tabela de datas entre o dia 1 de janeiro do ano com a compra
(DateFirstPurchase) mais antiga e o dia 31 de dezembro do ano com a compra mais
recente.
Obs1: Chame essa tabela de Calendario.
Obs2: A princípio, essa tabela deve conter apenas 1 coluna, chamada data e do tipo
DATE.
*/
SELECT * FROM ContosoRetailDW.dbo.DimCustomer
ORDER BY DateFirstPurchase
-- Criar a Tabela Calendario
CREATE TABLE Calendario (
    data DATE
)
-- Criar datas na tabela Calendario(Executar da linha 51 a 63)
DECLARE @varAnoInicial INT = YEAR((SELECT MIN(DateFirstPurchase)
    FROM ContosoRetailDw.dbo.DimCustomer))
DECLARE @varAnoFinal INT = YEAR((SELECT MAX(DateFirstPurchase)
    FROM ContosoRetailDw.dbo.DimCustomer))

DECLARE @varDataInicial DATE = DATEFROMPARTS(@varAnoInicial, 1, 1)
DECLARE @varDataFinal DATE = DATEFROMPARTS(@varAnoFinal, 12, 31)

WHILE @varDataFinal <= @varDataFinal
BEGIN
    INSERT INTO Calendario(Data) VALUES(@varDataInicial)
    SET @varDataInicial = DATEADD(DAY, 1, @varDataInicial)
END
-- Verificar se foi criado
SELECT * FROM Calendario

/*
PASSO 3: Crie colunas auxiliares na tabela Calendario chamadas: Ano, Mes, Dia, AnoMes
e NomeMes. Todas do tipo INT.
*/
ALTER TABLE Calendario
ADD Ano INT,
    Mes INT,
    Dia INT,
    AnoMes INT,
    NomeMes VARCHAR(50)
-- Verificar se foi criado
SELECT * FROM Calendario

/*
PASSO 4: Adicione na tabela os valores de Ano, Mês, Dia, AnoMes e NomeMes (nome do mês
em português). Dica: utilize a instrução CASE para verificar o mês e retornar o nome
certo.
*/
UPDATE Calendario SET Ano = YEAR(data)
UPDATE Calendario SET Mes = MONTH(data)
UPDATE Calendario SET Dia = DAY(data)
UPDATE Calendario SET AnoMes = CONCAT(YEAR(data), FORMAT(MONTH(data), '00'))
UPDATE Calendario SET NomeMes =
```

```

CASE
    WHEN MONTH(data) = 1 THEN 'Janeiro'
    WHEN MONTH(data) = 2 THEN 'Fevereiro'
    WHEN MONTH(data) = 3 THEN 'Março'
    WHEN MONTH(data) = 4 THEN 'Abril'
    WHEN MONTH(data) = 5 THEN 'Maio'
    WHEN MONTH(data) = 6 THEN 'Junho'
    WHEN MONTH(data) = 7 THEN 'Julho'
    WHEN MONTH(data) = 8 THEN 'Agosto'
    WHEN MONTH(data) = 9 THEN 'Setembro'
    WHEN MONTH(data) = 10 THEN 'Outubro'
    WHEN MONTH(data) = 11 THEN 'Novembro'
    WHEN MONTH(data) = 12 THEN 'Dezembro'
END
-- Verificar se foi criado
SELECT * FROM Calendario

/*
PASSO 5: Crie a View vwNovosClientes, que deve ter as colunas mostradas abaixo.
*/
CREATE VIEW vwNovosClientes AS
SELECT
    ROW_NUMBER() OVER(ORDER BY AnoMes) As 'ID',
    Ano,
    NomeMes,
    COUNT(DimCustomer.DateFirstPurchase) AS 'Novos_Clientes'
FROM Calendario
LEFT JOIN ContosoRetailDW.dbo.DimCustomer
    ON Calendario.data = DimCustomer.DateFirstPurchase
Group By Ano, NomeMes, AnoMes
-- Verificar se foi criado
SELECT * FROM vwNovosClientes

```

```

-- *****
/* COLLATION

```

O que é?

O COLLATION nos permite configurar se teremos diferenciação entre maiúsculas e minúsculas, ou entre palavras acentuadas.

O COLLATION pode ser definido em níveis diferentes no SQL Server. Abaixo estão os três níveis:

1. A nível SQL Server
2. A nível de Bancos de Dados
3. A nível de tabelas/colunas

1. A nível SQL Server

O COLLATION a princípio é definido durante a instalação do programa.

Por padrão, o COLLATION padrão é o seguinte:

Latin1_General_CI_AS

Onde CI significa Case Insensitive (não diferencia maiúsculas de minúsculas) e AS significa Accent Sensitive (sensível ao sotaque).

Para descobrir o COLLATION configurado, podemos utilizar o comando abaixo:*/

```

SELECT SERVERPROPERTY('collation')

```

```

/*2. A nível de Banco de Dados

```

Por padrão, todos os bancos de dados vão herdar a configuração do COLLATION do SQL Server feito durante a instalação.

Em Propriedades, conseguimos visualizar o COLLATION configurado.

Nós podemos também especificar o COLLATION do Banco de Dados no momento da sua criação.*/

```

CREATE DATABASE BD_Collation

```

```
COLLATE Latin1_General_CS_AS
```

```
/*Podemos também alterar o COLLATE após criar um banco de dados. Neste caso, usamos o comando abaixo:*/
```

```
ALTER DATABASE BD_Collation COLLATE Latin1_General_CI_AS
```

```
/*Para saber o COLLATION de um Banco de Dados específico, podemos usar o comando abaixo:*/
```

```
SELECT DATABASEPROPERTYEX('BD_Collation','collation')
```

```
/*3. A nível de Coluna/Tabela
```

```
Por padrão, uma nova coluna de tipo VARCHAR herda o COLLATION do banco de dados, a menos que você especifique o COLLATION explicitamente ao criar a tabela.
```

```
Para criar uma coluna com um COLLATION diferente, você pode especificar o agrupamento usando o comando Collate SQL.
```

```
*/
```

```
CREATE TABLE Nomes(  
    ID INT,  
    Nome1 VARCHAR(100),  
    Nome2 VARCHAR(100) COLLATE Latin1_General_CS_AS  
)
```

```
SELECT * FROM Nomes
```

```
/*Podemos ver o COLLATION de cada coluna da tabela usando o comando abaixo:*/
```

```
sp_help Nomes
```

```
-- *****  
-- COLLATE: Exemplo
```

```
CREATE DATABASE BD_Collation  
COLLATE SQL_Latin1_General_CP1_CI_AS
```

```
CREATE TABLE Tabela(  
    ID INT,  
    Nome VARCHAR(100) COLLATE SQL_Latin1_General_CP1_CS_AS  
)
```

```
INSERT INTO Tabela(ID, Nome)  
VALUES  
    (1, 'Matheus'), (2, 'Marcela'), (3, 'marcos'), (4, 'MAuricio'), (5, 'Marta'),  
    (6, 'Miranda'), (7, 'Melissa'), (8, 'Lucas'), (9, 'luisa'), (10, 'Pedro')
```

```
SELECT * FROM Tabela  
WHERE Nome = 'Marcela' -- Filtro ok  
-- WHERE Nome = 'marcela'
```

```
SELECT * FROM Tabela  
WHERE Nome COLLATE Latin1_General_CI_AS = 'marcela'
```

```
SELECT * FROM Tabela  
WHERE Nome COLLATE Latin1_General_CI_AS = 'Marcela'
```

```
-- LIKE: Case sensitive  
DROP TABLE Nomes
```

```
CREATE TABLE Nomes(  
    ID INT,  
    Nome VARCHAR(100) COLLATE SQL_Latin1_General_CP1_CS_AS)
```

```
INSERT INTO Nomes(ID, Nome)
```

```
VALUES
    (1, 'Matheus'), (2, 'Marcela'), (3, 'marcos'), (4, 'MAuricio'), (5, 'Marta'),
    (6, 'Miranda'), (7, 'Melissa'), (8, 'Lucas'), (9, 'luisa'), (10, 'Pedro')

SELECT * FROM Nomes

-- Case Sensitive (diferenciando maiúsculas de minúsculas)
-- LIKE padrão como aprendemos até agora:
SELECT *
FROM Nomes
WHERE Nome LIKE 'mar%'

-- Retorna as linhas onde a primeira letra seja 'm', a segunda seja 'a' e a terceira
seja 'r'
SELECT *
FROM Nomes
WHERE Nome LIKE '[m][a][r]%'

-- Retorna as linhas onde a primeira letra seja [M], a segunda seja 'a' e a terceira
seja 'r'
SELECT *
FROM Nomes
WHERE Nome LIKE '[M][a][r]%'

-- Retorna as linhas onde a primeira letra seja 'M' ou 'm', e a segunda seja 'A' ou
'a'
SELECT *
FROM Nomes
-- WHERE Nome LIKE '[Mm][Aa]%'
WHERE Nome LIKE '[M-m][A-a]%'

-- *****
```

MOD-19-Regex-Regular_Expressions

```
-- LIKE: Filtrando os primeiros caracteres + Case sensitive
USE BD_Collation

CREATE TABLE Textos(
    ID INT,
    Texto VARCHAR(100) COLLATE SQL_Latin1_General_CP1_CS_AS
)

INSERT INTO Textos(ID, Texto)
VALUES
    (1, 'Marcos'), (2, 'Excel'), (3, 'leandro'), (4, 'K'), (5, 'X7'), (6, 'l9'), (7,
'#M'), (8, '@9'), (9, 'M'), (10, 'RT')

SELECT * FROM Textos

-- Retornando nomes que começam com a letra 'M', 'E' ou 'K'
SELECT *
FROM Textos
WHERE Texto LIKE '[MEK]%'

-- Retornando nomes que possuem apenas 1 caracteres
SELECT *
FROM Textos
WHERE Texto LIKE '[A-z]'

-- Retornando nomes que possuem apenas 2 caracteres
SELECT *
FROM Textos
WHERE Texto LIKE '[A-z][A-z]'
```

```

-- Retornando nomes que possuem apenas 2 caracteres: o primeiro uma letra, o segundo
um número
SELECT *
FROM Textos
WHERE Texto LIKE '[A-z][0-9]'

-- LIKE: Aplicando um filtro ainda mais personalizado
CREATE TABLE Nomes(
ID INT,
Nome VARCHAR(100) COLLATE SQL_Latin1_General_CP1_CS_AS)

INSERT INTO Nomes(ID, Nome)
VALUES
(1, 'Matheus'), (2, 'Marcela'), (3, 'marcos'), (4, 'MAuricio'), (5, 'Marta'),
(6, 'Miranda'), (7, 'Melissa'), (8, 'Lucas'), (9, 'luisa'), (10, 'Pedro')

-- Retorna os nomes que:
-- 1. Começam com a letra 'M' ou 'm'
-- 2. O segundo caractere pode ser qualquer coisa ('_' é um curinga)
-- 3. O terceiro caractere pode ser a letra 'R' ou a letra 'r'
-- 4. Possui uma quantidade qualquer de caracteres depois do terceiro (por conta do
'%')

SELECT *
FROM Nomes
WHERE Nome LIKE '[Mm]_[Rr]%'

-- LIKE: Utilizando o operador de negação
CREATE TABLE Nomes(
ID INT,
Nome VARCHAR(100) COLLATE SQL_Latin1_General_CP1_CS_AS)

INSERT INTO Nomes(ID, Nome)
VALUES
(1, 'Matheus'), (2, 'Marcela'), (3, 'marcos'), (4, 'MAuricio'), (5, 'Marta'),
(6, 'Miranda'), (7, 'Melissa'), (8, 'Lucas'), (9, 'luisa'), (10, 'Pedro')

-- Retorna nomes que não começa com as letras 'L' ou 'l'
SELECT *
FROM Nomes
WHERE Nome LIKE '[^Ll]%'

-- Retorna nomes que não começa com as letras 'L' ou 'l'
SELECT *
FROM Nomes
WHERE Nome LIKE '_[^Ee]%'

-- *****
-- LIKE: Identificando caracteres especiais

USE BD_Collation
CREATE TABLE Textos(
ID INT,
Texto VARCHAR(100) COLLATE SQL_Latin1_General_CP1_CS_AS)

INSERT INTO Textos(ID, Texto)
VALUES
(1, 'Marcos'), (2, 'Excel'), (3, 'leandro'), (4, 'K'), (5, 'X7'), (6, 'l9'), (7,
'#M'), (8, '@9'), (9, 'M'), (10, 'RT')

-- Identificando caracteres especiais

```

```

SELECT *
FROM Textos
WHERE Texto LIKE '%[^A-z0-9]%'

-- *****
-- LIKE: Aplicações com números
USE BD_Collation

CREATE TABLE Numeros(
    Numero DECIMAL(20, 2)
)

INSERT INTO Numeros(Numero)
VALUES
    (50), (30.23), (9), (100.54), (15.9), (6.5), (10), (501.76), (1000.56), (31)

SELECT * FROM Numeros

-- Retornando os números que possuem 2 dígitos na parte inteira
SELECT *
FROM Numeros
WHERE Numero LIKE '[0-9][0-9].[0][0]'

-- Retornando linhas que:
-- 1. Possuem 3 dígitos na parte inteira, sendo o primeiro dígito igual a 5
-- 2. O primeiro número da parte decimal seja 7.
-- 3. O segundo número da parte decimal seja um número entre 0 e 9.
SELECT *
FROM Numeros
WHERE Numero LIKE '[5]__.[7][0-9]'

```

/*

MOD-20-Constraints

Objetivo: Entender o que são constraints e como trabalhar com elas na prática.
*/

```

-- I. CONSTRAINTS
-- Constraints no SQL são regras (restrições) que podemos definir para uma coluna de
-- uma tabela.
-- Abaixo temos uma lista de restrições:

-- 1. NOT NULL
-- 2. UNIQUE
-- 3. CHECK
-- 4. DEFAULT
-- 5. IDENTITY
-- 6. PRIMARY KEY
-- 7. FOREIGN KEY

-- 1. NOT NULL
-- Essa constraint não permite que sejam adicionados valores nulos na coluna.

-- 2. UNIQUE
-- Identifica uma coluna de forma única, sem permitir valores duplicados (mas, permite
-- NULL).

-- 3. CHECK
-- Verifica se o valor adicionado na coluna atende a uma determinada condição.

-- 4. DEFAULT
-- Retorna um valor default caso a coluna não seja preenchida.

```

```

-- 5. IDENTITY
-- Permite que uma coluna siga uma auto numeração (usada em colunas de ID).

-- 6. PRIMARY KEY
-- Uma CHAVE PRIMÁRIA (PRIMARY KEY) é uma coluna que identifica de forma única as
linhas
-- de uma tabela. Nenhum dos valores de uma coluna de chave primária deve ser nulo ou
se repetir.
-- Será através dessa coluna que criaremos relações entre as tabelas.

-- 7. FOREIGN KEY
-- Uma CHAVE ESTRANGEIRA (FOREIGN KEY) é uma coluna que será relacionada com a CHAVE
PRIMÁRIA
-- de uma outra tabela.

-- Deletar BD
DROP DATABASE Exercicios

-- Criar BD
CREATE DATABASE Exercicios

-- Usar BD
USE Exercicios

-- Apagar ou Deletar Tabelas, quando o BD existe de outros exercicios
DROP TABLE fContratos
DROP TABLE dCliente
DROP TABLE dGerente

-- Tabela 1: dCliente

-- A tabela dCliente deve conter as seguintes colunas:

-- Coluna 1: id_cliente do tipo INT          --> Chave Primária e deve ser auto
incrementada
-- Coluna 2: nome_cliente do tipo VARCHAR   --> Não aceita valores nulos
-- Coluna 3: genero VARCHAR                  --> Não aceita valores nulos e devem ser
('M', 'F', 'O', 'PND')
-- Coluna 4: data_nascimento DATE            --> Não aceita valores nulos
-- Coluna 5: cpf do tipo VARCHAR             --> Não aceita valores duplicados nem
valores nulos

-- Cria Tabela
CREATE TABLE dCliente(
    id_cliente INT IDENTITY(1, 1),
    nome_cliente VARCHAR(100) NOT NULL,
    genero VARCHAR(100) NOT NULL,
    data_de_nascimento DATE NOT NULL,
    cpf VARCHAR(100) NOT NULL,
    CONSTRAINT dcliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT dcliente_genero_ck CHECK(Genero IN ('M', 'F', 'O', 'PND')),
    CONSTRAINT dcliente_cpf_un UNIQUE(cpf)
)

-- Inserir dados na Tabela
INSERT INTO dCliente(Nome_Cliente, Genero, Data_de_Nascimento, CPF)
VALUES
    ('André Martins', 'M', '12/02/1989', '839.283.190-00'),
    ('Bárbara Campos', 'F', '07/05/1992', '351.391.410-02'),
    ('Carol Freitas', 'F', '23/04/1985', '139.274.921-12'),
    ('Diego Cardoso', 'M', '11/10/1994', '192.371.081-17'),
    ('Eduardo Pereira', 'M', '09/11/1988', '193.174.192-82'),
    ('Fabiana Silva', 'F', '02/09/1989', '231.298.471-98'),
    ('Gustavo Barbosa', 'M', '27/06/1993', '240.174.171-76'),
    ('Helen Viana', 'F', '11/02/1990', '193.129.183-01'),
    ('Igor Castro', 'M', '21/08/1989', '184.148.102-29'),

```

```
('Juliana Pires', 'F', '13/01/1991', '416.209.192-47')
```

```
SELECT * FROM dCliente
```

```
-- Tabela 2: dGerente
```

```
-- A tabela dGerente deve conter as seguintes colunas:
```

```
-- Coluna 1: id_gerente do tipo INT          --> Chave Primária e auto incrementada
-- Coluna 2: nome_gerente do tipo VARCHAR    --> Não aceita valores nulos
-- Coluna 3: data_contratacao VARCHAR        --> Não aceita valores nulos
-- Coluna 4: salario do tipo FLOAT          --> Não aceita valores nulos nem abaixo
de zero
```

```
CREATE TABLE dGerente(
    id_gerente INT IDENTITY(1, 1),
    nome_gerente VARCHAR(100) NOT NULL,
    data_contratacao VARCHAR(100) NOT NULL,
    salario FLOAT NOT NULL,
    CONSTRAINT dgerente_id_gerente_pk PRIMARY KEY(id_gerente),
    CONSTRAINT dgerente_salario_ck CHECK(salario > 0)
)
```

```
INSERT INTO dGerente(Nome_Gerente, Data_Contratacao, Salario)
VALUES
```

```
('Lucas Sampaio', '21/03/2015', 6700),
('Mariana Padilha', '10/01/2011', 9900),
('Nathália Santos', '03/10/2018', 7200),
('Otávio Costa', '18/04/2017', 11000)
```

```
SELECT * FROM dGerente
```

```
-- Tabela 3: fContratos
```

```
-- A tabela fContratos deve conter as seguintes colunas:
```

```
-- Coluna 1: id_contrato do tipo INT          --> Chave Primária e auto incremental
-- Coluna 2: data_assinatura do tipo DATE      --> Valor Padrão (GETDATE) caso não
seja preenchida
-- Coluna 3: id_cliente do tipo INT           --> Chave Estrangeira
-- Coluna 4: id_gerente do tipo INT           --> Chave Estrangeira
-- Coluna 5: valor_contrato do tipo FLOAT     --> Não aceita valores nulos e deve ser
maior que zero
```

```
CREATE TABLE fContratos(
    id_contrato INT IDENTITY(1, 1),
    data_assinatura DATE DEFAULT GETDATE(),
    id_cliente INT,
    id_gerente INT,
    valor_contrato FLOAT,
    CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato),
    CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES
dCliente(id_cliente),
    CONSTRAINT fcontratos_id_gerente_fk FOREIGN KEY(id_gerente) REFERENCES
dGerente(id_gerente),
    CONSTRAINT fcontratos_valor_contrato_ck CHECK(valor_contrato > 0)
)
```

```
INSERT INTO fContratos(Data_Assinatura, ID_Cliente, ID_Gerente, Valor_Contrato)
VALUES
```

```
('12/01/2019', 8, 1, 23000),
('10/02/2019', 3, 2, 15500),
('07/03/2019', 7, 2, 6500),
('15/03/2019', 1, 3, 33000),
('21/03/2019', 5, 4, 11100),
```

```

('23/03/2019', 4, 2, 5500),
('28/03/2019', 9, 3, 55000),
('04/04/2019', 2, 1, 31000),
('05/04/2019', 10, 4, 3400),
('05/04/2019', 6, 2, 9200)

```

```
SELECT * FROM fContratos
```

```
-- Violação de constraints
```

```
SELECT * FROM dCliente;
```

```
-- Exemplo 1: Violação NOT NULL e CHECK(Genero: F ou M)
```

```
INSERT INTO dCliente(Nome_Cliente, Genero, Data_de_Nascimento, CPF) VALUES
('Lucia Campos', 'F', '23/10/1994', '999.234.812-20');
```

```
-- Exemplo 2: Violação UNIQUE
```

```
SELECT * FROM dCliente
```

```
INSERT INTO dCliente(Nome_Cliente, Genero, Data_de_Nascimento, CPF) VALUES
('Lucia Campos', 'F', '23/10/1994', '184.148.102-29');
```

```
-- Exemplo 3: Violação de FK e PK
```

```
SELECT * FROM dCliente
```

```
SELECT * FROM dGerente
```

```
SELECT * FROM fContratos
```

```
INSERT INTO fContratos(Data_Assinatura, ID_Cliente, ID_Gerente, Valor_Contrato) VALUES
('12/03/2022', 4, 10, 1500);
```

```
-- III. Gerenciando CONSTRAINTS
```

```
-- 1. Adicionar constraints
```

```
-- 2. Renomear constraints
```

```
-- 3. Remover constraints
```

```
-- Remova a constraint PK da tabela fContratos.
```

```
ALTER TABLE fContratos
```

```
DROP CONSTRAINT fcontratos_id_contrato_pk
```

```
-- Remova a constraint FK Cliente da tabela fContratos.
```

```
ALTER TABLE fContratos
```

```
DROP CONSTRAINT fcontratos_id_cliente_fk
```

```
-- Adicione a constraint PK id_venda na tabela fContratos.
```

```
ALTER TABLE fContratos
```

```
ADD CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato)
```

```
-- Adicione a constraint FK id_cliente na tabela VENDAS.
```

```
ALTER TABLE fContratos
```

```
ADD CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES
dCliente(id_cliente)
```

```
-- Alterar o nome de uma constraint criada.
```

```
-- Com o botão direito sobre o campo da tabela opção 'Renomear'
```

```
/*
```

```
MÓDULO 20: EXERCÍCIOS - CONSTRAINTS
```

```
1. Você está responsável por criar um Banco de Dados com algumas tabelas que vão
armazenar informações associadas ao aluguel de carros de uma locadora.
```

```
a) O primeiro passo é criar um banco de dados chamado AlugaFacil.
```

```
*/
```

```
CREATE DATABASE AlugaFacil
```

```
USE AlugaFacil
```

/*
b) O seu banco de dados deve conter 3 tabelas e a descrição de cada uma delas é mostrada abaixo:

Obs: você identificará as restrições das tabelas a partir de suas descrições.

Tabela 1: Cliente

- id_cliente
- nome_cliente
- cnh
- cartao

*/

```
CREATE TABLE Cliente(  
    id_cliente INT IDENTITY(1, 1),  
    nome_cliente VARCHAR(100) NOT NULL,  
    cnh VARCHAR(100) NOT NULL,  
    cartao VARCHAR(100) NOT NULL,  
    CONSTRAINT cliente_id_cliente_pk PRIMARY KEY(id_cliente),  
    CONSTRAINT cliente_cnh_un UNIQUE(cnh)  
)
```

/*

A tabela Cliente possui 4 colunas.

A coluna id_cliente deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.

As colunas nome_cliente, cnh e cartao não podem aceitar valores nulos, ou seja, para todo cliente estes campos devem necessariamente ser preenchidos.

Por fim, a coluna cnh não pode aceitar valores duplicados.

Tabela 2: Carro

- id_carro
- placa
- modelo
- tipo

A tabela Carro possui 3 colunas.

A coluna id_carro deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.

As colunas modelo, tipo e placa não podem aceitar valores nulos.

Os tipos de carros cadastrados devem ser: Hatch, Sedan, SUV.

Por fim, a coluna placa não pode aceitar valores duplicados.

*/

```
CREATE TABLE Carro(  
    id_carro INT IDENTITY(1, 1),  
    placa VARCHAR(100) NOT NULL,  
    modelo VARCHAR(100) NOT NULL,  
    tipo VARCHAR(100) NOT NULL,  
    CONSTRAINT carro_id_carro_pk PRIMARY KEY(id_carro),  
    CONSTRAINT carro_placa_un UNIQUE(placa),  
    CONSTRAINT carro_tipo_ck CHECK(tipo IN('Hatch', 'SUV', 'Sedan'))  
)
```

/*

Tabela 3: Locacoes

- id_locacao
- data_locacao
- data_devolucao
- id_carro
- id_cliente

A tabela Locacoes possui 5 colunas.

A coluna id_locacao deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.

Nenhuma das demais colunas devem aceitar valores nulos.

As colunas id_carro e id_cliente são chaves estrangeiras que permitirão a relação da tabela Locacoes com as tabelas Carro e Cliente.

*/

```
CREATE TABLE Locacoes(  
    id_locacao INT IDENTITY(1, 1),  
    data_locacao DATE NOT NULL,  
    data_devolucao DATE NOT NULL,
```

```

        id_carro INT NOT NULL,
        id_cliente INT NOT NULL,
        CONSTRAINT locacoes_id_locacoes_pk PRIMARY KEY(id_locacao),
        CONSTRAINT locacoes_id_carro_pk FOREIGN KEY(id_carro)
            REFERENCES Carro(id_carro),
        CONSTRAINT locacoes_id_cliente_fk FOREIGN KEY(id_cliente)
            REFERENCES Cliente(id_cliente)
    )

```

```

SELECT * FROM Locacoes
SELECT * FROM Carro
SELECT * FROM Cliente

```

/*

2. Tente violar as constraints criadas para cada tabela. Este exercício é livre.

Obs: Para fazer o exercício de violação de constraints basta utilizar o comando INSERT INTO para adicionar valores nas tabelas que não respeitem as restrições (constraints) estabelecidas na criação das tabelas.

Ao final, exclua o banco de dados criado.

*/

-- ok

```

INSERT INTO Carro(placa, modelo, tipo)
VALUES('ABC-123', 'Hyundai HB20', 'Sedan')

```

-- Restrição de placa

```

INSERT INTO Carro(placa, modelo, tipo)
VALUES('ABC-123', 'Citroen Cactus', 'Sedan')

```

-- Restrição de tipo

```

INSERT INTO Carro(placa, modelo, tipo)
VALUES('GHI-456', 'Monza', 'Esportivo')

```

```

SELECT * FROM Carro

```

-- *****

/*

MOD-21-SEQUENCES

*/

-- V. SEQUENCES

-- O que é?

-- Uma sequência (Sequence) é um objeto que utilizamos para criação de números sequenciais automáticos. São usados especialmente para gerar valores sequenciais únicos para as chaves primárias das tabelas.

-- Dessa forma, não precisamos ficar preenchendo a sequência de ids manualmente (como fizemos

-- até então), podemos gerar automaticamente por meio de uma sequence.

/* Sintaxe

*

```

CREATE SEQUENCE nome_sequencia

```

```

AS tipo

```

```

START WITH n

```

```

INCREMENT BY n

```

```

MAXVALUE n | NO MAXVALUE

```

```

MINVALUE n | NO MINVALUE

```

```

CYCLE | NO CYCLE;      -- quando atinge o valor máximo, pode ou não voltar do começo

```

*/

-- Crie uma sequência para o id_cliente

```

CREATE SEQUENCE clientes_seq

```

```

AS INT
START WITH 1
INCREMENT BY 1
MAXVALUE 9999999
NO CYCLE

-- Próximo valor da sequência(Executar varias vezes e mostra a prox sequencia)
SELECT NEXT VALUE FOR clientes_seq

-- Excluir uma sequence
DROP SEQUENCE clientes_seq

-- *****
-- Usando a sequence na prática

-- Crie uma sequência para o id_projeto
CREATE SEQUENCE projetos_seq
AS INT
START WITH 1
INCREMENT BY 1
NO MAXVALUE
NO CYCLE

CREATE TABLE dProjeto(
    id_projeto INT,
    nome_projeto VARCHAR(100) NOT NULL,
    CONSTRAINT dareas_id_area_pk PRIMARY KEY(id_projeto)
)

SELECT * FROM dProjeto

INSERT INTO dProjeto(id_projeto, nome_projeto) VALUES
    (NEXT VALUE FOR projetos_seq, 'Planejamento Estratégico'),
    (NEXT VALUE FOR projetos_seq, 'Desenvolvimento de App'),
    (NEXT VALUE FOR projetos_seq, 'Plano de Negócios'),
    (NEXT VALUE FOR projetos_seq, 'Visualização 3D')

SELECT * FROM dProjeto

INSERT INTO dProjeto(id_projeto, nome_projeto) VALUES
    (NEXT VALUE FOR projetos_seq, 'Mapeamento de Processos')

SELECT * FROM dProjeto

```

/*

MÓDULO 21 – EXERCÍCIOS SEQUENCES

0. Crie o Banco de Dados AlugaFacil, onde serão criadas as sequences e tabelas dos exercícios.

```

*/
USE Exercicios
-- Apagar BD
DROP DATABASE AlugaFacil
-- Criar BD
CREATE DATABASE AlugaFacil
-- F5 e Usar BD
USE AlugaFacil

-- Criar Cliente Sequences
CREATE SEQUENCE clientes_seq
AS INT
START WITH 1
INCREMENT BY 1
NO MAXVALUE

```

```

-- Criar Carro Sequences
CREATE SEQUENCE carro_seq
AS INT
START WITH 1
INCREMENT BY 1
NO MAXVALUE

-- Criar Locações Sequences
CREATE SEQUENCE locacoes_seq
AS INT
START WITH 1
INCREMENT BY 1
NO MAXVALUE

DROP TABLE Locacoes
DROP TABLE Cliente
DROP TABLE Carro

-- Criar a Tabela Cliente Sem a IDENTITY(1, 1) no ID
CREATE TABLE Cliente(
    id_cliente INT,
    nome_cliente VARCHAR(100) NOT NULL,
    cnh VARCHAR(100) NOT NULL,
    cartao VARCHAR(100) NOT NULL,
    CONSTRAINT cliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT cliente_cnh_un UNIQUE(cnh)
)

SELECT * FROM Cliente

-- Criar a Tabela Carro
CREATE TABLE Carro(
    id_carro INT,
    placa VARCHAR(100) NOT NULL,
    modelo VARCHAR(100) NOT NULL,
    tipo VARCHAR(100) NOT NULL,
    CONSTRAINT carro_id_carro_pk PRIMARY KEY(id_carro),
    CONSTRAINT carro_placa_un UNIQUE(placa),
    CONSTRAINT carro_tipo_ck CHECK(tipo IN('Hatch', 'SUV', 'Sedan'))
)

SELECT * FROM Carro

-- Criar a Tabela Locações
CREATE TABLE Locacoes(
    id_locacao INT,
    data_locacao DATE NOT NULL,
    data_devolucao DATE NOT NULL,
    id_carro INT NOT NULL,
    id_cliente INT NOT NULL,
    CONSTRAINT locacoes_id_locacoes_pk PRIMARY KEY(id_locacao),
    CONSTRAINT locacoes_id_carro_pk FOREIGN KEY(id_carro)
        REFERENCES Carro(id_carro),
    CONSTRAINT locacoes_id_cliente_fk FOREIGN KEY(id_cliente)
        REFERENCES Cliente(id_cliente)
)

SELECT * FROM Locacoes
SELECT * FROM Carro
SELECT * FROM Cliente

/*
1. Vamos criar Sequences que serão utilizadas nas tabelas: Carro, Cliente e Locacoes.
Essas sequences serão chamadas de: cliente_seq, carro_seq e locacoes_seq.

```

Todas essas sequences devem começar pelo número 1, incrementar de 1 em 1 e não terem valor máximo.

*/

/*

2. Utilize as sequences nas 3 tabelas: Carro, Cliente e Locacoes. Você deve excluir as tabelas existentes e recriá-las. Lembre-se que não é necessário utilizar a constraint IDENTITY nas colunas de chave primária uma vez que nelas serão usadas as Sequences.

Tabela 1: Cliente

- id_cliente
- nome_cliente
- cnh
- cartao

A tabela Cliente possui 4 colunas.

A coluna id_cliente deve ser a chave primária da tabela.

As colunas nome_cliente, cnh e cartao não podem aceitar valores nulos, ou seja, para todo cliente estes campos devem necessariamente ser preenchidos.

Por fim, a coluna cnh não pode aceitar valores duplicados.

*/

```
INSERT INTO Cliente(id_cliente, nome_cliente, cnh, cartao) VALUES
  (NEXT VALUE FOR clientes_seq, 'Ana', '111111', '1111-1111-1111-1111'),
  (NEXT VALUE FOR clientes_seq, 'Jose', '222222', '22222-22222'),
  (NEXT VALUE FOR clientes_seq, 'Bruno', '333333', '3333333-33333333'),
  (NEXT VALUE FOR clientes_seq, 'Charles', '4444444', '4444444-44444444')
```

```
SELECT * FROM Cliente
```

/*

Tabela 2: Carro

- id_carro
- placa
- modelo
- tipo

A tabela Carro possui 3 colunas.

A coluna id_carro deve ser a chave primária da tabela.

As colunas modelo, tipo e placa não podem aceitar valores nulos.

Os tipos de carros cadastrados devem ser: Hatch, Sedan, SUV.

Por fim, a coluna placa não pode aceitar valores duplicados.

*/

```
INSERT INTO Carro(id_carro, placa, modelo, tipo) VALUES
  (NEXT VALUE FOR carro_seq, 'ABC-123', 'Hyundai HB20', 'Sedan'),
  (NEXT VALUE FOR carro_seq, 'DEF-123', 'FORD KA', 'SUV')
```

```
SELECT * FROM Carro
```

/*

Tabela 3: Locacoes

- id_locacao
- data_locacao
- data_devolucao
- id_carro
- id_cliente

A tabela Locacoes possui 5 colunas.

A coluna id_locacao deve ser a chave primária da tabela.

Nenhuma das demais colunas devem aceitar valores nulos.

As colunas id_carro e id_cliente são chaves estrangeiras que permitirão a relação da tabela Locacoes com as tabelas Carro e Cliente.

*/

```
INSERT INTO Locacoes(id_locacao, data_locacao, data_devolucao, id_carro, id_cliente)
VALUES
  (NEXT VALUE FOR locacoes_seq, '12/01/2025', '12/02/2025', 1, 3),
  (NEXT VALUE FOR locacoes_seq, '14/01/2025', '12/03/2025', 2, 4)
```

```
SELECT * FROM Locacoes
```

```
SELECT * FROM Carro
```

```

SELECT * FROM Cliente

/*
3. Exclua as sequences criadas.
*/
DROP SEQUENCE locacoes_seq
DROP SEQUENCE carro_seq
DROP SEQUENCE clientes_seq

SELECT * FROM Locacoes
SELECT * FROM Carro
SELECT * FROM Cliente

```

MOD – 22 – Transactions

```

-- 1. O que é uma transaction?
-- Uma transaction é uma ação realizada dentro do banco de dados. Essa ação pode ser
uma: atualização, inserção ou exclusão de dados do banco. Precisamos de transações
quando estamos alterando o banco de dados de alguma forma, seja inserindo, atualizando
ou excluindo dados.

-- Normalmente, não temos muito "controle" sobre transações, a menos que a gente
explícite no nosso código que queremos fazer isso. Assim, a ideia de uma transação é
agrupar um conjunto de instruções a serem executadas no banco de dados, e ter a
flexibilidade de:
-- a. Se algo der errado, desfazer aquela transação
-- b. Se tudo der certo, salvar aquela transação

-- O que podemos fazer com uma transaction?
-- BEGIN TRANSACTION      : iniciá-la
-- ROLLBACK TRANSACTION   : desfazê-la
-- COMMIT                 : salvá-la
USE Exercicios

SELECT * FROM dCliente

SELECT *
INTO cliente_aux
FROM dCliente

-- 1. Adicionamos novas linhas (registros) na tabela
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
('Maria Julia', 'F', '30/04/1995', '987.654.321-00')

SELECT * FROM cliente_aux
-- 2. Atualizamos linhas (registros) da tabela para MINUSCULO e MAIUSCULO
UPDATE cliente_aux
SET nome_cliente = LOWER(nome_cliente) -- MINUSCULO

UPDATE cliente_aux
SET nome_cliente = UPPER(nome_cliente) -- MAIUSCULO

-- 3. Excluimos linhas (registros) da tabela
DELETE FROM dCliente
WHERE id_cliente = 1

-- Quando executamos as ações acima, não temos controle sobre desfazê-las ou salvá-las
no banco, por exemplo. Mas, quando usamos transações, conseguimos controlar melhor
essas ações, por meio de commits e rollbacks.

-- COMMIT: Comando TCL para efetivar uma transação no banco
-- ROLLBACK: Comando TCL para desfazer uma transação no banco

-- Iniciando uma transação com COMMIT:

```

```

SELECT * FROM cliente_aux

BEGIN TRANSACTION -- BEGIN TRAN(Pode ser abreviado o comando para BEGIN TRAN)
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Maria Ana', 'F', '30/04/1996', '987.654.321-01')

-- Desfazer a transação(BEGIN TRANSACTION)
ROLLBACK TRANSACTION

-- Finalizar a transação(BEGIN TRANSACTION)
COMMIT TRANSACTION

-- *****
BEGIN TRANSACTION
UPDATE cliente_aux
SET cpf = '999.999.999-99'
WHERE id_cliente = 1

-- Desfazer a transação(BEGIN TRANSACTION)
ROLLBACK TRANSACTION

-- Finalizar a transação(BEGIN TRANSACTION)
COMMIT TRANSACTION

-- *****
-- 4. Nomeando uma transação
BEGIN TRANSACTION T1
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Naldo Reis', 'M', '10/02/1992', '412.889.311-90')

-- Commitando uma transação
COMMIT TRANSACTION T1

-- *****
-- 5. Commit e Rollback condicionais
-- Você deve inserir a cliente Ruth Campos no banco de dados. Se esse nome já existir,
desfaça a transação. Se não existir, salve a transação.

SELECT * FROM cliente_aux

DECLARE @contador INT

BEGIN TRANSACTION T1
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Ruth Campos', 'F', '23/03/1992', '324.731.903-89')

SELECT @contador = COUNT(*) FROM cliente_aux WHERE nome_cliente = 'Ruth Campos'

IF @contador = 1
    BEGIN
        COMMIT TRANSACTION T1
        PRINT 'Ruth Campos cadastrada com sucesso.'
    END
ELSE
    BEGIN
        ROLLBACK TRANSACTION T1
        PRINT 'Ruth Campos já foi cadastrada na tabela. Insert abortado.'
    END

SELECT * FROM cliente_aux

-- *****
-- 6. Tratamento de Erros em Transações
BEGIN TRY
    BEGIN TRANSACTION T1

```

```

        UPDATE cliente_aux
        -- SET data_de_nascimento = '15 de março de 1992' -- Erro de tipo
        SET data_de_nascimento = '15/03/1992'
        WHERE id_cliente = 4
    COMMIT TRANSACTION T1
    PRINT 'Data atualizada com sucesso'
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION T1
    PRINT 'Data cadastrada inválida'
END CATCH

SELECT * FROM cliente_aux

-- *****
-- 7. Transações aninhadas (Rollback e Commit)
BEGIN TRANSACTION T1 -- Abriu a transação T1
    PRINT @@TRANCOUNT -- Verifica se tem transação aberta
    BEGIN TRANSACTION T2
        PRINT @@TRANCOUNT -- Verifica se tem transação aberta
        COMMIT TRANSACTION T2 -- Concluiu a transação
        PRINT @@TRANCOUNT -- Verifica se tem transação aberta
    COMMIT TRANSACTION T1 -- Concluiu a transação

    PRINT @@TRANCOUNT -- Verifica se tem transação aberta

ROLLBACK

-- *****
-- Transações aninhadas (Rollback e Commit)
DROP TABLE cliente_aux
SELECT *
INTO cliente_aux
FROM dCliente

SELECT * FROM cliente_aux

BEGIN TRAN T1 -- Início transação 1
    UPDATE cliente_aux
    SET cpf = '999.999.999-99'
    WHERE id_cliente = 1
    SELECT * FROM cliente_aux

    BEGIN TRAN T2 -- Início transação 2
        UPDATE cliente_aux
        SET cpf = '111.111.111-11'
        WHERE id_cliente = 2
        SELECT * FROM cliente_aux
    COMMIT TRAN T2 -- Commit transação 2

    SELECT * FROM cliente_aux

ROLLBACK TRAN T1 -- Rollback transação 1

SELECT * FROM cliente_aux

PRINT @@TRANCOUNT

-- *****
-- 8. Savepoints
DROP TABLE cliente_aux
SELECT *
INTO cliente_aux
FROM dCliente

```

```

BEGIN TRAN T1

    SAVE TRAN SV1

    UPDATE cliente_aux
    SET cpf = '999.999.999-99'
    WHERE id_cliente = 1

    SELECT * FROM cliente_aux

BEGIN TRAN T2
    SAVE TRAN SV2

    UPDATE cliente_aux
    SET cpf = '111.111.111-11'
    WHERE id_cliente = 2

    SELECT * FROM cliente_aux

COMMIT TRAN T2

    SELECT * FROM cliente_aux

ROLLBACK TRAN SV1

SELECT * FROM cliente_aux

```

/*

MÓDULO 22 – EXERCÍCIOS TRANSACTIONS

1. Crie uma tabela chamada Carro com os dados abaixo.
Obs: não se preocupe com constraints, pode criar uma tabela simples.

*/

```

DROP DATABASE AlugaFacil
CREATE DATABASE AlugaFacil
USE AlugaFacil

SELECT * FROM Carro

CREATE TABLE Carro(
    id_carro INT IDENTITY(1, 1),
    placa VARCHAR(100) NOT NULL,
    modelo VARCHAR(100) NOT NULL,
    tipo VARCHAR(100) NOT NULL,
    CONSTRAINT carro_id_carro_pk PRIMARY KEY(id_carro),
    CONSTRAINT carro_placa_un UNIQUE(placa),
    CONSTRAINT carro_tipo_ck CHECK(tipo IN('Hatch', 'SUV', 'Sedan'))
)

INSERT INTO Carro(placa, modelo, tipo) VALUES
    ('ABC-123', 'Hyundai HB20', 'Sedan'),
    ('DAS-1412', 'Fiat Cronos', 'Sedan'),
    ('JHG-3902', 'Hyundai HB20', 'Hatch'),
    ('IPW-9018', 'Citroen C4', 'SUV'),
    ('JKR-8891', 'Nissa Kicks', 'SUV'),
    ('TRF-5934', 'Chevrolet Onix Joy', 'Sedan')

SELECT * FROM Carro

```

/*

2. Execute as seguintes transações no banco de dados, sempre na tabela Carro. Lembre-se de dar um COMMIT para efetivar cada uma das transações.

a) Inserir uma nova linha com os seguintes valores:

id_carro = 6

placa = CDR-0090

```

modelo = Fiat Argo
tipo = Hatch
*/
BEGIN TRANSACTION
INSERT INTO Carro(placa, modelo, tipo)
VALUES ('DFG-124', 'Hyundai HB30', 'Hatch')
COMMIT

SELECT * FROM Carro
/*
b) Atualizar o tipo do carro de id = 1 de Hatch para Sedan.
*/
BEGIN TRANSACTION
UPDATE Carro
SET tipo = 'Hatch'
WHERE id_carro = 1
COMMIT

SELECT * FROM Carro

/*
c) Deletar a linha referente ao carro de id = 6.
*/
BEGIN TRANSACTION
DELETE FROM Carro
WHERE id_carro = 6
COMMIT

SELECT * FROM Carro

```

-- Mod - 23 - Functions

```

-- FUNÇÕES: São encontradas dentro do BD / Programação / Funções (várias Funções)
-- Exemplo: Na coluna da esquerda clique no BD Exercicios e siga o caminho
acima.
-- 1. O que é uma Function?
-- Uma function é um conjunto de comandos que executam ações e retorna um valor
escalar. As functions ajudam a simplificar um código. Por exemplo, se você tem um
cálculo complexo que aparece diversas vezes no seu código, em vez de repetir várias
vezes aquela série de comandos, você pode simplesmente criar uma função e reaproveitá-
la sempre que precisar.
-- O próprio SQL tem diversas funções prontas e até agora, já vimos vários exemplos de
funções deste tipo, como funções de data, texto, e assim vai.
-- Podemos visualizar as funções do sistema na pasta Programação > Funções > Funções
do Sistema

-- 2. Como criar e utilizar uma Function
-- Imagine que você queira fazer uma formatação diferenciada na coluna
data_de_nascimento, utilizando a função DATENAME.
USE Exercicios

SELECT * FROM dCliente
-- SQL sem Função
SELECT
    nome_cliente,
    data_de_nascimento,
    DATENAME(DW, data_de_nascimento) + ', ' +
    DATENAME(D, data_de_nascimento) + ' de ' +
    DATENAME(M, data_de_nascimento) + ' de ' +
    DATENAME(YY, data_de_nascimento)
FROM
    dCliente

-- Criando uma função para formatação de data usando a DATENAME

```

```

CREATE FUNCTION fnDataCompleta(@data AS DATE)
RETURNS VARCHAR(MAX)
AS
BEGIN
    RETURN DATENAME(DW, @Data) + ', ' +
           DATENAME(D, @Data) + ' de ' +
           DATENAME(M, @Data) + ' de ' +
           DATENAME(Y, @Data)
END

-- Atualizar BD (F5) em Funções com Valor Escalar
-- Executar SQL com a Função anterior criada
SELECT
    nome_cliente,
    data_de_nascimento,
    [dbo].[fnDataCompleta](data_de_nascimento)
FROM
    dCliente

-- *****
-- 3. Alterando e excluindo uma function
CREATE OR ALTER FUNCTION fnDataCompleta(@data AS DATE)
RETURNS VARCHAR(MAX)
AS
BEGIN
    RETURN DATENAME(DW, @Data) + ', ' +
           DATENAME(D, @Data) + ' de ' +
           DATENAME(M, @Data) + ' de ' +
           DATENAME(Y, @Data) + ' - ' +
           CASE
               WHEN MONTH(@Data) <= 6 THEN
                   '(1º Semestre)'
               ELSE
                   '(2º Semestre)'
           END
END

-- Consultar
SELECT
    nome_cliente,
    data_de_nascimento,
    [dbo].[fnDataCompleta](data_de_nascimento)
FROM
    dCliente

-- *****
-- 4. Criando funções complexas

-- Crie uma função para retornar o primeiro nome de cada gerente
SELECT * FROM dGerente

INSERT INTO dGerente(nome_gerente, data_contratacao, salario) VALUES
    ('João', '10/01/2019', 3100)

SELECT
    nome_gerente,
    dbo.fnPrimeiroNome(nome_gerente) AS primeiro_nome
FROM dGerente

CREATE OR ALTER FUNCTION fnPrimeiroNome(@nomeCompleto AS VARCHAR(MAX))
RETURNS VARCHAR(MAX)
AS
BEGIN
    DECLARE @posicaoEspaco AS INT
    DECLARE @resposta AS VARCHAR(MAX)

```

```

SET @posicaoEspaco = CHARINDEX(' ', @nomeCompleto)

IF @posicaoEspaco = 0
    SET @resposta = @nomeCompleto
ELSE
    SET @resposta = LEFT(@nomeCompleto, @posicaoEspaco - 1)

RETURN @resposta
END

SELECT
    nome_gerente,
    dbo.fnPrimeiroNome(nome_gerente) AS primeiro_nome
FROM dGerente

```

/*

MODULO 23 – EXERCÍCIOS – Functions

Todas as functions dos exercícios a seguir devem ser criadas para manipulação de dados do banco de dados ContosoRetailDW.

1. Crie uma Function que calcule o tempo (em anos) entre duas datas. Essa function deve receber dois argumentos: data_inicial e data_final. Caso a data_final não seja informada, a function deve automaticamente considerar a data atual do sistema. Essa function será usada para calcular o tempo de casa de cada funcionário.

Obs: a função DATEDIFF não é suficiente para resolver este problema.

*/

```

USE ContosoRetailDW
SELECT * FROM DimEmployee

```

```

SELECT
    FirstName,
    HireDate,
    EndDate,
    DATEDIFF(YEAR, HireDate, EndDate)
FROM DimEmployee

```

```

CREATE OR ALTER FUNCTION CalculaDiferencaDatas(@data_inicial DATE, @data_final DATE)
RETURNS INT
AS
BEGIN

```

```

    IF @data_final IS NULL SET @data_final = GETDATE()
    RETURN DATEDIFF(YEAR, @data_inicial, @data_final)

```

```

END

```

-- Atualizar

```

SELECT
    FirstName,
    HireDate,
    EndDate,
    DATEDIFF(YEAR, HireDate, EndDate),
    dbo.CalculaDiferencaDatas(HireDate, EndDate)
FROM DimEmployee

```

/*

2. Crie uma function que calcula a bonificação de cada funcionário (5% a mais em relação ao BaseRate). Porém, tome cuidado! Nem todos os funcionários deverão receber bônus...

*/

```

SELECT * FROM DimEmployee

```

```

CREATE OR ALTER FUNCTION CalculaBonus (@salario FLOAT, @status VARCHAR(100),
@percentual FLOAT)
RETURNS FLOAT
AS
BEGIN
    DECLARE @bonus AS FLOAT

```

```

        IF @status = 'Current'
            SET @bonus = @salario * @percentual
        ELSE
            SET @bonus = 0

        RETURN @bonus
END
-- Atualizar
SELECT
    FirstName,
    BaseRate,
    Status,
    dbo.CalculaBonus(BaseRate, status, 0.5)
FROM DimEmployee

/*
3. Crie uma Function que retorna uma tabela. Esta function deve receber como parâmetro
o gênero do cliente e retornar todos os clientes que são do gênero informado na
function.
Observe que esta function será utilizada particularmente com a tabela DimCustomer.
*/
SELECT * FROM DimCustomer

CREATE OR ALTER FUNCTION select_genero(@genero VARCHAR(100))
RETURNS TABLE
AS
RETURN (SELECT * FROM DimCustomer WHERE gender = @genero)
-- Atualizar
SELECT * FROM dbo.select_genero('F')

/*
4. Crie uma Function que retorna uma tabela resumo com o total de produtos por cores.
Sua function deve receber 1 argumento, onde será possível especificar de qual marca
você deseja o resumo.
*/
CREATE OR ALTER FUNCTION analisa_cores(@marca VARCHAR(100))
RETURNS TABLE
AS
RETURN (SELECT ColorName, COUNT(*) Total
        FROM DimProduct
        WHERE BrandName = @marca
        GROUP BY ColorName)
-- Atualizar
SELECT * FROM dbo.analisa_cores('Contoso')

```

-- MOD - 24 - PROCEDURES

```

-- 1. O que é uma Procedure
-- É um bloco de código que possui um nome e pode ser armazenado no banco de dados.
-- Ele pode incluir uma série de comandos SQL para executar alguma tarefa.

-- 2. Por que usar uma Procedure
-- Procedures são usadas para fazer tarefas repetitivas que não são possíveis em
queries do SQL ou que dariam muito trabalho.
-- Pode incluir estruturas de controle e comandos.

-- 3. Tipos de Procedure
-- Uma Procedure pode ou não aceitar parâmetros de entrada

-- Procedures Com Parâmetros
-- Procedures Sem Parâmetros

-- *****

```

-- 2. Criando uma Procedure Sem Parâmetros

-- Exemplo 1. Crie uma Procedure que executa um SELECT simples (sem parâmetros).

USE Exercicios

CREATE PROCEDURE prOrdenaGerentes

AS

BEGIN

SELECT

id_gerente,
nome_gerente,
salario

FROM dGerente

ORDER BY salario DESC

END

-- Atualizar

EXECUTE prOrdenaGerentes

-- *****

-- 3. Criando uma Procedure Com 1 Parâmetro

-- Exemplo 2. Crie uma Procedure que executa um SELECT que recebe um parâmetro de entrada para filtrar a tabela dClientes de acordo com o gênero informado.

USE Exercicios

SELECT * FROM dCliente

CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX))

AS

BEGIN

SELECT

nome_cliente,
genero,
data_de_nascimento,
cpf

FROM dCliente

WHERE genero = @gen

END

-- Atualizar

EXECUTE prListaClientes 'M'

-- *****

-- 4. Criando uma Procedure com mais de 1 parâmetro

-- Exemplo 3. Crie uma Procedure que executa um SELECT que recebe um parâmetro de entrada para filtrar a tabela dClientes de acordo com o gênero informado e ano de nascimento informado.

USE Exercicios

CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX), @ano INT)

AS

BEGIN

SELECT

nome_cliente,
genero,
data_de_nascimento,
cpf

FROM dCliente

WHERE genero = @gen AND YEAR(data_de_nascimento) = @ano

END

EXECUTE prListaClientes 'M', 1989

-- *****

-- 5. Criando uma Procedure com Parâmetro Default

-- Exemplo 3. Crie uma Procedure que executa um SELECT que recebe um parâmetro de entrada para filtrar a tabela dClientes de acordo com o gênero informado e ano de nascimento informado.

USE Exercicios

```
CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX)='M', @ano INT)
AS
BEGIN
    SELECT
        nome_cliente,
        genero,
        data_de_nascimento,
        cpf
    FROM dCliente
    WHERE genero = @gen AND YEAR(data_de_nascimento) = @ano
END
```

EXECUTE prListaClientes @gen='F', @ano=1989

-- 6. Criando uma Procedure mais Complexa para Cadastro de Contratos

-- Exemplo: Crie uma procedure para cadastrar uma nova assinatura de um contrato na tabela fContratos (com parâmetros).

-- Gerente: Lucas Sampaio

-- Cliente: Gustavo Barbosa

-- Valor do Contrato: 5000

-- 1º Passo: Definir as variáveis a serem utilizadas.

-- 2º Passo: Armazenar o valor de id_gerente de acordo com o gerente associado

-- 3º Passo: Armazenar o valor de id_cliente de acordo com o nome do cliente

-- 4º Passo: Armazenar a data da assinatura como sendo a data atual do sistema

-- 5º Passo: Utilizar o INSERT INTO para inserir os dados na tabela fContratos

USE Exercicios

```
SELECT * FROM dCliente
SELECT * FROM dGerente
SELECT * FROM fContratos
```

```
CREATE OR ALTER PROCEDURE prRegistraContrato(@gerente VARCHAR(MAX), @cliente
VARCHAR(MAX), @valor FLOAT)
AS
BEGIN
    DECLARE
        @vIdGerente INT,
        @vIdCliente INT

    SELECT
        @vIdGerente = id_gerente
    FROM dGerente
    WHERE nome_gerente = @gerente

    SELECT
        @vIdCliente = id_cliente
    FROM dCliente
    WHERE nome_cliente = @cliente

    INSERT INTO fContratos(data_assinatura, id_cliente, id_gerente, valor_contrato)
VALUES
    (GETDATE(), @vIdCliente, @vIdGerente, @valor)

    PRINT 'Contrato registrado com sucesso!'
END
```

```
EXECUTE prRegistraContrato @gerente='Lucas Sampaio', @cliente='Gustavo Barbosa',  
@valor=5000
```

```
SELECT * FROM fContratos  
SELECT * FROM dCliente  
SELECT * FROM dGerente
```

```
-- *****  
-- 7. Excluindo uma Procedure  
DROP PROCEDURE prRegistraContrato  
  
-- *****  
-- 8. Functions vs. Procedures  
/*
```

Temos abaixo uma lista de principais diferenças entre Functions e Procedures.

Diferença 1.

- Procedures são usadas para executar um processo, uma sequência de comandos e blocos SQL.
- Functions são usadas para fazer cálculos

Diferença 2.

- Procedures não podem ser 'chamadas' dentro da cláusula SELECT
- Functions podem ser 'chamadas' dentro da cláusula SELECT (desde que não contenham comandos SELECT)

Diferença 3.

- Procedures não precisam retornar nenhum valor
 - Functions precisam retornar algum valor
- ```
*/
```

```
/*
```

## MOD – 24 – EXERCICIOS – PROCEDURES

Obs. As Procedures dos exercícios 1 a 3 serão executadas nas tabelas originais do banco de dados ContosoRetailDW.

1. Crie uma Procedure que resume o total de produtos por nome da categoria. Essa Procedure deve solicitar ao usuário qual marca deve ser considerada na análise.

```
*/
```

```
USE ContosoRetailDW
```

```
SELECT TOP(5) * FROM DimProduct
SELECT TOP(5) * FROM DimProductSubcategory
SELECT TOP(5) * FROM DimProductCategory
```

```
CREATE OR ALTER PROCEDURE analisa_produtos(@marca VARCHAR(100))
AS
BEGIN
 SELECT
 c.ProductCategoryName,
 COUNT(*)
 FROM DimProduct p
 INNER JOIN DimProductSubcategory s ON p.ProductSubcategoryKey =
s.ProductSubcategoryKey
 INNER JOIN DimProductCategory c ON s.ProductCategoryKey =
c.ProductCategoryKey
 WHERE p.BrandName = @marca
 GROUP BY c.ProductCategoryName
END
```

```
EXECUTE analisa_produtos 'Contoso'
```

```
EXECUTE analisa_produtos 'Litware'
```

```

/*
2. Crie uma Procedure que lista os top N clientes de acordo com a data de primeira
compra. O valor de N deve ser um parâmetro de entrada da sua Procedure.
*/

```

```

USE ContosoRetailDW

```

```

SELECT TOP(5) * FROM DimCustomer
CREATE OR ALTER PROCEDURE lista_top_clientes(@topn INT)
AS
BEGIN
 SELECT TOP(@topn)
 FirstName,
 EmailAddress,
 DateFirstPurchase
 FROM DimCustomer
 WHERE CustomerType = 'Person'
 ORDER BY DatefirstPurchase
END

```

```

EXECUTE lista_top_clientes 100

```

```

/*
3. Crie uma Procedure que recebe 2 argumentos: MÊS (de 1 a 12) e ANO (1996 a 2003).
Sua Procedure deve listar todos os funcionários que foram contratados no mês/ano
informado.
*/

```

```

SE ContosoRetailDW

```

```

CREATE OR ALTER PROCEDURE lista_funcionarios(@mes INT, @ano INT)
AS
BEGIN
 SELECT *
 FROM DimEmployee
 WHERE DATEPART(MM, HireDate) = @mes AND DATEPART(YYYY, HireDate) = @ano
 ORDER BY HireDate
END

```

```

EXECUTE lista_funcionarios 1, 2000

```

```

/*
Obs. Para os exercícios 4, 5 e 6, utilize os códigos abaixo.
*/

```

```

DROP DATABASE AlugaFacil

```

```

CREATE DATABASE AlugaFacil

```

```

USE AlugaFacil

```

```

CREATE TABLE Carro(
 id_carro INT,
 placa VARCHAR(100) NOT NULL,
 modelo VARCHAR(100) NOT NULL,
 tipo VARCHAR(100) NOT NULL,
 valor FLOAT NOT NULL,
 CONSTRAINT carro_id_carro_pk PRIMARY KEY(id_carro)
)

```

```

INSERT INTO Carro(id_carro, placa, modelo, tipo, valor) VALUES
(1, 'CRU-1111', 'Chevrolet Cruze', 'Sedan', 140000),
(2, 'ARG-2222', 'Fiat Argo', 'Hatch', 80000),
(3, 'COR-3333', 'Toyota Corolla', 'Sedan', 170000),
(4, 'TIG-4444', 'Caoa Chery Tiggo', 'SUV', 190000)

```

```

SELECT * FROM Carro
-- 4. Crie uma Procedure que insere uma nova linha na tabela Carro. Essa nova linha
deve conter os seguintes dados:
-- id = 5
-- placa = GOL-5555
-- modelo = Volkswagen Gol
-- tipo = Hatch
-- valor = 80000

```

```

CREATE OR ALTER PROCEDURE cadastro_carro
AS
BEGIN
 BEGIN TRANSACTION
 INSERT INTO Carro(id_carro, placa, modelo, tipo, valor)
 VALUES (7, 'GOL-5555', 'Volkswagem Gol', 'Hatch', 80000)
 COMMIT
END

```

```
EXECUTE cadastro_carro
```

```
SELECT * FROM Carro
```

```

-- 5. Crie uma Procedure que altera o valor de venda de um carro. A Procedure deve
receber como parâmetros o id_carro e o novo valor.

```

```

CREATE OR ALTER PROCEDURE altera_valor(@id INT, @novo_valor FLOAT)
AS
BEGIN
 BEGIN TRANSACTION
 UPDATE Carro
 SET valor = @novo_valor
 WHERE id_carro = @id
 COMMIT
END

```

```
SELECT * FROM Carro
```

```
EXECUTE altera_valor 1, 200000
```

```
SELECT * FROM Carro
```

```

-- 6. Crie uma Procedure que exclui um carro a partir do id informado.

```

```

CREATE OR ALTER PROCEDURE deleta_carro(@id INT)
AS
BEGIN
 BEGIN TRANSACTION
 DELETE FROM Carro
 WHERE id_carro = @id
 COMMIT
END

```

```
SELECT * FROM Carro
```

```
EXECUTE deleta_carro 1
```

```
SELECT * FROM Carro
```

```
/*
```

## MOD - 25 - Triggers DML

```

-- Um Trigger é um gatilho que será disparado automaticamente quando acontecer um
evento.
-- Triggers podem ser disparadas por eventos DDL (CREATE, ALTER, DROP) e DML (INSERT,
UPDATE, DELETE).

```

```
-- Triggers DML
-- 1. Uma Trigger DML é disparada quando um comando INSERT, UPDATE ou DELETE é
executado sobre uma tabela ou view.
-- 2. Na hora de criar uma trigger, podemos definir alguns elementos, podemos definir
se ela será do tipo AFTER ou INSTEAD OF
*/
```

```
-- 2. Criando uma Trigger DML simples
-- Crie uma Trigger que seja disparada APÓS um evento INSERT, UPDATE, DELETE seja
executado na tabela dCliente
USE Exercicios
```

```
CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
 PRINT 'Os dados da tabela dCliente foram alterados!'
END

SELECT * FROM dCliente

INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
('Zacarias Neto', 'M', '13/02/1999', '139.543.189-00')

SELECT * FROM dCliente

UPDATE dCliente
SET cpf = '130.451.892-10'
WHERE id_cliente = 12

SELECT * FROM dCliente

DELETE FROM dCliente
WHERE id_cliente = 12

SELECT * FROM dCliente

-- 3. Tabelas INSERTED e DELETED
-- Vamos alterar a trigger anterior para deixá-la mais clara.

CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
 SELECT * FROM INSERTED
 -- PRINT 'Os dados da tabela dCliente foram alterados!'

 -- SELECT * FROM DELETED
 -- PRINT 'Os dados da tabela dCliente foram alterados!'

 -- SELECT * FROM UPDATED -- O evento UPDATED não existe
END

INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
('Zacarias Neto', 'M', '13/02/1999', '139.543.189-00')

SELECT * FROM dCliente

DELETE FROM dCliente
WHERE id_cliente = 13

SELECT * FROM dCliente
```

```

UPDATE dCliente
SET cpf = '130.451.892-10'
WHERE id_cliente = 13

SELECT * FROM dCliente

-- 04 - Identificando na Trigger o Evento DML Relacionado
CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
 IF EXISTS (SELECT * FROM INSERTED) AND EXISTS (SELECT * FROM DELETED)
 PRINT 'Dados Foram Atualizados na Tabela'
 ELSE IF EXISTS (SELECT * FROM INSERTED)
 PRINT 'Dados Foram Inseridos na Tabela'
 ELSE IF EXISTS (SELECT * FROM DELETED)
 PRINT 'Dados Foram Excluidos na Tabela'
END

INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
('Eleonora Batista', 'F', '13/02/2003', '453.543.189-00')

SELECT * FROM dCliente

DELETE FROM dCliente
WHERE id_cliente = 14

SELECT * FROM dCliente

UPDATE dCliente
SET cpf = '678.333.111-13'
WHERE id_cliente = 1

SELECT * FROM dCliente

-- Se a trigger for de INSERT, a INSERTED terá registros e a DELETED ficará vazia
-- Se a trigger for de DELETE, a INSERTED ficará vazia e a DELETED terá registros
-- Se a trigger for de UPDATE, a INSERTED e a DELETED terão registros

-- Criando uma Trigger para Controle de Permissão de Cadastro - INSTEAD OF

-- Crie uma Trigger que seja disparada sempre EM VEZ de um INSERT, UPDATE e DELETE que
for executado na tabela dCliente. O que deve acontecer: se o dia de cadastro for
sábado ou domingo, não pode deixar alterar pois está fora do horário comercial.

SELECT FORMAT(GETDATE(), 'dddd') -- Dia atual

CREATE OR ALTER TRIGGER tgControleRegistros
ON dCliente
INSTEAD OF INSERT
AS
BEGIN
 IF FORMAT(GETDATE(), 'dddd') IN ('domingo')
 BEGIN
 RAISERROR('O cadastro de clientes só pode ser feito de segunda à sexta',
1, 1)
 ROLLBACK
 END
 ELSE
 BEGIN
 INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf)
 SELECT i.nome_cliente, i.genero, i.data_de_nascimento, i.cpf FROM
INSERTED i
 END
END

```

END

```
INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
('Lucio Dantas', 'M', '13/02/2003', '111.222.333-00')
```

```
SELECT * FROM dCliente
```

```
-- Habilitando uma Trigger DML
```

```
ENABLE TRIGGER tgControleRegistros ON dCliente
```

```
-- Desabilitando uma Trigger DML
```

```
DISABLE TRIGGER tgControleRegistros ON dCliente
```

```
-- Habilitando TODAS as Triggers DML de uma tabela
```

```
ENABLE TRIGGER ALL ON dCliente
```

```
-- Desabilitando TODAS as Triggers DML de uma tabela
```

```
DISABLE TRIGGER ALL ON dCliente
```

```
-- Excluindo uma Trigger DML
```

```
DROP TRIGGER tgControleRegistros;
```

```
/*
```

## MOD - 26 - Triggers DDL

```
-- Um Trigger é um gatilho que será disparado automaticamente quando acontecer um evento.
```

```
-- Triggers podem ser disparadas por eventos DDL (CREATE, ALTER, DROP) e DML (INSERT, UPDATE, DELETE).
```

```
-- Triggers DDL
```

```
-- Uma Trigger DML é disparada quando um comando CREATE, ALTER ou DROP é executado.
```

```
*/
```

```
-- 2. Criando uma Trigger DDL simples
```

```
USE Exercicios
```

```
CREATE OR ALTER TRIGGER tgRecusarTabelas
```

```
ON DATABASE
```

```
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
```

```
AS
```

```
BEGIN
```

```
 PRINT 'Não é permitido criação, alteração ou exclusão de tabelas'
```

```
 ROLLBACK -- Para não concretizar
```

```
END
```

```
-- Executa
```

```
CREATE TABLE teste(ID INT)
```

```
-- *****
```

```
-- Habilitando ou Desabilitando uma Trigger DDL
```

```
-- Desabilitando
```

```
DISABLE TRIGGER tgRecusarTabelas ON DATABASE
```

```
CREATE TABLE teste1(ID INT)
```

```
-- Habilitando
```

```
ENABLE TRIGGER tgRecusarTabelas ON DATABASE
```

```
CREATE TABLE teste2(ID INT)
```

```
DROP TABLE teste2
```

```
-- Habilitando ou Desabilitando TODAS as Triggers DDL de uma database
```

```
DISABLE TRIGGER ALL ON DATABASE
```

```
-- Excluindo uma Trigger DDL
```

```
DROP TRIGGER tgRecusarTabelas ON DATABASE
```

```

/*
MOD - 27 - Pivot Table
*/
USE ContosoRetailDW

-- Operador Pivot

-- 1. O que o Operador Pivot faz
SELECT
 BrandName,
 COUNT(*) AS Total_Produtos
FROM
 DimProduct
GROUP BY BrandName

-- Criando uma Pivot Table
SELECT
 BrandName,
 COUNT(ProductKey) AS Total_Produtos
FROM
 DimProduct
GROUP BY BrandName

-- 1º) O primeiro passo é selecionar os dados que serão usados como base para criação
da Pivot Table

-- 2º) Como não conseguimos aplicar o Pivot diretamente nos dados acima, precisaremos
fazer isso indiretamente

-- 3º) Agora sim podemos aplicar o Pivot, incluindo o cálculo desejado e os nomes das
colunas a serem consideradas

-- Selecionar os departamentos
SELECT DISTINCT
 ', ' + QUOTENAME(TRIM(DepartmentName)) -- Coloca os colchetes
FROM DimEmployee

SELECT * FROM
 (SELECT
 ProductKey,
 BrandName
 FROM
 DimProduct) AS Dados
PIVOT(
 COUNT(ProductKey)
 FOR BrandName
 IN ([Northwind Traders]
 , [Contoso]
 , [Tailspin Toys]
 , [Adventure Works]
 , [Southridge Video]
 , [Wide World Importers]
 , [The Phone Company]
 , [Fabrikam]
 , [Litware]
 , [A. Datum]
 , [Proseware]
)
) AS PivotTable

-- Adicionando Grupos de Linha à Pivot Table

-- Calculando o total de funcionários por departamento

SELECT

```

```

 DepartmentName,
 COUNT(EmployeeKey) Total_Funcionarios
FROM DimEmployee
GROUP BY DepartmentName

```

-- Adicionando mais linhas ao grupo

```

SELECT * FROM
 (SELECT
 EmployeeKey,
 YEAR(HireDate) AS Ano,
 DepartmentName
 FROM DimEmployee) AS Dados
PIVOT(
 COUNT(EmployeeKey)
 FOR DepartmentName
 IN([Tool Design]
 , [Shipping and Receiving]
 , [Sales]
 , [Research and Development]
 , [Quality Assurance]
 , [Purchasing]
 , [Production]
 , [Production Control]
 , [Marketing]
 , [Information Services]
 , [Human Resources]
 , ['Human Resources Contral]
 , [Finance]
 , [Facilities and Maintenance]
 , [Executive]
 , [Engineering]
 , [Document Control])
) AS PivotTable
ORDER BY Ano DESC

```

-- Limitações da Pivot Table

```

SELECT * FROM
 (SELECT
 EmployeeKey,
 YEAR(HireDate) AS Ano,
 DATENAME(MM, HireDate) AS Mes,
 DepartmentName
 FROM
 DimEmployee) AS Dados
PIVOT (
 COUNT(EmployeeKey)
 FOR DepartmentName
 IN ([Document Control]
 , [Engineering]
 , [Executive]
 , [Facilities and Maintenance]
 , [Finance]
 , ['Human Resources Contral]
 , [Human Resources]
 , [Information Services]
 , [Marketing]
 , [Production Control]
 , [Production]
 , [Purchasing]
 , [Quality Assurance]
 , [Research and Development]
 , [Sales]
 , [Shipping and Receiving]
)

```

```

 , [Tool Design])
) AS PivotTable
ORDER BY Ano, Mes

-- Corrigindo a limitação da Pivot Table

DECLARE @NomeColunas NVARCHAR(MAX) = ''
DECLARE @SQL NVARCHAR(MAX) = ''

SELECT @NomeColunas += QUOTENAME(TRIM(DepartmentName)) + ', '
FROM
 (SELECT DISTINCT
 Departmentname
 FROM DimEmployee) AS Aux

SET @NomeColunas = LEFT(@NomeColunas, LEN(@NomeColunas) - 1)

-- PRINT @NomeColunas
-- Transforma em texto a instrução
SET @SQL = 'SELECT * FROM
 (SELECT
 EmployeeKey,
 YEAR(HireDate) AS Ano,
 DATENAME(MM, HireDate) AS Mes,
 DepartmentName
 FROM
 DimEmployee) AS Dados
PIVOT (
 COUNT(EmployeeKey)
 FOR DepartmentName
 IN (' + @NomeColunas + ')
) AS PivotTable
ORDER BY Ano'

-- Até aqui é para vê o @SQL
PRINT @SQL
-- Story Procedure
EXECUTE sp_executesql @SQL

-- *****

DECLARE @NomeColunas2 NVARCHAR(MAX) = ''
DECLARE @SQL2 NVARCHAR(MAX) = ''

SELECT @NomeColunas2 += QUOTENAME(TRIM(DepartmentName)) + ', '
FROM
 (SELECT DISTINCT
 DepartmentName
 FROM DimEmployee) AS Aux

SET @NomeColunas2 = LEFT(@NomeColunas2, LEN(@NomeColunas2) - 1)

PRINT @NomeColunas2

SET @SQL2 =
'SELECT * FROM
 (SELECT
 EmployeeKey,
 YEAR(HireDate) AS Ano,
 DATENAME(MM, HireDate) AS Mes,
 DepartmentName
 FROM
 DimEmployee) AS Dados
PIVOT (
 COUNT(EmployeeKey)

```

```

 FOR DepartmentName
 IN (' + @NomeColunas2 + ')
) AS PivotTable
ORDER BY Ano DESC'

```

```
PRINT @SQL2
```

```
EXECUTE sp_executesql @SQL2
```

```
/*
```

## MOD - 28 - Projeto 01 - Integração SQL SERVER e EXCEL

1º - Baixar o Banco de Dados de Teste

- Pesquisar: Bancos de dados de exemplo do AdventureWorks
- Baixar a Versão: AdventureWorksDW2014.bak

2º - Gravar/Salvar o BD no Local

F:\MeusArq\Cursos\SQL-Impressionador\BD\SQL-MS-AdventureWorks2014

3º - No SSMS

- Clicar em Bancos de Dados(botão direito)
  - Restaurar Bancos de Dados...
    - F:\MeusArq\Cursos\SQL-Impressionador\BD\SQL-MS-AdventureWorks2014
    - ok... ok... ok...

```
*/
```

```

-- #####
-- PROJETO DE INTEGRAÇÃO SQL SERVER e EXCEL
-- #####

```

- 1. Apresentação
- 2. Download Banco de Dados AdventureWorks 2014

```
/*
```

<https://docs.microsoft.com/pt-br/sql/samples/adventureworks-install-configure?view=sql-server-ver16&tabs=ssms>

```
*/
```

- 3. Definindo os indicadores do projeto
- i) Total de Vendas Internet por Categoria do Produto
- ii) Receita Total Internet por Mês do Pedido
- iii) Receita e Custo Total Internet por País
- iv) Total de Vendas Internet por Sexo do Cliente

-- OBS: O ANO DE ANÁLISE SERÁ APENAS 2013 (ANO DO PEDIDO)

USE AdventureWorksDW2014

```

SELECT * FROM FactInternetSales
SELECT * FROM DimProductSubCategory
SELECT * FROM DimSalesTerritory
SELECT * FROM DimProduct

```

-- 4. Definindo as tabelas a serem analisadas

```

-- TABELA 1: FactInternetSales
-- TABELA 2: DimCustomer
-- TABELA 3: DimSalesTerritory
-- TABELA 4: DimProductCategory ***

```

-- \*\*\* Aqui precisaremos fazer um relacionamento em cadeia

-- 5. Definindo as colunas da view VENDAS\_INTERNET

-- VIEW FINAL VENDAS\_INTERNET

-- Colunas:

```

-- SalesOrderNumber (TABELA 1: FactInternetSales)
-- OrderDate (TABELA 1: FactInternetSales)
-- EnglishProductCategoryName (TABELA 4: DimProductCategory)
-- FirstName + LastName (TABELA 2: DimCustomer)
-- Gender (TABELA 2: DimCustomer)
-- SalesTerritoryCountry (TABELA 3: DimSalesTerritory)
-- OrderQuantity (TABELA 1: FactInternetSales)
-- TotalProductCost (TABELA 1: FactInternetSales)
-- SalesAmount (TABELA 1: FactInternetSales)

-- 6. Criando o código da view VENDAS_INTERNET
-- i) Total de Vendas Internet por Categoria do Produto
-- ii) Receita Total Internet por Mês do Pedido
-- iii) Receita e Custo Total Internet por País
-- iv) Total de Vendas Internet por Sexo do Cliente

-- OBS: O ANO DE ANÁLISE SERÁ APENAS 2013 (ANO DO PEDIDO)
USE AdventureWorksDW2014

CREATE OR ALTER VIEW VENDAS_INTERNET AS -- Cria View
SELECT
 fis.SalesOrderNumber AS 'Nº PEDIDO',
 fis.OrderDate AS 'DATA PEDIDO',
 dpc.EnglishProductCategoryName AS 'CATEGORIA PRODUTO',
 dc.FirstName + ' ' + dc.LastName AS 'NOME CLIENTE',
 Gender as 'SEXO',
 SalesTerritoryCountry AS 'PAÍS',
 fis.OrderQuantity AS 'QTD. VENDIDA',
 fis.TotalProductCost AS 'CUSTO VENDA',
 fis.SalesAmount AS 'RECEITA VENDA'
FROM FactInternetSales fis
INNER JOIN DimProduct dp ON fis.ProductKey = dp.ProductKey
 INNER JOIN DimProductSubcategory dps ON dp.ProductSubcategoryKey =
dps.ProductSubcategoryKey
 INNER JOIN DimProductCategory dpc ON dps.ProductCategoryKey =
dpc.ProductCategoryKey
INNER JOIN DimCustomer dc ON fis.CustomerKey = dc.CustomerKey
INNER JOIN DimSalesTerritory dst ON fis.SalesTerritoryKey = dst.SalesTerritoryKey
WHERE YEAR(OrderDate) = 2013

-- Atualize o BD e vá em Tabelas/Exibições e estará a View VENDAS_INTERNET
SELECT * FROM VENDAS_INTERNET

-- *****
-- ABRA O EXCELL
-- FAZER A CONEXÃO ENTRE EXCEL E SQL SERVER SERÁ UTILIZADO O POWER QUERIE
-- Na Aba Dados/Obter Dados/Do banco de Dados/Do Banco de Dados SQL Server
-- Copie o nome do BD: J2CM\SQLEXPRESS
-- Copie o nome do BD: AdventureWorksDW2014
-- Clicar em OK
-- Apos Carregar a tabela no Excel qualquer alteração na VIEW deve se feita no
Excel e Design de Tabelas/Atualizar

-- Alterando o banco de dados e atualizando no Excel

-- *****
-- Trabalhando a instrução SQL direto do EXCEL
-- Copie a instrução SQL somente
-- Na Aba Dados/Obter Dados/Do banco de Dados/Do Banco de Dados SQL Server
-- Copie o nome do BD: J2CM\SQLEXPRESS
-- Copie o nome do BD: AdventureWorksDW2014
-- Clique em Opções Avançadas
-- Instrução SQL, cola a instrução SQL

SELECT
 fis.SalesOrderNumber AS 'Nº PEDIDO',

```

```

 fis.OrderDate AS 'DATA PEDIDO',
 dpc.EnglishProductCategoryName AS 'CATEGORIA PRODUTO',
 dc.FirstName + ' ' + dc.LastName AS 'NOME CLIENTE',
 Gender as 'SEXO',
 SalesTerritoryCountry AS 'PAÍS',
 fis.OrderQuantity AS 'QTD. VENDIDA',
 fis.TotalProductCost AS 'CUSTO VENDA',
 fis.SalesAmount AS 'RECEITA VENDA'
FROM FactInternetSales fis
INNER JOIN DimProduct dp ON fis.ProductKey = dp.ProductKey
 INNER JOIN DimProductSubcategory dps ON dp.ProductSubcategoryKey =
dps.ProductSubcategoryKey
 INNER JOIN DimProductCategory dpc ON dps.ProductCategoryKey =
dpc.ProductCategoryKey
INNER JOIN DimCustomer dc ON fis.CustomerKey = dc.CustomerKey
INNER JOIN DimSalesTerritory dst ON fis.SalesTerritoryKey = dst.SalesTerritoryKey
WHERE YEAR(OrderDate) = 2013
 -- Clicar em OK
 -- Aparecera os dados da consulta via instrução, clique em Carregar.
 -- Apos Carregar a tabela no Excel qualquer alteração na VIEW deve se feita no
Excel clicando na tabela e Consulta/Editar

 -- Para Alterar a consulta clique na Consulta01/, abre janela,
Consulta01/EDITAR,
 -- ETAPAS APLICADAS / Fontes / Figura da engrenagem
 -- Clique na figura da Engrenagem
 -- Aparecerá a Instrução SQL para ser Alterada.
 -- Clicar em OK
 -- Para atualizar a Tabela é só clicar em Fechar e Carregar e será
atuliada.

-- *****
BEGIN TRANSACTION T1

 UPDATE FactInternetSales
 SET OrderQuantity = 20
 WHERE ProductKey = 361 -- Categoria Bike

COMMIT TRANSACTION T1

SELECT * FROM FactInternetSales

-- 12 - Tratamentos Basicos no Power Query
-- No Excel arquivo Pasta02.xls na aba consulta/editar, abrirar o Power
Query,
-- clique com o botão direito sobre a coluna sexo opção Substituir Valores -
F por Feminino - em opções avançadas Coincidir Conteudo da Celular inteira = OK.

/*
28. [SQL Server] Projeto 1 - Integração SQL Server e Excel28. [SQL Server] Projeto 1 -
Integração SQL Server e Excel
Obs.: Os itens de 11 a 23 são sobre EXCEL e POWER QUERY
11. Alterando o banco e atualizando a Visualização da Tabela
12. Tratamentos Básicos no Power Query
13. Criando a Tabela Dinâmica VENDAS POR CATEGORIA
14. Criando a Tabela Dinâmica VENDAS POR MÊS
15. Criando a Tabela Dinâmica VENDAS POR GÊNERO
16. Criando a Tabela Dinâmica RECEITA e CUSTO POR PAÍS
17. Criando o Gráfico VENDAS POR CATEGORIA
18. Criando o Gráfico VENDAS POR GÊNERO
19. Criando o Gráfico VENDAS POR MÊS
20. Criando o Gráfico RECEITA e CUSTO POR PAÍS
21. Finalizando o Relatório no Excel
22. Alterando o Banco de Dados e Atualizando no Excel
23. Salvando o Arquivo e Finalizando o Projeto */

```